TypeScript 教程

阮一峰 · 网道（WangDoc.com）开源教程

原文：wangdoc.com/typescript · github.com/wangdoc/typescript-tutorial

许可：CC BY-SA 4.0 · 2026-07 版本 · 2026-10-13 排版

目录

TypeScript 语言简介
TypeScript 基本用法
any 类型，unknown 类型，never 类型
TypeScript 的类型系统
TypeScript 的数组类型
TypeScript 的元组类型
TypeScript 的 symbol 类型
TypeScript 的函数类型
TypeScript 的对象类型
TypeScript 的 interface 接口
TypeScript 的 class 类型
TypeScript 泛型
TypeScript 的 Enum 类型
TypeScript 的类型断言
TypeScript 模块
TypeScript namespace
TypeScript 装饰器
装饰器（旧语法）
declare 关键字
d.ts 类型声明文件
TypeScript 类型运算符
TypeScript 的类型映射
TypeScript 类型工具
TypeScript 的注释指令
tsconfig.json
tsc 命令行编译器

TypeScript 语言简介

概述

TypeScript（简称 TS）是微软公司开发的一种基于 JavaScript （简称 JS）语言的编程语言。

它的目的并不是创造一种全新语言，而是增强 JavaScript 的功能，使其更适合多人合作的企业级项目。

TypeScript 可以看成是 JavaScript 的超集（superset），即它继承了后者的全部语法，所有 JavaScript 脚本都可以当作 TypeScript 脚本（但是可能会报错），此外它再增加了一些自己的语法。

TypeScript 对 JavaScript 添加的最主要部分，就是一个独立的类型系统。

类型的概念

类型（type）指的是一组具有相同特征的值。如果两个值具有某种共同的特征，就可以说，它们属于同一种类型。

举例来说，`123`和`456`这两个值，共同特征是都能进行数值运算，所以都属于“数值”（number）这个类型。

一旦确定某个值的类型，就意味着，这个值具有该类型的所有特征，可以进行该类型的所有运算。凡是适用该类型的地方，都可以使用这个值；凡是不适用该类型的地方，使用这个值都会报错。

可以这样理解，**类型是人为添加的一种编程约束和用法提示**。 主要目的是在软件开发过程中，为编译器和开发工具提供更多的验证和帮助，帮助提高代码质量，减少错误。

下面是一段简单的 TypeScript 代码，演示一下类型系统的作用。

```
function addOne(n:number) {
  return n + 1;
}
```

上面示例中，函数`addOne()`有一个参数`n`，类型为数值（number），表示这个位置只能使用数值，传入其他类型的值就会报错。

```
addOne('hello') // 报错
```

上面示例中，函数`addOne()`传入了一个字符串`hello`，TypeScript 发现类型不对，就报错了，指出这个位置只能传入数值，不能传入字符串。

JavaScript 语言就没有这个功能，不会检查类型对不对。开发阶段很可能发现不了这个问题，代码也许就会原样发布，导致用户在使用时遇到错误。

作为比较，TypeScript 是在开发阶段报错，这样有利于提早发现错误，避免使用时报错。另一方面，函数定义里面加入类型，具有提示作用，可以告诉开发者这个函数怎么用。

动态类型与静态类型

前面说了，TypeScript 的主要功能是为 JavaScript 添加类型系统。大家可能知道，JavaScript 语言本身就有一套自己的类型系统，比如数值`123`和字符串`Hello`。

但是，JavaScript 的类型系统非常弱，而且没有使用限制，运算符可以接受各种类型的值。在语法上，JavaScript 属于动态类型语言。

请看下面的 JavaScript 代码。

```
// 例一
let x = 1;
x = 'hello';

// 例二
let y = { foo: 1 };
delete y.foo;
y.bar = 2;
```

上面的例一，变量`x`声明时，值的类型是数值，但是后面可以改成字符串。所以，无法提前知道变量的类型是什么，也就是说，变量的类型是动态的。

上面的例二，变量`y`是一个对象，有一个属性`foo`，但是这个属性是可以删掉的，并且还可以新增其他属性。所以，对象有什么属性，这个属性还在不在，也是动态的，没法提前知道。

正是因为存在这些动态变化，所以 JavaScript 的类型系统是动态的，不具有很强的约束性。这对于提前发现代码错误，非常不利。

TypeScript 引入了一个更强大、更严格的类型系统，属于静态类型语言。

上面的代码在 TypeScript 里面都会报错。

```
// 例一
let x = 1;
x = 'hello'; // 报错

// 例二
let y = { foo: 1 };
delete y.foo; // 报错
y.bar = 2; // 报错
```

上面示例中，例一的报错是因为变量赋值时，TypeScript 已经推断确定了类型，后面就不允许再赋值为其他类型的值，即变量的类型是静态的。例二的报错是因为对象的属性也是静态的，不允许随意增删。

TypeScript 的作用，就是为 JavaScript 引入这种静态类型特征。

静态类型的优点

静态类型有很多好处，这也是 TypeScript 想要达到的目的。

（1）有利于代码的静态分析。

有了静态类型，不必运行代码，就可以确定变量的类型，从而推断代码有没有错误。这就叫做代码的静态分析。

这对于大型项目非常重要，单单在开发阶段运行静态检查，就可以发现很多问题，避免交付有问题的代码，大大降低了线上风险。

（2）有利于发现错误。

由于每个值、每个变量、每个运算符都有严格的类型约束，TypeScript 就能轻松发现拼写错误、语义错误和方法调用错误，节省程序员的时间。

```
let obj = { message: '' };
console.log(obj.messege); // 报错
```

上面示例中，不小心把`message`拼错了，写成`messege`。TypeScript 就会报错，指出没有定义过这个属性。JavaScript 遇到这种情况是不报错的。

```
const a = 0;
const b = true;
const result = a + b; // 报错
```

上面示例是合法的 JavaScript 代码，但是没有意义，不应该将数值`a`与布尔值`b`相加。TypeScript 就会直接报错，提示运算符`+`不能用于数值和布尔值的相加。

```
function hello() {
  return 'hello world';
}

hello().find('hello'); // 报错
```

上面示例中，`hello()`返回的是一个字符串，TypeScript 发现字符串没有`find()`方法，所以报错了。如果是 JavaScript，只有到运行阶段才会报错。

（3）更好的 IDE 支持，做到语法提示和自动补全。

IDE（集成开发环境，比如 VSCode）一般都会利用类型信息，提供语法提示功能（编辑器自动提示函数用法、参数等）和自动补全功能（只键入一部分的变量名或函数名，编辑器补全后面的部分）。

（4）提供了代码文档。

类型信息可以部分替代代码文档，解释应该如何使用这些代码，熟练的开发者往往只看类型，就能大致推断代码的作用。借助类型信息，很多工具能够直接生成文档。

（5）有助于代码重构。

修改他人的 JavaScript 代码，往往非常痛苦，项目越大越痛苦，因为不确定修改后是否会影响到其他部分的代码。

类型信息大大减轻了重构的成本。一般来说，只要函数或对象的参数和返回值保持类型不变，就能基本确定，重构后的代码也能正常运行。如果还有配套的单元测试，就完全可以放心重构。越是大型的、多人合作的项目，类型信息能够提供的帮助越大。

综上所述，TypeScript 有助于提高代码质量，保证代码安全，更适合用在大型的企业级项目。这就是为什么大量 JavaScript 项目转成 TypeScript 的原因。

静态类型的缺点

静态类型也存在一些缺点。

（1）丧失了动态类型的代码灵活性。

动态类型有非常高的灵活性，给予程序员很大的自由，静态类型将这些灵活性都剥夺了。

（2）增加了编程工作量。

有了类型之后，程序员不仅需要编写功能，还需要编写类型声明，确保类型正确。这增加了不少工作量，有时会显著拖长项目的开发时间。

（3）更高的学习成本。

类型系统通常比较复杂，要学习的东西更多，要求开发者付出更高的学习成本。

（4）引入了独立的编译步骤。

原生的 JavaScript 代码，可以直接在 JavaScript 引擎运行。添加类型系统以后，就多出了一个单独的编译步骤，检查类型是否正确，并将 TypeScript 代码转成 JavaScript 代码，这样才能运行。

（5）兼容性问题。

TypeScript 依赖 JavaScript 生态，需要用到很多外部模块。但是，过去大部分 JavaScript 项目都没有做 TypeScript 适配，虽然可以自己动手做适配，不过使用时难免还是会有一些兼容性问题。

总的来说，这些缺点使得 TypeScript 不一定适合那些小型的、短期的个人项目。

TypeScript 的历史

下面简要介绍 TypeScript 的发展历史。

2012年，微软公司宣布推出 TypeScript 语言，设计者是著名的编程语言设计大师 Anders Hejlsberg，他也是 C# 和 .NET 的设计师。

微软推出这门语言的主要目的，是让 JavaScript 程序员可以参与 Windows 8 应用程序的开发。

当时，Windows 8 即将发布，它的应用程序开发除了使用 C# 和 Visual Basic，还可以使用 HTML + JavaScript。微软希望，TypeScript 既能让 JavaScript 程序员快速上手，也能让 .NET 程序员感到熟悉。

这就是说，TypeScript 的最初动机是减少 .NET 程序员的转移和学习成本。所以，它的很多语法概念跟 .NET 很类似。

另外，TypeScript 是一个开源项目，接受社区的参与，核心的编译器采用 Apache 2.0 许可证。微软希望通过这种做法，迅速提高这门语言在社区的接受度。

2013年，微软的 Visual Studio 2013 开始内置支持 TypeScript 语言。

2014年，TypeScript 1.0 版本发布。同年，代码仓库搬到了 GitHub。

2016年，TypeScript 2.0 版本发布，引入了很多重大的语法功能。

2018年，TypeScript 3.0 版本发布。

2020年，TypeScript 4.0 版本发布。

2023年，TypeScript 5.0 版本发布。

如何学习

学习 TypeScript，必须先了解 JavaScript 的语法。因为真正的实际功能都是 JavaScript 引擎完成的，TypeScript 只是添加了一个类型系统。

本书假定读者已经了解 JavaScript 语言，就不再介绍它的语法了，只介绍 TypeScript 引入的新语法，主要是类型系统。

如果你对 JavaScript 还不熟悉，建议先阅读《JavaScript 教程》和《ES6 教程》，再来阅读本书。

TypeScript 基本用法

本章介绍 TypeScript 的一些最基本的语法和用法。

类型声明

TypeScript 代码最明显的特征，就是为 JavaScript 变量加上了类型声明。

```
let foo:string;
```

上面示例中，变量`foo`的后面使用冒号，声明了它的类型为`string`。

类型声明的写法，一律为在标识符后面添加“冒号 + 类型”。函数参数和返回值，也是这样来声明类型。

```
function toString(num:number):string {
  return String(num);
}
```

上面示例中，函数`toString()`的参数`num`的类型是`number`。参数列表的圆括号后面，声明了返回值的类型是`string`。更详细的介绍，参见《函数》一章。

注意，变量的值应该与声明的类型一致，如果不一致，TypeScript 就会报错。

```
// 报错
let foo:string = 123;
```

上面示例中，变量`foo`的类型是字符串，但是赋值为数值`123`，TypeScript 就报错了。

另外，TypeScript 规定，变量只有赋值后才能使用，否则就会报错。

```
let x:number;
console.log(x) // 报错
```

上面示例中，变量`x`没有赋值就被读取，导致报错。而 JavaScript 允许这种行为，不会报错，没有赋值的变量会返回`undefined`。

类型推断

类型声明并不是必需的，如果没有，TypeScript 会自己推断类型。

```
let foo = 123;
```

上面示例中，变量`foo`并没有类型声明，TypeScript 就会推断它的类型。由于它被赋值为一个数值，因此 TypeScript 推断它的类型为`number`。

后面，如果变量`foo`更改为其他类型的值，跟推断的类型不一致，TypeScript 就会报错。

```
let foo = 123;
foo = 'hello'; // 报错
```

上面示例中，变量`foo`的类型推断为`number`，后面赋值为字符串，TypeScript 就报错了。

TypeScript 也可以推断函数的返回值。

```
function toString(num:number) {
  return String(num);
}
```

上面示例中，函数`toString()`没有声明返回值的类型，但是 TypeScript 推断返回的是字符串。正是因为 TypeScript 的类型推断，所以函数返回值的类型通常是省略不写的。

从这里可以看到，TypeScript 的设计思想是，类型声明是可选的，你可以加，也可以不加。即使不加类型声明，依然是有效的 TypeScript 代码，只是这时不能保证 TypeScript 会正确推断出类型。由于这个原因，所有 JavaScript 代码都是合法的 TypeScript 代码。

这样设计还有一个好处，将以前的 JavaScript 项目改为 TypeScript 项目时，你可以逐步地为老代码添加类型，即使有些代码没有添加，也不会无法运行。

TypeScript 的编译

JavaScript 的运行环境（浏览器和 Node.js）不认识 TypeScript 代码。所以，TypeScript 项目要想运行，必须先转为 JavaScript 代码，这个代码转换的过程就叫做“编译”（compile）。

TypeScript 官方没有做运行环境，只提供编译器。编译时，会将类型声明和类型相关的代码全部删除，只留下能运行的 JavaScript 代码，并且不会改变 JavaScript 的运行结果。

因此，TypeScript 的类型检查只是编译时的类型检查，而不是运行时的类型检查。一旦代码编译为 JavaScript，运行时就不再检查类型了。

值与类型

学习 TypeScript 需要分清楚“值”（value）和“类型”（type）。

“类型”是针对“值”的，可以视为是后者的一个元属性。每一个值在 TypeScript 里面都是有类型的。比如，`3`是一个值，它的类型是`number`。

TypeScript 代码只涉及类型，不涉及值。所有跟“值”相关的处理，都由 JavaScript 完成。

这一点务必牢记。TypeScript 项目里面，其实存在两种代码，一种是底层的“值代码”，另一种是上层的“类型代码”。前者使用 JavaScript 语法，后者使用 TypeScript 的类型语法。

它们是可以分离的，TypeScript 的编译过程，实际上就是把“类型代码”全部拿掉，只保留“值代码”。

编写 TypeScript 项目时，不要混淆哪些是值代码，哪些是类型代码。

TypeScript Playground

最简单的 TypeScript 使用方法，就是使用官网的在线编译页面，叫做 TypeScript Playground。

只要打开这个网页，把 TypeScript 代码贴进文本框，它就会在当前页面自动编译出 JavaScript 代码，还可以在浏览器执行编译产物。如果编译报错，它也会给出详细的报错信息。

这个页面还具有完整的 IDE 支持，可以自动语法提示。此外，它支持把代码片段和编译器设置保存成 URL，分享给他人。

本书的示例都建议放到这个页面，进行查看和编译。

tsc 编译器

TypeScript 官方提供的编译器叫做 tsc，可以将 TypeScript 脚本编译成 JavaScript 脚本。本机想要编译 TypeScript 代码，必须安装 tsc。

根据约定，TypeScript 脚本文件使用`.ts`后缀名，JavaScript 脚本文件使用`.js`后缀名。tsc 的作用就是把`.ts`脚本转变成`.js`脚本。

安装

tsc 是一个 npm 模块，使用下面的命令安装（必须先安装 npm）。

```
$ npm install -g typescript
```

上面命令是全局安装 tsc，也可以在项目中将 tsc 安装为一个依赖模块。

安装完成后，检查一下是否安装成功。

```
# 或者 tsc --version
$ tsc -v
Version 5.1.6
```

上面命令中，`-v`或`--version`参数可以输出当前安装的 tsc 版本。

帮助信息

`-h`或`--help`参数输出帮助信息。

```
$ tsc -h
```

默认情况下，“--help”参数仅显示基本的可用选项。我们可以使用“--all”参数，查看完整的帮助信息。

```
$ tsc --all
```

编译脚本

安装 tsc 之后，就可以编译 TypeScript 脚本了。

`tsc`命令后面，加上 TypeScript 脚本文件，就可以将其编译成 JavaScript 脚本。

```
$ tsc app.ts
```

上面命令会在当前目录下，生成一个`app.js`脚本文件，这个脚本就完全是编译后生成的 JavaScript 代码。

`tsc`命令也可以一次编译多个 TypeScript 脚本。

```
$ tsc file1.ts file2.ts file3.ts
```

上面命令会在当前目录生成三个 JavaScript 脚本文件`file1.js`、`file2.js`、`file3.js`。

tsc 有很多参数，可以调整编译行为。

（1）--outFile

如果想将多个 TypeScript 脚本编译成一个 JavaScript 文件，使用`--outFile`参数。

```
$ tsc file1.ts file2.ts --outFile app.js
```

上面命令将`file1.ts`和`file2.ts`两个脚本编译成一个 JavaScript 文件`app.js`。

（2）--outDir

编译结果默认都保存在当前目录，`--outDir`参数可以指定保存到其他目录。

```
$ tsc app.ts --outDir dist
```

上面命令会在`dist`子目录下生成`app.js`。

（3）--target

为了保证编译结果能在各种 JavaScript 引擎运行，tsc 默认会将 TypeScript 代码编译成很低版本的 JavaScript，即3.0版本（以`es3`表示）。这通常不是我们想要的结果。

这时可以使用`--target`参数，指定编译后的 JavaScript 版本。建议使用`es2015`，或者更新版本。

```
$ tsc --target es2015 app.ts
```

编译错误的处理

编译过程中，如果没有报错，`tsc`命令不会有任何显示。所以，如果你没有看到任何提示，就表示编译成功了。

如果编译报错，`tsc`命令就会显示报错信息，但是这种情况下，依然会编译生成 JavaScript 脚本。

举例来说，下面是一个错误的 TypeScript 脚本`app.ts`。

```
// app.ts
let foo:number = 123;
foo = 'abc'; // 报错
```

上面示例中，变量`foo`是数值类型，赋值为字符串，`tsc`命令编译这个脚本就会报错。

```
$ tsc app.ts

app.ts:2:1 - error TS2322: Type 'string' is not assignable to type 'number'.

2 foo = 'abc';
  ~~~

Found 1 error in app.ts:2
```

上面示例中，`tsc`命令输出报错信息，表示变量`foo`被错误地赋值为字符串。

这种情况下，编译产物`app.js`还是会照样生成，下面就是编译后的结果。

```
// app.js
var foo = 123;
foo = 'abc';
```

可以看到，尽管有错，tsc 依然原样将 TypeScript 编译成 JavaScript 脚本。

这是因为 TypeScript 团队认为，编译器的作用只是给出编译错误，至于怎么处理这些错误，那就是开发者自己的判断了。开发者更了解自己的代码，所以不管怎样，编译产物都会生成，让开发者决定下一步怎么处理。

如果希望一旦报错就停止编译，不生成编译产物，可以使用`--noEmitOnError`参数。

```
$ tsc --noEmitOnError app.ts
```

上面命令在报错后，就不会生成`app.js`。

tsc 还有一个`--noEmit`参数，只检查类型是否正确，不生成 JavaScript 文件。

```
$ tsc --noEmit app.ts
```

上面命令只检查是否有编译错误，不会生成`app.js`。

tsc 命令的更多参数，详见《tsc 编译器》一章。

tsconfig.json

TypeScript 允许将`tsc`的编译参数，写在配置文件`tsconfig.json`。只要当前目录有这个文件，`tsc`就会自动读取，所以运行时可以不写参数。

```
$ tsc file1.ts file2.ts --outFile dist/app.js
```

上面这个命令写成`tsconfig.json`，就是下面这样。

```
{
  "files": ["file1.ts", "file2.ts"],
  "compilerOptions": {
    "outFile": "dist/app.js"
  }
}
```

有了这个配置文件，编译时直接调用`tsc`命令就可以了。

```
$ tsc
```

`tsconfig.json`的详细介绍，参见《tsconfig.json 配置文件》一章。

ts-node 模块

ts-node 是一个非官方的 npm 模块，可以直接运行 TypeScript 代码。

使用时，可以先全局安装它。

```
$ npm install -g ts-node
```

安装后，就可以直接运行 TypeScript 脚本。

```
$ ts-node script.ts
```

上面命令运行了 TypeScript 脚本`script.ts`，给出运行结果。

如果不安装 ts-node，也可以通过 npx 调用它来运行 TypeScript 脚本。

```
$ npx ts-node script.ts
```

上面命令中，`npx`会在线调用 ts-node，从而在不安装的情况下，运行`script.ts`。

如果执行 ts-node 命令不带有任何参数，它会提供一个 TypeScript 的命令行 REPL 运行环境，你可以在这个环境中输入 TypeScript 代码，逐行执行。

```
$ ts-node
>
```

上面示例中，单独运行`ts-node`命令，会给出一个大于号，这就是 TypeScript 的 REPL 运行环境，可以逐行输入代码运行。

```
$ ts-node
> const twice = (x:string) => x + x;
> twice('abc')
'abcabc'
>
```

上面示例中，在 TypeScript 命令行 REPL 环境中，先输入一个函数`twice`，然后调用该函数，就会得到结果。

要退出这个 REPL 环境，可以按下 Ctrl + d，或者输入`.exit`。

如果只是想简单运行 TypeScript 代码看看结果，ts-node 不失为一个便捷的方法。

any 类型，unknown 类型，never 类型

本章介绍 TypeScript 的三种特殊类型，它们可以作为学习 TypeScript 类型系统的起点。

any 类型

基本含义

any 类型表示没有任何限制，该类型的变量可以赋予任意类型的值。

```
let x:any;

x = 1; // 正确
x = 'foo'; // 正确
x = true; // 正确
```

上面示例中，变量`x`的类型是`any`，就可以被赋值为任意类型的值。

变量类型一旦设为`any`，TypeScript 实际上会关闭这个变量的类型检查。即使有明显的类型错误，只要句法正确，都不会报错。

```
let x:any = 'hello';

x(1) // 不报错
x.foo = 100; // 不报错
```

上面示例中，变量`x`的值是一个字符串，但是把它当作函数调用，或者当作对象读取任意属性，TypeScript 编译时都不报错。原因就是`x`的类型是`any`，TypeScript 不对其进行类型检查。

由于这个原因，应该尽量避免使用`any`类型，否则就失去了使用 TypeScript 的意义。

实际开发中，`any`类型主要适用以下两个场合。

（1）出于特殊原因，需要关闭某些变量的类型检查，就可以把该变量的类型设为`any`。

（2）为了适配以前老的 JavaScript 项目，让代码快速迁移到 TypeScript，可以把变量类型设为`any`。有些年代很久的大型 JavaScript 项目，尤其是别人的代码，很难为每一行适配正确的类型，这时你为那些类型复杂的变量加上`any`，TypeScript 编译时就不会报错。

总之，TypeScript 认为，只要开发者使用了`any`类型，就表示开发者想要自己来处理这些代码，所以就不对`any`类型进行任何限制，怎么使用都可以。

从集合论的角度看，`any`类型可以看成是所有其他类型的全集，包含了一切可能的类型。TypeScript 将这种类型称为“顶层类型”（top type），意为涵盖了所有下层。

类型推断问题

对于开发者没有指定类型、TypeScript 必须自己推断类型的那些变量，如果无法推断出类型，TypeScript 就会认为该变量的类型是`any`。

```
function add(x, y) {
  return x + y;
}

add(1, [1, 2, 3]) // 不报错
```

上面示例中，函数`add()`的参数变量`x`和`y`，都没有足够的信息，TypeScript 无法推断出它们的类型，就会认为这两个变量和函数返回值的类型都是`any`。以至于后面就不再对函数`add()`进行类型检查了，怎么用都可以。

这显然是很糟糕的情况，所以对于那些类型不明显的变量，一定要显式声明类型，防止被推断为`any`。

TypeScript 提供了一个编译选项`noImplicitAny`，打开该选项，只要推断出`any`类型就会报错。

```
$ tsc --noImplicitAny app.ts
```

上面命令使用了`noImplicitAny`编译选项进行编译，这时上面的函数`add()`就会报错。

这里有一个特殊情况，即使打开了`noImplicitAny`，使用`let`和`var`命令声明变量，但不赋值也不指定类型，是不会报错的。

```
var x; // 不报错
let y; // 不报错
```

上面示例中，变量`x`和`y`声明时没有赋值，也没有指定类型，TypeScript 会推断它们的类型为`any`。这时即使打开了`noImplicitAny`，也不会报错。

```
let x;

x = 123;
x = { foo: 'hello' };
```

上面示例中，变量`x`的类型推断为`any`，但是不报错，可以顺利通过编译。

由于这个原因，建议使用`let`和`var`声明变量时，如果不赋值，就一定要显式声明类型，否则可能存在安全隐患。

`const`命令没有这个问题，因为 JavaScript 语言规定`const`声明变量时，必须同时进行初始化（赋值）。

```
const x; // 报错
```

上面示例中，`const`命令声明的`x`是不能改变值的，声明时必须同时赋值，否则报错，所以它不存在类型推断为`any`的问题。

污染问题

`any`类型除了关闭类型检查，还有一个很大的问题，就是它会“污染”其他变量。它可以赋值给其他任何类型的变量（因为没有类型检查），导致其他变量出错。

```
let x:any = 'hello';
let y:number;

y = x; // 不报错

y * 123 // 不报错
y.toFixed() // 不报错
```

上面示例中，变量`x`的类型是`any`，实际的值是一个字符串。变量`y`的类型是`number`，表示这是一个数值变量，但是它被赋值为`x`，这时并不会报错。然后，变量`y`继续进行各种数值运算，TypeScript 也检查不出错误，问题就这样留到运行时才会暴露。

污染其他具有正确类型的变量，把错误留到运行时，这就是不宜使用`any`类型的另一个主要原因。

unknown 类型

为了解决`any`类型“污染”其他变量的问题，TypeScript 3.0 引入了`unknown`类型。它与`any`含义相同，表示类型不确定，可能是任意类型，但是它的使用有一些限制，不像`any`那样自由，可以视为严格版的`any`。

`unknown`跟`any`的相似之处，在于所有类型的值都可以分配给`unknown`类型。

```
let x:unknown;

x = true; // 正确
x = 42; // 正确
x = 'Hello World'; // 正确
```

上面示例中，变量`x`的类型是`unknown`，可以赋值为各种类型的值。这与`any`的行为一致。

`unknown`类型跟`any`类型的不同之处在于，它不能直接使用。主要有以下几个限制。

首先，`unknown`类型的变量，不能直接赋值给其他类型的变量（除了`any`类型和`unknown`类型）。

```
let v:unknown = 123;

let v1:boolean = v; // 报错
let v2:number = v; // 报错
```

上面示例中，变量`v`是`unknown`类型，赋值给`any`和`unknown`以外类型的变量都会报错，这就避免了污染问题，从而克服了`any`类型的一大缺点。

其次，不能直接调用`unknown`类型变量的方法和属性。

```
let v1:unknown = { foo: 123 };
v1.foo  // 报错

let v2:unknown = 'hello';
v2.trim() // 报错

let v3:unknown = (n = 0) => n + 1;
v3() // 报错
```

上面示例中，直接调用`unknown`类型变量的属性和方法，或者直接当作函数执行，都会报错。

再次，`unknown`类型变量能够进行的运算是有限的，只能进行比较运算（运算符`==`、`===`、`!=`、`!==`、`||`、`&&`、`?`）、取反运算（运算符`!`）、`typeof`运算符和`instanceof`运算符这几种，其他运算都会报错。

```
let a:unknown = 1;

a + 1 // 报错
a === 1 // 正确
```

上面示例中，`unknown`类型的变量`a`进行加法运算会报错，因为这是不允许的运算。但是，进行比较运算就是可以的。

那么，怎么才能使用`unknown`类型变量呢?

答案是只有经过“类型缩小”，`unknown`类型变量才可以使用。所谓“类型缩小”，就是缩小`unknown`变量的类型范围，确保不会出错。

```
let a:unknown = 1;

if (typeof a === 'number') {
  let r = a + 10; // 正确
}
```

上面示例中，`unknown`类型的变量`a`经过`typeof`运算以后，能够确定实际类型是`number`，就能用于加法运算了。这就是“类型缩小”，即将一个不确定的类型缩小为更明确的类型。

下面是另一个例子。

```
let s:unknown = 'hello';

if (typeof s === 'string') {
  s.length; // 正确
}
```

上面示例中，确定变量`s`的类型为字符串以后，才能调用它的`length`属性。

这样设计的目的是，只有明确`unknown`变量的实际类型，才允许使用它，防止像`any`那样可以随意乱用，“污染”其他变量。类型缩小以后再使用，就不会报错。

总之，`unknown`可以看作是更安全的`any`。一般来说，凡是需要设为`any`类型的地方，通常都应该优先考虑设为`unknown`类型。

在集合论上，`unknown`也可以视为所有其他类型（除了`any`）的全集，所以它和`any`一样，也属于 TypeScript 的顶层类型。

never 类型

为了保持与集合论的对应关系，以及类型运算的完整性，TypeScript 还引入了“空类型”的概念，即该类型为空，不包含任何值。

由于不存在任何属于“空类型”的值，所以该类型被称为`never`，即不可能有这样的值。

```
let x:never;
```

上面示例中，变量`x`的类型是`never`，就不可能赋给它任何值，否则都会报错。

`never`类型的使用场景，主要是在一些类型运算之中，保证类型运算的完整性，详见后面章节。另外，不可能返回值的函数，返回值的类型就可以写成`never`，详见《函数》一章。

如果一个变量可能有多种类型（即联合类型），通常需要使用分支处理每一种类型。这时，处理所有可能的类型之后，剩余的情况就属于`never`类型。

```
function fn(x:string|number) {
  if (typeof x === 'string') {
    // ...
  } else if (typeof x === 'number') {
    // ...
  } else {
    x; // never 类型
  }
}
```

上面示例中，参数变量`x`可能是字符串，也可能是数值，判断了这两种情况后，剩下的最后那个`else`分支里面，`x`就是`never`类型了。

`never`类型的一个重要特点是，可以赋值给任意其他类型。

```
function f():never {
  throw new Error('Error');
}

let v1:number = f(); // 不报错
let v2:string = f(); // 不报错
let v3:boolean = f(); // 不报错
```

上面示例中，函数`f()`会抛出错误，所以返回值类型可以写成`never`，即不可能返回任何值。各种其他类型的变量都可以赋值为`f()`的运行结果（`never`类型）。

为什么`never`类型可以赋值给任意其他类型呢？这也跟集合论有关，空集是任何集合的子集。TypeScript 就相应规定，任何类型都包含了`never`类型。因此，`never`类型是任何其他类型所共有的，TypeScript 把这种情况称为“底层类型”（bottom type）。

总之，TypeScript 有两个“顶层类型”（`any`和`unknown`），但是“底层类型”只有`never`唯一一个。

TypeScript 的类型系统

本章是 TypeScript 类型系统的总体介绍。

TypeScript 继承了 JavaScript 的类型，在这个基础上，定义了一套自己的类型系统。

基本类型

概述

JavaScript 语言（注意，不是 TypeScript）将值分成8种类型。

- boolean
- string
- number
- bigint
- symbol
- object
- undefined
- null

TypeScript 继承了 JavaScript 的类型设计，以上8种类型可以看作 TypeScript 的基本类型。

注意，上面所有类型的名称都是小写字母，首字母大写的`Number`、`String`、`Boolean`等在 JavaScript 语言中都是内置对象，而不是类型名称。

另外，undefined 和 null 既可以作为值，也可以作为类型，取决于在哪里使用它们。

这8种基本类型是 TypeScript 类型系统的基础，复杂类型由它们组合而成。

以下是它们的简单介绍。

boolean 类型

`boolean`类型只包含`true`和`false`两个布尔值。

```
const x:boolean = true;
const y:boolean = false;
```

上面示例中，变量`x`和`y`就属于 boolean 类型。

string 类型

`string`类型包含所有字符串。

```
const x:string = 'hello';
const y:string = `${x} world`;
```

上面示例中，普通字符串和模板字符串都属于 string 类型。

number 类型

`number`类型包含所有整数和浮点数。

```
const x:number = 123;
const y:number = 3.14;
const z:number = 0xffff;
```

上面示例中，整数、浮点数和非十进制数都属于 number 类型。

bigint 类型

bigint 类型包含所有的大整数。

```
const x:bigint = 123n;
const y:bigint = 0xffffn;
```

上面示例中，变量`x`和`y`就属于 bigint 类型。

bigint 与 number 类型不兼容。

```
const x:bigint = 123; // 报错
const y:bigint = 3.14; // 报错
```

上面示例中，`bigint`类型赋值为整数和小数，都会报错。

注意，bigint 类型是 ES2020 标准引入的。如果使用这个类型，TypeScript 编译的目标 JavaScript 版本不能低于 ES2020（即编译参数`target`不低于`es2020`）。

symbol 类型

symbol 类型包含所有的 Symbol 值。

```
const x:symbol = Symbol();
```

上面示例中，`Symbol()`函数的返回值就是 symbol 类型。

symbol 类型的详细介绍，参见《Symbol》一章。

object 类型

根据 JavaScript 的设计，object 类型包含了所有对象、数组和函数。

```
const x:object = { foo: 123 };
const y:object = [1, 2, 3];
const z:object = (n:number) => n + 1;
```

上面示例中，对象、数组、函数都属于 object 类型。

undefined 类型，null 类型

undefined 和 null 是两种独立类型，它们各自都只有一个值。

undefined 类型只包含一个值`undefined`，表示未定义（即还未给出定义，以后可能会有定义）。

```
let x:undefined = undefined;
```

上面示例中，变量`x`就属于 undefined 类型。两个`undefined`里面，第一个是类型，第二个是值。

null 类型也只包含一个值`null`，表示为空（即此处没有值）。

```
const x:null = null;
```

上面示例中，变量`x`就属于 null 类型。

注意，如果没有声明类型的变量，被赋值为`undefined`或`null`，在关闭编译设置`noImplicitAny`和`strictNullChecks`时，它们的类型会被推断为`any`。

```
// 关闭 noImplicitAny 和 strictNullChecks

let a = undefined;   // any
const b = undefined; // any

let c = null;        // any
const d = null;      // any
```

如果希望避免这种情况，则需要打开编译选项`strictNullChecks`。

```
// 打开编译设置 strictNullChecks

let a = undefined;   // undefined
const b = undefined; // undefined

let c = null;        // null
const d = null;      // null
```

上面示例中，打开编译设置`strictNullChecks`以后，赋值为`undefined`的变量会被推断为`undefined`类型，赋值为`null`的变量会被推断为`null`类型。

包装对象类型

包装对象的概念

JavaScript 的8种类型之中，`undefined`和`null`其实是两个特殊值，`object`属于复合类型，剩下的五种属于原始类型（primitive value），代表最基本的、不可再分的值。

- boolean
- string
- number
- bigint
- symbol

上面这五种原始类型的值，都有对应的包装对象（wrapper object）。所谓“包装对象”，指的是这些值在需要时，会自动产生的对象。

```
'hello'.charAt(1) // 'e'
```

上面示例中，字符串`hello`执行了`charAt()`方法。但是，在 JavaScript 语言中，只有对象才有方法，原始类型的值本身没有方法。这行代码之所以可以运行，就是因为在调用方法时，字符串会自动转为包装对象，`charAt()`方法其实是定义在包装对象上。

这样的设计大大方便了字符串处理，省去了将原始类型的值手动转成对象实例的麻烦。

五种包装对象之中，symbol 类型和 bigint 类型无法直接获取它们的包装对象（即`Symbol()`和`BigInt()`不能作为构造函数使用），但是剩下三种可以。

- `Boolean()`
- `String()`
- `Number()`

以上三个构造函数，执行后可以直接获取某个原始类型值的包装对象。

```
const s = new String('hello');
typeof s // 'object'
s.charAt(1) // 'e'
```

上面示例中，`s`就是字符串`hello`的包装对象，`typeof`运算符返回`object`，不是`string`，但是本质上它还是字符串，可以使用所有的字符串方法。

注意，`String()`只有当作构造函数使用时（即带有`new`命令调用），才会返回包装对象。如果当作普通函数使用（不带有`new`命令），返回就是一个普通字符串。其他两个构造函数`Number()`和`Boolean()`也是如此。

包装对象类型与字面量类型

由于包装对象的存在，导致每一个原始类型的值都有包装对象和字面量两种情况。

```
'hello' // 字面量
new String('hello') // 包装对象
```

上面示例中，第一行是字面量，第二行是包装对象，它们都是字符串。

为了区分这两种情况，TypeScript 对五种原始类型分别提供了大写和小写两种类型。

- Boolean 和 boolean
- String 和 string
- Number 和 number
- BigInt 和 bigint
- Symbol 和 symbol

其中，大写类型同时包含包装对象和字面量两种情况，小写类型只包含字面量，不包含包装对象。

```
const s1:String = 'hello'; // 正确
const s2:String = new String('hello'); // 正确

const s3:string = 'hello'; // 正确
const s4:string = new String('hello'); // 报错
```

上面示例中，`String`类型可以赋值为字符串的字面量，也可以赋值为包装对象。但是，`string`类型只能赋值为字面量，赋值为包装对象就会报错。

建议只使用小写类型，不使用大写类型。因为绝大部分使用原始类型的场合，都是使用字面量，不使用包装对象。而且，TypeScript 把很多内置方法的参数，定义成小写类型，使用大写类型会报错。

```
const n1:number = 1;
const n2:Number = 1;

Math.abs(n1) // 1
Math.abs(n2) // 报错
```

上面示例中，`Math.abs()`方法的参数类型被定义成小写的`number`，传入大写的`Number`类型就会报错。

上一小节说过，`Symbol()`和`BigInt()`这两个函数不能当作构造函数使用，所以没有办法直接获得 symbol 类型和 bigint 类型的包装对象，除非使用下面的写法。但是，它们没有使用场景，因此`Symbol`和`BigInt`这两个类型虽然存在，但是完全没有使用的理由。

```
let a = Object(Symbol());
let b = Object(BigInt());
```

上面示例中，得到的就是 Symbol 和 BigInt 的包装对象，但是没有使用的意义。

注意，目前在 TypeScript 里面，`symbol`和`Symbol`两种写法没有差异，`bigint`和`BigInt`也是如此，不知道是否属于官方的疏忽。建议始终使用小写的`symbol`和`bigint`，不使用大写的`Symbol`和`BigInt`。

Object 类型与 object 类型

TypeScript 的对象类型也有大写`Object`和小写`object`两种。

Object 类型

大写的`Object`类型代表 JavaScript 语言里面的广义对象。所有可以转成对象的值，都是`Object`类型，这囊括了几乎所有的值。

```
let obj:Object;

obj = true;
obj = 'hi';
obj = 1;
obj = { foo: 123 };
obj = [1, 2];
obj = (a:number) => a + 1;
```

上面示例中，原始类型值、对象、数组、函数都是合法的`Object`类型。

事实上，除了`undefined`和`null`这两个值不能转为对象，其他任何值都可以赋值给`Object`类型。

```
let obj:Object;

obj = undefined; // 报错
obj = null; // 报错
```

上面示例中，`undefined`和`null`赋值给`Object`类型，就会报错。

另外，空对象`{}`是`Object`类型的简写形式，所以使用`Object`时常常用空对象代替。

```
let obj:{};

obj = true;
obj = 'hi';
obj = 1;
obj = { foo: 123 };
obj = [1, 2];
obj = (a:number) => a + 1;
```

上面示例中，变量`obj`的类型是空对象`{}`，就代表`Object`类型。

显然，无所不包的`Object`类型既不符合直觉，也不方便使用。

object 类型

小写的`object`类型代表 JavaScript 里面的狭义对象，即可以用字面量表示的对象，只包含对象、数组和函数，不包括原始类型的值。

```
let obj:object;

obj = { foo: 123 };
obj = [1, 2];
obj = (a:number) => a + 1;
obj = true; // 报错
obj = 'hi'; // 报错
obj = 1; // 报错
```

上面示例中，`object`类型不包含原始类型值，只包含对象、数组和函数。

大多数时候，我们使用对象类型，只希望包含真正的对象，不希望包含原始类型。所以，建议总是使用小写类型`object`，不使用大写类型`Object`。

注意，无论是大写的`Object`类型，还是小写的`object`类型，都只包含 JavaScript 内置对象原生的属性和方法，用户自定义的属性和方法都不存在于这两个类型之中。

```
const o1:Object = { foo: 0 };
const o2:object = { foo: 0 };

o1.toString() // 正确
o1.foo // 报错

o2.toString() // 正确
o2.foo // 报错
```

上面示例中，`toString()`是对象的原生方法，可以正确访问。`foo`是自定义属性，访问就会报错。如何描述对象的自定义属性，详见《对象类型》一章。

undefined 和 null 的特殊性

`undefined`和`null`既是值，又是类型。

作为值，它们有一个特殊的地方：任何其他类型的变量都可以赋值为`undefined`或`null`。

```
let age:number = 24;

age = null;      // 正确
age = undefined; // 正确
```

上面代码中，变量`age`的类型是`number`，但是赋值为`null`或`undefined`并不报错。

这并不是因为`undefined`和`null`包含在`number`类型里面，而是故意这样设计，任何类型的变量都可以赋值为`undefined`和`null`，以便跟 JavaScript 的行为保持一致。

JavaScript 的行为是，变量如果等于`undefined`就表示还没有赋值，如果等于`null`就表示值为空。所以，TypeScript 就允许了任何类型的变量都可以赋值为这两个值。

但是有时候，这并不是开发者想要的行为，也不利于发挥类型系统的优势。

```
const obj:object = undefined;
obj.toString() // 编译不报错，运行就报错
```

上面示例中，变量`obj`等于`undefined`，编译不会报错。但是，实际执行时，调用`obj.toString()`就报错了，因为`undefined`不是对象，没有这个方法。

为了避免这种情况，及早发现错误，TypeScript 提供了一个编译选项`strictNullChecks`。只要打开这个选项，`undefined`和`null`就不能赋值给其他类型的变量（除了`any`类型和`unknown`类型）。

下面是 tsc 命令打开这个编译选项的例子。

```
// tsc --strictNullChecks app.ts

let age:number = 24;

age = null;      // 报错
age = undefined; // 报错
```

上面示例中，打开`--strictNullChecks`以后，`number`类型的变量`age`就不能赋值为`undefined`和`null`。

这个选项在配置文件`tsconfig.json`的写法如下。

```
{
  "compilerOptions": {
    "strictNullChecks": true
    // ...
  }
}
```

打开`strictNullChecks`以后，`undefined`和`null`这两种值也不能互相赋值了。

```
// 打开 strictNullChecks

let x:undefined = null; // 报错
let y:null = undefined; // 报错
```

上面示例中，`undefined`类型的变量赋值为`null`，或者`null`类型的变量赋值为`undefined`，都会报错。

总之，打开`strictNullChecks`以后，`undefined`和`null`只能赋值给自身，或者`any`类型和`unknown`类型的变量。

```
let x:any     = undefined;
let y:unknown = null;
```

值类型

TypeScript 规定，单个值也是一种类型，称为“值类型”。

```
let x:'hello';

x = 'hello'; // 正确
x = 'world'; // 报错
```

上面示例中，变量`x`的类型是字符串`hello`，导致它只能赋值为这个字符串，赋值为其他字符串就会报错。

TypeScript 推断类型时，遇到`const`命令声明的变量，如果代码里面没有注明类型，就会推断该变量是值类型。

```
// x 的类型是 "https"
const x = 'https';

// y 的类型是 string
const y:string = 'https';
```

上面示例中，变量`x`是`const`命令声明的，TypeScript 就会推断它的类型是值`https`，而不是`string`类型。

这样推断是合理的，因为`const`命令声明的变量，一旦声明就不能改变，相当于常量。值类型就意味着不能赋为其他值。

注意，`const`命令声明的变量，如果赋值为对象，并不会推断为值类型。

```
// x 的类型是 { foo: number }
const x = { foo: 1 };
```

上面示例中，变量`x`没有被推断为值类型，而是推断属性`foo`的类型是`number`。这是因为 JavaScript 里面，`const`变量赋值为对象时，属性值是可以改变的。

值类型可能会出现一些很奇怪的报错。

```
const x:5 = 4 + 1; // 报错
```

上面示例中，等号左侧的类型是数值`5`，等号右侧`4 + 1`的类型，TypeScript 推测为`number`。由于`5`是`number`的子类型，`number`是`5`的父类型，父类型不能赋值给子类型，所以报错了（详见本章后文）。

但是，反过来是可以的，子类型可以赋值给父类型。

```
let x:5 = 5;
let y:number = 4 + 1;

x = y; // 报错
y = x; // 正确
```

上面示例中，变量`x`属于子类型，变量`y`属于父类型。子类型`x`不能赋值为父类型`y`，但是反过来是可以的。

如果一定要让子类型可以赋值为父类型的值，就要用到类型断言（详见《类型断言》一章）。

```
const x:5 = (4 + 1) as 5; // 正确
```

上面示例中，在`4 + 1`后面加上`as 5`，就是告诉编译器，可以把`4 + 1`的类型视为值类型`5`，这样就不会报错了。

只包含单个值的值类型，用处不大。实际开发中，往往将多个值结合，作为联合类型使用。

联合类型

联合类型（union types）指的是多个类型组成的一个新类型，使用符号`|`表示。

联合类型`A|B`表示，任何一个类型只要属于`A`或`B`，就属于联合类型`A|B`。

```
let x:string|number;

x = 123; // 正确
x = 'abc'; // 正确
```

上面示例中，变量`x`就是联合类型`string|number`，表示它的值既可以是字符串，也可以是数值。

联合类型可以与值类型相结合，表示一个变量的值有若干种可能。

```
let setting:true|false;

let gender:'male'|'female';

let rainbowColor:'赤'|'橙'|'黄'|'绿'|'青'|'蓝'|'紫';
```

上面的示例都是由值类型组成的联合类型，非常清晰地表达了变量的取值范围。其中，`true|false`其实就是布尔类型`boolean`。

前面提到，打开编译选项`strictNullChecks`后，其他类型的变量不能赋值为`undefined`或`null`。这时，如果某个变量确实可能包含空值，就可以采用联合类型的写法。

```
let name:string|null;

name = 'John';
name = null;
```

上面示例中，变量`name`的值可以是字符串，也可以是`null`。

联合类型的第一个成员前面，也可以加上竖杠`|`，这样便于多行书写。

```
let x:
  | 'one'
  | 'two'
  | 'three'
  | 'four';
```

上面示例中，联合类型的第一个成员`one`前面，加上了竖杠。

如果一个变量有多种类型，读取该变量时，往往需要进行“类型缩小”（type narrowing），区分该值到底属于哪一种类型，然后再进一步处理。

```
function printId(
  id:number|string
) {
    console.log(id.toUpperCase()); // 报错
}
```

上面示例中，参数变量`id`可能是数值，也可能是字符串，这时直接对这个变量调用`toUpperCase()`方法会报错，因为这个方法只存在于字符串，不存在于数值。

解决方法就是对参数`id`做一下类型缩小，确定它的类型以后再进行处理。

```
function printId(
  id:number|string
) {
  if (typeof id === 'string') {
    console.log(id.toUpperCase());
  } else {
    console.log(id);
  }
}
```

上面示例中，函数体内部会判断一下变量`id`的类型，如果是字符串，就对其执行`toUpperCase()`方法。

“类型缩小”是 TypeScript 处理联合类型的标准方法，凡是遇到可能为多种类型的场合，都需要先缩小类型，再进行处理。实际上，联合类型本身可以看成是一种“类型放大”（type widening），处理时就需要“类型缩小”（type narrowing）。

下面是“类型缩小”的另一个例子。

```
function getPort(
  scheme: 'http'|'https'
) {
  switch (scheme) {
    case 'http':
      return 80;
    case 'https':
      return 443;
  }
}
```

上面示例中，函数体内部对参数变量`scheme`进行类型缩小，根据不同的值类型，返回不同的结果。

交叉类型

交叉类型（intersection types）指的是多个类型组成的一个新类型，使用符号`&`表示。

交叉类型`A&B`表示，任何一个类型必须同时属于`A`和`B`，才属于交叉类型`A&B`，即交叉类型同时满足`A`和`B`的特征。

```
let x:number&string;
```

上面示例中，变量`x`同时是数值和字符串，这当然是不可能的，所以 TypeScript 会认为`x`的类型实际是`never`。

交叉类型的主要用途是表示对象的合成。

```
let obj:
  { foo: string } &
  { bar: string };

obj = {
  foo: 'hello',
  bar: 'world'
};
```

上面示例中，变量`obj`同时具有属性`foo`和属性`bar`。

交叉类型常常用来为对象类型添加新属性。

```
type A = { foo: number };

type B = A & { bar: number };
```

上面示例中，类型`B`是一个交叉类型，用来在`A`的基础上增加了属性`bar`。

type 命令

`type`命令用来定义一个类型的别名。

```
type Age = number;

let age:Age = 55;
```

上面示例中，`type`命令为`number`类型定义了一个别名`Age`。这样就能像使用`number`一样，使用`Age`作为类型。

别名可以让类型的名字变得更有意义，也能增加代码的可读性，还可以使复杂类型用起来更方便，便于以后修改变量的类型。

别名不允许重名。

```
type Color = 'red';
type Color = 'blue'; // 报错
```

上面示例中，同一个别名`Color`声明了两次，就报错了。

别名的作用域是块级作用域。这意味着，代码块内部定义的别名，影响不到外部。

```
type Color = 'red';

if (Math.random() < 0.5) {
  type Color = 'blue';
}
```

上面示例中，`if`代码块内部的类型别名`Color`，跟外部的`Color`是不一样的。

别名支持使用表达式，也可以在定义一个别名时，使用另一个别名，即别名允许嵌套。

```
type World = "world";
type Greeting = `hello ${World}`;
```

上面示例中，别名`Greeting`使用了模板字符串，读取另一个别名`World`。

`type`命令属于类型相关的代码，编译成 JavaScript 的时候，会被全部删除。

typeof 运算符

JavaScript 语言中，typeof 运算符是一个一元运算符，返回一个字符串，代表操作数的类型。

```
typeof 'foo'; // 'string'
```

上面示例中，`typeof`运算符返回字符串`foo`的类型是`string`。

注意，这时 typeof 的操作数是一个值。

JavaScript 里面，`typeof`运算符只可能返回八种结果，而且都是字符串。

```
typeof undefined; // "undefined"
typeof true; // "boolean"
typeof 1337; // "number"
typeof "foo"; // "string"
typeof {}; // "object"
typeof parseInt; // "function"
typeof Symbol(); // "symbol"
typeof 127n // "bigint"
```

上面示例是`typeof`运算符在 JavaScript 语言里面，可能返回的八种结果。

TypeScript 将`typeof`运算符移植到了类型运算，它的操作数依然是一个值，但是返回的不是字符串，而是该值的 TypeScript 类型。

```
const a = { x: 0 };

type T0 = typeof a;   // { x: number }
type T1 = typeof a.x; // number
```

上面示例中，`typeof a`表示返回变量`a`的 TypeScript 类型（`{ x: number }`）。同理，`typeof a.x`返回的是属性`x`的类型（`number`）。

这种用法的`typeof`返回的是 TypeScript 类型，所以只能用在类型运算之中（即跟类型相关的代码之中），不能用在值运算。

也就是说，同一段代码可能存在两种`typeof`运算符，一种用在值相关的 JavaScript 代码部分，另一种用在类型相关的 TypeScript 代码部分。

```
let a = 1;
let b:typeof a;

if (typeof a === 'number') {
  b = a;
}
```

上面示例中，用到了两个`typeof`，第一个是类型运算，第二个是值运算。它们是不一样的，不要混淆。

JavaScript 的 typeof 遵守 JavaScript 规则，TypeScript 的 typeof 遵守 TypeScript 规则。它们的一个重要区别在于，编译后，前者会保留，后者会被全部删除。

上例的代码编译结果如下。

```
let a = 1;
let b;
if (typeof a === 'number') {
    b = a;
}
```

上面示例中，只保留了原始代码的第二个 typeof，删除了第一个 typeof。

由于编译时不会进行 JavaScript 的值运算，所以TypeScript 规定，typeof 的参数只能是标识符，不能是需要运算的表达式。

```
type T = typeof Date(); // 报错
```

上面示例会报错，原因是 typeof 的参数不能是一个值的运算式，而`Date()`需要运算才知道结果。

另外，`typeof`命令的参数不能是类型。

```
type Age = number;
type MyAge = typeof Age; // 报错
```

上面示例中，`Age`是一个类型别名，用作`typeof`命令的参数就会报错。

typeof 是一个很重要的 TypeScript 运算符，有些场合不知道某个变量`foo`的类型，这时使用`typeof foo`就可以获得它的类型。

块级类型声明

TypeScript 支持块级类型声明，即类型可以声明在代码块（用大括号表示）里面，并且只在当前代码块有效。

```
if (true) {
  type T = number;
  let v:T = 5;
} else {
  type T = string;
  let v:T = 'hello';
}
```

上面示例中，存在两个代码块，其中分别有一个类型`T`的声明。这两个声明都只在自己的代码块内部有效，在代码块外部无效。

类型的兼容

TypeScript 的类型存在兼容关系，某些类型可以兼容其他类型。

```
type T = number|string;

let a:number = 1;
let b:T = a;
```

上面示例中，变量`a`和`b`的类型是不一样的，但是变量`a`赋值给变量`b`并不会报错。这时，我们就认为，`b`的类型兼容`a`的类型。

TypeScript 为这种情况定义了一个专门术语。如果类型`A`的值可以赋值给类型`B`，那么类型`A`就称为类型`B`的子类型（subtype）。在上例中，类型`number`就是类型`number|string`的子类型。

TypeScript 的一个规则是，凡是可以使用父类型的地方，都可以使用子类型，但是反过来不行。

```
let a:'hi' = 'hi';
let b:string = 'hello';

b = a; // 正确
a = b; // 报错
```

上面示例中，`hi`是`string`的子类型，`string`是`hi`的父类型。所以，变量`a`可以赋值给变量`b`，但是反过来就会报错。

之所以有这样的规则，是因为子类型继承了父类型的所有特征，所以可以用在父类型的场合。但是，子类型还可能有一些父类型没有的特征，所以父类型不能用在子类型的场合。

TypeScript 的数组类型

JavaScript 数组在 TypeScript 里面分成两种类型，分别是数组（array）和元组（tuple）。

本章介绍数组，下一章介绍元组。

简介

TypeScript 数组有一个根本特征：所有成员的类型必须相同，但是成员数量是不确定的，可以是无限数量的成员，也可以是零成员。

数组的类型有两种写法。第一种写法是在数组成员的类型后面，加上一对方括号。

```
let arr:number[] = [1, 2, 3];
```

上面示例中，数组`arr`的类型是`number[]`，其中`number`表示数组成员类型是`number`。

如果数组成员的类型比较复杂，可以写在圆括号里面。

```
let arr:(number|string)[];
```

上面示例中，数组`arr`的成员类型是`number|string`。

这个例子里面的圆括号是必须的，否则因为竖杠`|`的优先级低于`[]`，TypeScript 会把`number|string[]`理解成`number`和`string[]`的联合类型。

如果数组成员可以是任意类型，写成`any[]`。当然，这种写法是应该避免的。

```
let arr:any[];
```

数组类型的第二种写法是使用 TypeScript 内置的 Array 接口。

```
let arr:Array<number> = [1, 2, 3];
```

上面示例中，数组`arr`的类型是`Array<number>`，其中`number`表示成员类型是`number`。

这种写法对于成员类型比较复杂的数组，代码可读性会稍微好一些。

```
let arr:Array<number|string>;
```

这种写法本质上属于泛型，这里只要知道怎么写就可以了，详细解释参见《泛型》一章。另外，数组类型还有第三种写法，因为很少用到，本章就省略了，详见《interface 接口》一章。

数组类型声明了以后，成员数量是不限制的，任意数量的成员都可以，也可以是空数组。

```
let arr:number[];
arr = [];
arr = [1];
arr = [1, 2];
arr = [1, 2, 3];
```

上面示例中，数组`arr`无论有多少个成员，都是正确的。

这种规定的隐藏含义就是，数组的成员是可以动态变化的。

```
let arr:number[] = [1, 2, 3];

arr[3] = 4;
arr.length = 2;

arr // [1, 2]
```

上面示例中，数组增加成员或减少成员，都是可以的。

正是由于成员数量可以动态变化，所以 TypeScript 不会对数组边界进行检查，越界访问数组并不会报错。

```
let arr:number[] = [1, 2, 3];
let foo = arr[3]; // 正确
```

上面示例中，变量`foo`的值是一个不存在的数组成员，TypeScript 并不会报错。

TypeScript 允许使用方括号读取数组成员的类型。

```
type Names = string[];
type Name = Names[0]; // string
```

上面示例中，类型`Names`是字符串数组，那么`Names[0]`返回的类型就是`string`。

由于数组成员的索引类型都是`number`，所以读取成员类型也可以写成下面这样。

```
type Names = string[];
type Name = Names[number]; // string
```

上面示例中，`Names[number]`表示数组`Names`所有数值索引的成员类型，所以返回`string`。

数组的类型推断

如果数组变量没有声明类型，TypeScript 就会推断数组成员的类型。这时，推断行为会因为值的不同，而有所不同。

如果变量的初始值是空数组，那么 TypeScript 会推断数组类型是`any[]`。

```
// 推断为 any[]
const arr = [];
```

后面，为这个数组赋值时，TypeScript 会自动更新类型推断。

```
const arr = [];
arr // 推断为 any[]

arr.push(123);
arr // 推断类型为 number[]

arr.push('abc');
arr // 推断类型为 (string|number)[]
```

上面示例中，数组变量`arr`的初始值是空数组，然后随着新成员的加入，TypeScript 会自动修改推断的数组类型。

但是，类型推断的自动更新只发生初始值为空数组的情况。如果初始值不是空数组，类型推断就不会更新。

```
// 推断类型为 number[]
const arr = [123];

arr.push('abc'); // 报错
```

上面示例中，数组变量`arr`的初始值是`[123]`，TypeScript 就推断成员类型为`number`。新成员如果不是这个类型，TypeScript 就会报错，而不会更新类型推断。

只读数组，const 断言

JavaScript 规定，`const`命令声明的数组变量是可以改变成员的。

```
const arr = [0, 1];
arr[0] = 2;
```

上面示例中，修改`const`命令声明的数组的成员是允许的。

但是，很多时候确实有声明为只读数组的需求，即不允许变动数组成员。

TypeScript 允许声明只读数组，方法是在数组类型前面加上`readonly`关键字。

```
const arr:readonly number[] = [0, 1];

arr[1] = 2; // 报错
arr.push(3); // 报错
delete arr[0]; // 报错
```

上面示例中，`arr`是一个只读数组，删除、修改、新增数组成员都会报错。

TypeScript 将`readonly number[]`与`number[]`视为两种不一样的类型，后者是前者的子类型。

这是因为只读数组没有`pop()`、`push()`之类会改变原数组的方法，所以`number[]`的方法数量要多于`readonly number[]`，这意味着`number[]`其实是`readonly number[]`的子类型。

我们知道，子类型继承了父类型的所有特征，并加上了自己的特征，所以子类型`number[]`可以用于所有使用父类型的场合，反过来就不行。

```
let a1:number[] = [0, 1];
let a2:readonly number[] = a1; // 正确

a1 = a2; // 报错
```

上面示例中，子类型`number[]`可以赋值给父类型`readonly number[]`，但是反过来就会报错。

由于只读数组是数组的父类型，所以它不能代替数组。这一点很容易产生令人困惑的报错。

```
function getSum(s:number[]) {
  // ...
}

const arr:readonly number[] = [1, 2, 3];

getSum(arr) // 报错
```

上面示例中，函数`getSum()`的参数`s`是一个数组，传入只读数组就会报错。原因就是只读数组是数组的父类型，父类型不能替代子类型。这个问题的解决方法是使用类型断言`getSum(arr as number[])`，详见《类型断言》一章。

注意，`readonly`关键字不能与数组的泛型写法一起使用。

```
// 报错
const arr:readonly Array<number> = [0, 1];
```

上面示例中，`readonly`与数组的泛型写法一起使用，就会报错。

实际上，TypeScript 提供了两个专门的泛型，用来生成只读数组的类型。

```
const a1:ReadonlyArray<number> = [0, 1];

const a2:Readonly<number[]> = [0, 1];
```

上面示例中，泛型`ReadonlyArray<T>`和`Readonly<T[]>`都可以用来生成只读数组类型。两者尖括号里面的写法不一样，`Readonly<T[]>`的尖括号里面是整个数组（`number[]`），而`ReadonlyArray<T>`的尖括号里面是数组成员（`number`）。

只读数组还有一种声明方法，就是使用“const 断言”。

```
const arr = [0, 1] as const;

arr[0] = [2]; // 报错
```

上面示例中，`as const`告诉 TypeScript，推断类型时要把变量`arr`推断为只读数组，从而使得数组成员无法改变。

多维数组

TypeScript 使用`T[][]`的形式，表示二维数组，`T`是最底层数组成员的类型。

```
var multi:number[][] =
  [[1,2,3], [23,24,25]];
```

上面示例中，变量`multi`的类型是`number[][]`，表示它是一个二维数组，最底层的数组成员类型是`number`。

TypeScript 的元组类型

简介

元组（tuple）是 TypeScript 特有的数据类型，JavaScript 没有单独区分这种类型。它表示成员类型可以自由设置的数组，即数组的各个成员的类型可以不同。

由于成员的类型可以不一样，所以元组必须明确声明每个成员的类型。

```
const s:[string, string, boolean]
  = ['a', 'b', true];
```

上面示例中，元组`s`的前两个成员的类型是`string`，最后一个成员的类型是`boolean`。

元组类型的写法，与上一章的数组有一个重大差异。数组的成员类型写在方括号外面（`number[]`），元组的成员类型是写在方括号里面（`[number]`）。TypeScript 的区分方法就是，成员类型写在方括号里面的就是元组，写在外面的就是数组。

```
// 数组
let a:number[] = [1];

// 元组
let t:[number] = [1];
```

上面示例中，变量`a`和`t`的值都是`[1]`，但是它们的类型是不一样的。`a`是一个数组，成员类型`number`写在方括号外面；`t`是一个元组，成员类型`number`写在方括号里面。

使用元组时，必须明确给出类型声明（上例的`[number]`），不能省略，否则 TypeScript 会把一个值自动推断为数组。

```
// a 的类型被推断为 (number | boolean)[]
let a = [1, true];
```

上面示例中，变量`a`的值其实是一个元组，但是 TypeScript 会将其推断为一个联合类型的数组，即`a`的类型为`(number | boolean)[]`。所以，元组必须显式给出类型声明。

元组成员的类型可以添加问号后缀（`?`），表示该成员是可选的。

```
let a:[number, number?] = [1];
```

上面示例中，元组`a`的第二个成员是可选的，可以省略。

注意，问号只能用于元组的尾部成员，也就是说，所有可选成员必须在必选成员之后。

```
type myTuple = [
  number,
  number,
  number?,
  string?
];
```

上面示例中，元组`myTuple`的最后两个成员是可选的。也就是说，它的成员数量可能有两个、三个和四个。

由于需要声明每个成员的类型，所以大多数情况下，元组的成员数量是有限的，从类型声明就可以明确知道，元组包含多少个成员，越界的成员会报错。

```
let x:[string, string] = ['a', 'b'];

x[2] = 'c'; // 报错
```

上面示例中，变量`x`是一个只有两个成员的元组，如果对第三个成员赋值就报错了。

但是，使用扩展运算符（`...`），可以表示不限成员数量的元组。

```
type NamedNums = [
  string,
  ...number[]
];

const a:NamedNums = ['A', 1, 2];
const b:NamedNums = ['B', 1, 2, 3];
```

上面示例中，元组类型`NamedNums`的第一个成员是字符串，后面的成员使用扩展运算符来展开一个数组，从而实现了不定数量的成员。

扩展运算符（`...`）用在元组的任意位置都可以，它的后面只能是一个数组或元组。

```
type t1 = [string, number, ...boolean[]];
type t2 = [string, ...boolean[], number];
type t3 = [...boolean[], string, number];
```

上面示例中，扩展运算符分别在元组的尾部、中部和头部，`...`的后面是一个数组`boolean[]`。

如果不确定元组成员的类型和数量，可以写成下面这样。

```
type Tuple = [...any[]];
```

上面示例中，元组`Tuple`可以放置任意数量和类型的成员。但是这样写，也就失去了使用元组和 TypeScript 的意义。

元组的成员可以添加成员名，这个成员名是说明性的，可以任意取名，没有实际作用。

```
type Color = [
  red: number,
  green: number,
  blue: number
];

const c:Color = [255, 255, 255];
```

上面示例中，类型`Color`是一个元组，它有三个成员。每个成员都有一个名字，写在具体类型的前面，使用冒号分隔。这几个名字可以随便取，没有实际作用，只是用来说明每个成员的含义。

元组可以通过方括号，读取成员类型。

```
type Tuple = [string, number];
type Age = Tuple[1]; // number
```

上面示例中，`Tuple[1]`返回1号位置的成员类型。

由于元组的成员都是数值索引，即索引类型都是`number`，所以可以像下面这样读取。

```
type Tuple = [string, number, Date];
type TupleEl = Tuple[number];  // string|number|Date
```

上面示例中，`Tuple[number]`表示元组`Tuple`的所有数值索引的成员类型，所以返回`string|number|Date`，即这个类型是三种值的联合类型。

只读元组

元组也可以是只读的，不允许修改，有两种写法。

```
// 写法一
type t = readonly [number, string]

// 写法二
type t = Readonly<[number, string]>
```

上面示例中，两种写法都可以得到只读元组，其中写法二是一个泛型，用到了工具类型`Readonly<T>`。

跟数组一样，只读元组是元组的父类型。所以，元组可以替代只读元组，而只读元组不能替代元组。

```
type t1 = readonly [number, number];
type t2 = [number, number];

let x:t2 = [1, 2];
let y:t1 = x; // 正确

x = y; // 报错
```

上面示例中，类型`t1`是只读元组，类型`t2`是普通元组。`t2`类型可以赋值给`t1`类型，反过来就会报错。

由于只读元组不能替代元组，所以会产生一些令人困惑的报错。

```
function distanceFromOrigin([x, y]:[number, number]) {
  return Math.sqrt(x**2 + y**2);
}

let point = [3, 4] as const;

distanceFromOrigin(point); // 报错
```

上面示例中，函数`distanceFromOrigin()`的参数是一个元组，传入只读元组就会报错，因为只读元组不能替代元组。

读者可能注意到了，上例中`[3, 4] as const`的写法，在上一章讲到，生成的是只读数组，其实生成的同时也是只读元组。因为它生成的实际上是一个只读的“值类型”`readonly [3, 4]`，把它解读成只读数组或只读元组都可以。

上面示例报错的解决方法，就是使用类型断言，在最后一行将传入的参数断言为普通元组，详见《类型断言》一章。

```
distanceFromOrigin(
  point as [number, number]
)
```

成员数量的推断

如果没有可选成员和扩展运算符，TypeScript 会推断出元组的成员数量（即元组长度）。

```
function f(point: [number, number]) {
  if (point.length === 3) {  // 报错
    // ...
  }
}
```

上面示例会报错，原因是 TypeScript 发现元组`point`的长度是`2`，不可能等于`3`，这个判断无意义。

如果包含了可选成员，TypeScript 会推断出可能的成员数量。

```
function f(
  point:[number, number?, number?]
) {
  if (point.length === 4) {  // 报错
    // ...
  }
}
```

上面示例会报错，原因是 TypeScript 发现`point.length`的类型是`1|2|3`，不可能等于`4`。

如果使用了扩展运算符，TypeScript 就无法推断出成员数量。

```
const myTuple:[...string[]]
  = ['a', 'b', 'c'];

if (myTuple.length === 4) { // 正确
  // ...
}
```

上面示例中，`myTuple`只有三个成员，但是 TypeScript 推断不出它的成员数量，因为它的类型用到了扩展运算符，TypeScript 把`myTuple`当成数组看待，而数组的成员数量是不确定的。

一旦扩展运算符使得元组的成员数量无法推断，TypeScript 内部就会把该元组当成数组处理。

扩展运算符与成员数量

扩展运算符（`...`）将数组（注意，不是元组）转换成一个逗号分隔的序列，这时 TypeScript 会认为这个序列的成员数量是不确定的，因为数组的成员数量是不确定的。

这导致如果函数调用时，使用扩展运算符传入函数参数，可能发生参数数量与数组长度不匹配的报错。

```
const arr = [1, 2];

function add(x:number, y:number){
  // ...
}

add(...arr) // 报错
```

上面示例会报错，原因是函数`add()`只能接受两个参数，但是传入的是`...arr`，TypeScript 认为转换后的参数个数是不确定的。

有些函数可以接受任意数量的参数，这时使用扩展运算符就不会报错。

```
const arr = [1, 2, 3];
console.log(...arr) // 正确
```

上面示例中，`console.log()`可以接受任意数量的参数，所以传入`...arr`就不会报错。

解决这个问题的一个方法，就是把成员数量不确定的数组，写成成员数量确定的元组，再使用扩展运算符。

```
const arr:[number, number] = [1, 2];

function add(x:number, y:number){
  // ...
}
```

```
add(...arr) // 正确
```

上面示例中，`arr`是一个拥有两个成员的元组，所以 TypeScript 能够确定`...arr`可以匹配函数`add()`的参数数量，就不会报错了。

另一种写法是使用`as const`断言。

```
const arr = [1, 2] as const;
```

上面这种写法也可以，因为 TypeScript 会认为`arr`的类型是`readonly [1, 2]`，这是一个只读的值类型，可以当作数组，也可以当作元组。

TypeScript 的 symbol 类型

简介

Symbol 是 ES2015 新引入的一种原始类型的值。它类似于字符串，但是每一个 Symbol 值都是独一无二的，与其他任何值都不相等。

Symbol 值通过`Symbol()`函数生成。在 TypeScript 里面，Symbol 的类型使用`symbol`表示。

```
let x:symbol = Symbol();
let y:symbol = Symbol();

x === y // false
```

上面示例中，变量`x`和`y`的类型都是`symbol`，且都用`Symbol()`生成，但是它们是不相等的。

unique symbol

`symbol`类型包含所有的 Symbol 值，但是无法表示某一个具体的 Symbol 值。

比如，`5`是一个具体的数值，就用`5`这个字面量来表示，这也是它的值类型。但是，Symbol 值不存在字面量，必须通过变量来引用，所以写不出只包含单个 Symbol 值的那种值类型。

为了解决这个问题，TypeScript 设计了`symbol`的一个子类型`unique symbol`，它表示单个的、某个具体的 Symbol 值。

因为`unique symbol`表示单个值，所以这个类型的变量是不能修改值的，只能用`const`命令声明，不能用`let`声明。

```
// 正确
const x:unique symbol = Symbol();

// 报错
let y:unique symbol = Symbol();
```

上面示例中，`let`命令声明的变量，不能是`unique symbol`类型，会报错。

`const`命令为变量赋值 Symbol 值时，变量类型默认就是`unique symbol`，所以类型可以省略不写。

```
const x:unique symbol = Symbol();
// 等同于
const x = Symbol();
```

每个声明为`unique symbol`类型的变量，它们的值都是不一样的，其实属于两个值类型。

```
const a:unique symbol = Symbol();
const b:unique symbol = Symbol();

a === b // 报错
```

上面示例中，变量`a`和变量`b`的类型虽然都是`unique symbol`，但其实是两个值类型。不同类型的值肯定是不相等的，所以最后一行就报错了。

由于 Symbol 类似于字符串，可以参考下面的例子来理解。

```
const a:'hello' = 'hello';
const b:'world' = 'world';

a === b // 报错
```

上面示例中，变量`a`和`b`都是字符串，但是属于不同的值类型，不能使用严格相等运算符进行比较。

而且，由于变量`a`和`b`是两个类型，就不能把一个赋值给另一个。

```
const a:unique symbol = Symbol();
const b:unique symbol = a; // 报错
```

上面示例中，变量`a`和变量`b`的类型都是`unique symbol`，但是其实类型不同，所以把`a`赋值给`b`会报错。

上例变量`b`的类型，如果要写成与变量`a`同一个`unique symbol`值类型，只能写成类型为`typeof a`。

```
const a:unique symbol = Symbol();
const b:typeof a = a; // 正确
```

不过我们知道，相同参数的`Symbol.for()`方法会返回相同的 Symbol 值。TypeScript 目前无法识别这种情况，所以可能出现多个 unique symbol 类型的变量，等于同一个 Symbol 值的情况。

```
const a:unique symbol = Symbol.for('foo');
const b:unique symbol = Symbol.for('foo');
```

上面示例中，变量`a`和`b`是两个不同的值类型，但是它们的值其实是相等的。

unique symbol 类型是 symbol 类型的子类型，所以可以将前者赋值给后者，但是反过来就不行。

```
const a:unique symbol = Symbol();

const b:symbol = a; // 正确

const c:unique symbol = b; // 报错
```

上面示例中，unique symbol 类型（变量`a`）赋值给 symbol 类型（变量`b`）是可以的，但是 symbol 类型（变量`b`）赋值给 unique symbol 类型（变量`c`）会报错。

unique symbol 类型的一个作用，就是用作属性名，这可以保证不会跟其他属性名冲突。在 5.8.3 版本之前，如果要把某一个特定的 Symbol 值当作属性名，TypeScript 只允许它的类型是 unique symbol，不能是 symbol，5.8.3 版本之后没有了这个限制。

```
const x:unique symbol = Symbol();
const y:symbol = Symbol();

interface Foo {
  [x]: string; // 正确
  [y]: string; // 5.8.3 版本之前报错
}
```

上面示例中，变量`y`当作属性名，但是`y`的类型是 symbol，不是固定不变的值，5.8.3 版本之前会报错，之后就不会。

`unique symbol`类型也可以用作类（class）的属性值，但只能赋值给类的`readonly static`属性。

```
class C {
  static readonly foo:unique symbol = Symbol();
}
```

上面示例中，静态只读属性`foo`的类型就是`unique symbol`。注意，这时`static`和`readonly`两个限定符缺一不可，这是为了保证这个属性是固定不变的。

类型推断

如果变量声明时没有给出类型，TypeScript 会推断某个 Symbol 值变量的类型。

`let`命令声明的变量，推断类型为 symbol。

```
// 类型为 symbol
let x = Symbol();
```

`const`命令声明的变量，推断类型为 unique symbol。

```
// 类型为 unique symbol
const x = Symbol();
```

但是，`const`命令声明的变量，如果赋值为另一个 symbol 类型的变量，则推断类型为 symbol。

```
let x = Symbol();

// 类型为 symbol
const y = x;
```

`let`命令声明的变量，如果赋值为另一个 unique symbol 类型的变量，则推断类型还是 symbol。

```
const x = Symbol();

// 类型为 symbol
let y = x;
```

TypeScript 的函数类型

简介

函数的类型声明，需要在声明函数时，给出参数的类型和返回值的类型。

```
function hello(
  txt:string
):void {
  console.log('hello ' + txt);
}
```

上面示例中，函数`hello()`在声明时，需要给出参数`txt`的类型（string），以及返回值的类型（`void`），后者写在参数列表的圆括号后面。`void`类型表示没有返回值，详见后文。

如果不指定参数类型（比如上例不写`txt`的类型），TypeScript 就会推断参数类型，如果缺乏足够信息，就会推断该参数的类型为`any`。

返回值的类型通常可以不写，因为 TypeScript 自己会推断出来。

```
function hello(txt:string) {
  console.log('hello ' + txt);
}
```

上面示例中，由于没有`return`语句，TypeScript 会推断出函数`hello()`没有返回值。

不过，有时候出于文档目的，或者为了防止不小心改掉返回值，还是会写返回值的类型。

如果变量被赋值为一个函数，变量的类型有两种写法。

```
// 写法一
const hello = function (txt:string) {
  console.log('hello ' + txt);
}

// 写法二
const hello:
  (txt:string) => void
= function (txt) {
  console.log('hello ' + txt);
};
```

上面示例中，变量`hello`被赋值为一个函数，它的类型有两种写法。写法一是通过等号右边的函数类型，推断出变量`hello`的类型；写法二则是使用箭头函数的形式，为变量`hello`指定类型，参数的类型写在箭头左侧，返回值的类型写在箭头右侧。

写法二有两个地方需要注意。

首先，函数的参数要放在圆括号里面，不放会报错。

其次，类型里面的参数名（本例是`txt`）是必须的。有的语言的函数类型可以不写参数名（比如 C 语言），但是 TypeScript 不行。如果写成`(string) => void`，TypeScript 会理解成函数有一个名叫 string 的参数，并且这个`string`参数的类型是`any`。

```
type MyFunc = (string, number) => number;
// (string: any, number: any) => number
```

上面示例中，函数类型没写参数名，导致 TypeScript 认为参数类型都是`any`。

函数类型里面的参数名与实际参数名，可以不一致。

```
let f:(x:number) => number;

f = function (y:number) {
  return y;
};
```

上面示例中，函数类型里面的参数名为`x`，实际的函数定义里面，参数名为`y`，两者并不相同。

如果函数的类型定义很冗长，或者多个函数使用同一种类型，写法二用起来就很麻烦。因此，往往用`type`命令为函数类型定义一个别名，便于指定给其他变量。

```
type MyFunc = (txt:string) => void;

const hello:MyFunc = function (txt) {
  console.log('hello ' + txt);
};
```

上面示例中，`type`命令为函数类型定义了一个别名`MyFunc`，后面使用就很方便，变量可以指定为这个类型。

函数的实际参数个数，可以少于类型指定的参数个数，但是不能多于，即 TypeScript 允许省略参数。

```
let myFunc:
  (a:number, b:number) => number;

myFunc = (a:number) => a; // 正确

myFunc = (
  a:number, b:number, c:number
) => a + b + c; // 报错
```

上面示例中，变量`myFunc`的类型只能接受两个参数，如果被赋值为只有一个参数的函数，并不报错。但是，被赋值为有三个参数的函数，就会报错。

这是因为 JavaScript 函数在声明时往往有多余的参数，实际使用时可以只传入一部分参数。比如，数组的`forEach()`方法的参数是一个函数，该函数默认有三个参数`(item, index, array) => void`，实际上往往只使用第一个参数`(item) => void`。因此，TypeScript 允许函数传入的参数不足。

```
let x = (a:number) => 0;
let y = (b:number, s:string) => 0;

y = x; // 正确
x = y; // 报错
```

上面示例中，函数`x`只有一个参数，函数`y`有两个参数，`x`可以赋值给`y`，反过来就不行。

如果一个变量要套用另一个函数类型，有一个小技巧，就是使用`typeof`运算符。

```
function add(
  x:number,
  y:number
) {
  return x + y;
}

const myAdd:typeof add = function (x, y) {
  return x + y;
}
```

上面示例中，函数`myAdd()`的类型与函数`add()`是一样的，那么就可以定义成`typeof add`。因为函数名`add`本身不是类型，而是一个值，所以要用`typeof`运算符返回它的类型。

这是一个很有用的技巧，任何需要类型的地方，都可以使用`typeof`运算符从一个值获取类型。

函数类型还可以采用对象的写法。

```
let add:{
  (x:number, y:number):number
};

add = function (x, y) {
  return x + y;
};
```

上面示例中，变量`add`的类型就写成了一个对象。

函数类型的对象写法如下。

```
{
  (参数列表): 返回值
}
```

注意，这种写法的函数参数与返回值之间，间隔符是冒号`:`，而不是正常写法的箭头`=>`，因为这里采用的是对象类型的写法，对象的属性名与属性值之间使用的是冒号。

这种写法平时很少用，但是非常合适用在一个场合：函数本身存在属性。

```
function f(x:number) {
  console.log(x);
}

f.version = '1.0';
```

上面示例中，函数`f()`本身还有一个属性`version`。这时，`f`完全就是一个对象，类型就要使用对象的写法。

```
let foo: {
  (x:number): void;
  version: string
} = f;
```

函数类型也可以使用 Interface 来声明，这种写法就是对象写法的翻版，详见《Interface》一章。

```
interface myfn {
  (a:number, b:number): number;
}

var add:myfn = (a, b) => a + b;
```

上面示例中，interface 命令定义了接口`myfn`，这个接口的类型就是一个用对象表示的函数。

Function 类型

TypeScript 提供 Function 类型表示函数，任何函数都属于这个类型。

```
function doSomething(f:Function) {
  return f(1, 2, 3);
}
```

上面示例中，参数`f`的类型就是`Function`，代表这是一个函数。

Function 类型的值都可以直接执行。

Function 类型的函数可以接受任意数量的参数，每个参数的类型都是`any`，返回值的类型也是`any`，代表没有任何约束，所以不建议使用这个类型，给出函数详细的类型声明会更好。

箭头函数

箭头函数是普通函数的一种简化写法，它的类型写法与普通函数类似。

```
const repeat = (
  str:string,
```

```
  times:number
):string => str.repeat(times);
```

上面示例中，变量`repeat`被赋值为一个箭头函数，类型声明写在箭头函数的定义里面。其中，参数的类型写在参数名后面，返回值类型写在参数列表的圆括号后面。

注意，类型写在箭头函数的定义里面，与使用箭头函数表示函数类型，写法有所不同。

```
function greet(
  fn:(a:string) => void
):void {
  fn('world');
}
```

上面示例中，函数`greet()`的参数`fn`是一个函数，类型就用箭头函数表示。这时，`fn`的返回值类型要写在箭头右侧，而不是写在参数列表的圆括号后面。

下面再看一个例子。

```
type Person = { name: string };

const people = ['alice', 'bob', 'jan'].map(
  (name):Person => ({name})
);
```

上面示例中，`Person`是一个类型别名，代表一个对象，该对象有属性`name`。变量`people`是数组的`map()`方法的返回值。

`map()`方法的参数是一个箭头函数`(name):Person => ({name})`，该箭头函数的参数`name`的类型省略了，因为可以从`map()`的类型定义推断出来，箭头函数的返回值类型为`Person`。相应地，变量`people`的类型是`Person[]`。

至于箭头后面的`({name})`，表示返回一个对象，该对象有一个属性`name`，它的属性值为变量`name`的值。这里的圆括号是必须的，否则`(name):Person => {name}`的大括号表示函数体，即函数体内有一行语句`name`，同时由于没有`return`语句，这个函数不会返回任何值。

注意，下面两种写法都是不对的。

```
// 错误
(name:Person) => ({name})

// 错误
name:Person => ({name})
```

上面的两种写法在本例中都是错的。第一种写法表示，箭头函数的参数`name`的类型是`Person`，同时没写函数返回值的类型，让 TypeScript 自己去推断。第二种写法中，函数参数缺少圆括号。

可选参数

如果函数的某个参数可以省略，则在参数名后面加问号表示。

```
function f(x?:number) {
  // ...
}

f(); // OK
f(10); // OK
```

上面示例中，参数`x`后面有问号，表示该参数可以省略。

参数名带有问号，表示该参数的类型实际上是`原始类型|undefined`，它有可能为`undefined`。比如，上例的`x`虽然类型声明为`number`，但是实际上是`number|undefined`。

```
function f(x?:number) {
  return x;
}

f(undefined) // 正确
```

上面示例中，参数`x`是可选的，等同于说`x`可以赋值为`undefined`。

但是，反过来就不成立，类型显式设为`undefined`的参数，就不能省略。

```
function f(x:number|undefined) {
  return x;
}

f() // 报错
```

上面示例中，参数`x`的类型是`number|undefined`，表示要么传入一个数值，要么传入`undefined`，如果省略这个参数，就会报错。

函数的可选参数只能在参数列表的尾部，跟在必选参数的后面。

```
let myFunc:
  (a?:number, b:number) => number; // 报错
```

上面示例中，可选参数在必选参数前面，就报错了。

如果前部参数有可能为空，这时只能显式注明该参数类型可能为`undefined`。

```
let myFunc:
  (
    a:number|undefined,
    b:number
  ) => number;
```

上面示例中，参数`a`有可能为空，就只能显式注明类型包括`undefined`，传参时也要显式传入`undefined`。

函数体内部用到可选参数时，需要判断该参数是否为`undefined`。

```
let myFunc:
  (a:number, b?:number) => number;

myFunc = function (x, y) {
  if (y === undefined) {
    return x;
  }
  return x + y;
}
```

上面示例中，由于函数的第二个参数为可选参数，所以函数体内部需要判断一下，该参数是否为空。

参数默认值

TypeScript 函数的参数默认值写法，与 JavaScript 一致。

设置了默认值的参数，就是可选的。如果不传入该参数，它就会等于默认值。

```
function createPoint(
  x:number = 0,
  y:number = 0
):[number, number] {
  return [x, y];
}

createPoint() // [0, 0]
```

上面示例中，参数`x`和`y`的默认值都是`0`，调用`createPoint()`时，这两个参数都是可以省略的。这里其实可以省略`x`和`y`的类型声明，因为可以从默认值推断出来。

```
function createPoint(
  x = 0, y = 0
) {
  return [x, y];
}
```

可选参数与默认值不能同时使用。

```
// 报错
function f(x?: number = 0) {
  // ...
}
```

上面示例中，`x`是可选参数，还设置了默认值，结果就报错了。

设有默认值的参数，如果传入`undefined`，也会触发默认值。

```
function f(x = 456) {
  return x;
}

f(undefined) // 456
```

具有默认值的参数如果不位于参数列表的末尾，调用时不能省略，如果要触发默认值，必须显式传入`undefined`。

```
function add(
  x:number = 0,
  y:number
) {
  return x + y;
}

add(1) // 报错
add(undefined, 1) // 正确
```

参数解构

函数参数如果存在变量解构，类型写法如下。

```
function f(
  [x, y]: [number, number]
) {
  // ...
}

function sum(
  { a, b, c }: {
     a: number;
     b: number;
     c: number
  }
) {
  console.log(a + b + c);
}
```

参数解构可以结合类型别名（type 命令）一起使用，代码会看起来简洁一些。

```
type ABC = { a:number; b:number; c:number };

function sum({ a, b, c }:ABC) {
  console.log(a + b + c);
}
```

rest 参数

rest 参数表示函数剩余的所有参数，它可以是数组（剩余参数类型相同），也可能是元组（剩余参数类型不同）。

```
// rest 参数为数组
function joinNumbers(...nums:number[]) {
  // ...
}

// rest 参数为元组
function f(...args:[boolean, number]) {
  // ...
}
```

注意，元组需要声明每一个剩余参数的类型。如果元组里面的参数是可选的，则要使用可选参数。

```
function f(
  ...args: [boolean, string?]
) {}
```

下面是一个 rest 参数的例子。

```
function multiply(n:number, ...m:number[]) {
  return m.map((x) => n * x);
}
```

上面示例中，参数`m`就是 rest 类型，它的类型是一个数组。

rest 参数甚至可以嵌套。

```
function f(...args:[boolean, ...string[]]) {
  // ...
}
```

rest 参数可以与变量解构结合使用。

```
function repeat(
  ...[str, times]: [string, number]
):string {
  return str.repeat(times);
}

// 等同于
function repeat(
  str: string,
  times: number
):string {
  return str.repeat(times);
}
```

readonly 只读参数

如果函数内部不能修改某个参数，可以在函数定义时，在参数类型前面加上`readonly`关键字，表示这是只读参数。

```
function arraySum(
  arr:readonly number[]
) {
  // ...
  arr[0] = 0; // 报错
}
```

上面示例中，参数`arr`的类型是`readonly number[]`，表示为只读参数。如果函数体内部修改这个数组，就会报错。

注意，`readonly`关键字目前只允许用在数组和元组类型的参数前面，如果用在其他类型的参数前面，就会报错。

void 类型

void 类型表示函数没有返回值。

```
function f():void {
  console.log('hello');
}
```

上面示例中，函数`f`没有返回值，类型就要写成`void`。

如果返回其他值，就会报错。

```
function f():void {
  return 123; // 报错
}
```

上面示例中，函数`f()`的返回值类型是`void`，但是实际返回了一个数值，编译时就报错了。

void 类型允许返回`undefined`或`null`。

```
function f():void {
  return undefined; // 正确
}

function f():void {
  return null; // 正确
}
```

如果打开了`strictNullChecks`编译选项，那么 void 类型只允许返回`undefined`。如果返回`null`，就会报错。这是因为 JavaScript 规定，如果函数没有返回值，就等同于返回`undefined`。

```
// 打开编译选项 strictNullChecks

function f():void {
  return undefined; // 正确
}

function f():void {
  return null; // 报错
}
```

需要特别注意的是，如果变量、对象方法、函数参数是一个返回值为 void 类型的函数，那么并不代表不能赋值为有返回值的函数。恰恰相反，该变量、对象方法和函数参数可以接受返回任意值的函数，这时并不会报错。

```
type voidFunc = () => void;

const f:voidFunc = () => {
  return 123;
};
```

上面示例中，变量`f`的类型是`voidFunc`，是一个没有返回值的函数。但是实际上，`f`的值可以是一个有返回值的函数（返回`123`），编译时不会报错。

这是因为，这时 TypeScript 认为，这里的 void 类型只是表示该函数的返回值没有利用价值，或者说不应该使用该函数的返回值。只要不用到这里的返回值，就不会报错。

这样设计是有现实意义的。举例来说，数组方法`Array.prototype.forEach(fn)`的参数`fn`是一个函数，而且这个函数应该没有返回值，即返回值类型是`void`。

但是，实际应用中，很多时候传入的函数是有返回值，但是它的返回值不重要，或者不产生作用。

```
const src = [1, 2, 3];
const ret = [];

src.forEach(el => ret.push(el));
```

上面示例中，`push()`有返回值，表示插入新元素后数组的长度。但是，对于`forEach()`方法来说，这个返回值是没有作用的，根本用不到，所以 TypeScript 不会报错。

如果后面使用了这个函数的返回值，就违反了约定，则会报错。

```
type voidFunc = () => void;

const f:voidFunc = () => {
  return 123;
};

f() * 2 // 报错
```

上面示例中，最后一行报错了，因为根据类型声明，`f()`没有返回值，但是却用到了它的返回值，因此报错了。

注意，这种情况仅限于变量、对象方法和函数参数，函数字面量如果声明了返回值是 void 类型，还是不能有返回值。

```
function f():void {
  return true; // 报错
}

const f3 = function ():void {
  return true; // 报错
};
```

上面示例中，函数字面量声明了返回`void`类型，这时只要有返回值（除了`undefined`和`null`）就会报错。

函数的运行结果如果是抛出错误，也允许将返回值写成`void`。

```
function throwErr():void {
  throw new Error('something wrong');
}
```

上面示例中，函数`throwErr()`会抛出错误，返回值类型写成`void`是允许的。

除了函数，其他变量声明为`void`类型没有多大意义，因为这时只能赋值为`undefined`或者`null`（假定没有打开`strictNullChecks`）。

```
let foo:void = undefined;

// 没有打开 strictNullChecks 的情况下
let bar:void = null;
```

never 类型

`never`类型表示肯定不会出现的值。它用在函数的返回值，就表示某个函数肯定不会返回值，即函数不会正常执行结束。

它主要有以下两种情况。

（1）抛出错误的函数。

```
function fail(msg:string):never {
  throw new Error(msg);
}
```

上面示例中，函数`fail()`会抛出错误，不会正常退出，所以返回值类型是`never`。

注意，只有抛出错误，才是 never 类型。如果显式用`return`语句返回一个 Error 对象，返回值就不是 never 类型。

```
function fail():Error {
  return new Error("Something failed");
}
```

上面示例中，函数`fail()`返回一个 Error 对象，所以返回值类型是 Error。

另外，由于抛出错误的情况属于`never`类型或`void`类型，所以无法从返回值类型中获知，抛出的是哪一种错误。

（2）无限执行的函数。

```
const sing = function():never {
  while (true) {
    console.log('sing');
  }
};
```

上面示例中，函数`sing()`会永远执行，不会返回，所以返回值类型是`never`。

注意，`never`类型不同于`void`类型。前者表示函数没有执行结束，不可能有返回值；后者表示函数正常执行结束，但是不返回值，或者说返回`undefined`。

```
// 正确
function sing():void {
  console.log('sing');
}

// 报错
function sing():never {
  console.log('sing');
}
```

上面示例中，函数`sing()`虽然没有`return`语句，但实际上是省略了`return undefined`这行语句，真实的返回值是`undefined`。所以，它的返回值类型要写成`void`，而不是`never`，写成`never`会报错。

如果一个函数抛出了异常或者陷入了死循环，那么该函数无法正常返回一个值，因此该函数的返回值类型就是`never`。如果程序中调用了一个返回值类型为`never`的函数，那么就意味着程序会在该函数的调用位置终止，永远不会继续执行后续的代码。

```
function neverReturns():never {
  throw new Error();
}

function f(
  x:string|undefined
) {
  if (x === undefined) {
```

```
    neverReturns();
  }

  x; // 推断为 string
}
```

上面示例中，函数`f()`的参数`x`的类型为`string|undefined`。但是，`x`类型为`undefined`时，调用了`neverReturns()`。这个函数不会返回，因此 TypeScript 可以推断出，判断语句后面的那个`x`，类型一定是`string`。

一个函数如果某些条件下有正常返回值，另一些条件下抛出错误，这时它的返回值类型可以省略`never`。

```
function sometimesThrow():number {
  if (Math.random() > 0.5) {
    return 100;
  }

  throw new Error('Something went wrong');
}

const result = sometimesThrow();
```

上面示例中，函数`sometimesThrow()`的返回值其实是`number|never`，但是一般都写成`number`，包括最后一行的变量`result`的类型，也是被推断为`number`。

原因是前面章节提到过，`never`是 TypeScript 的唯一一个底层类型，所有其他类型都包括了`never`。从集合论的角度看，`number|never`等同于`number`。这也提示我们，函数的返回值无论是什么类型，都可能包含了抛出错误的情况。

局部类型

函数内部允许声明其他类型，该类型只在函数内部有效，称为局部类型。

```
function hello(txt:string) {
  type message = string;
  let newTxt:message = 'hello ' + txt;
  return newTxt;
}

const newTxt:message = hello('world'); // 报错
```

上面示例中，类型`message`是在函数`hello()`内部定义的，只能在函数内部使用。在函数外部使用，就会报错。

高阶函数

一个函数的返回值还是一个函数，那么前一个函数就称为高阶函数（higher-order function）。

下面就是一个例子，箭头函数返回的还是一个箭头函数。

```
(someValue: number) => (multiplier: number) => someValue * multiplier;
```

函数重载

有些函数可以接受不同类型或不同个数的参数，并且根据参数的不同，会有不同的函数行为。这种根据参数类型不同，执行不同逻辑的行为，称为函数重载（function overload）。

```
reverse('abc') // 'cba'
reverse([1, 2, 3]) // [3, 2, 1]
```

上面示例中，函数`reverse()`可以将参数颠倒输出。参数可以是字符串，也可以是数组。

这意味着，该函数内部有处理字符串和数组的两套逻辑，根据参数类型的不同，分别执行对应的逻辑。这就叫“函数重载”。

TypeScript 对于“函数重载”的类型声明方法是，逐一定义每一种情况的类型。

```
function reverse(str:string):string;
function reverse(arr:any[]):any[];
```

上面示例中，分别对函数`reverse()`的两种参数情况，给予了类型声明。但是，到这里还没有结束，后面还必须对函数`reverse()`给予完整的类型声明。

```
function reverse(str:string):string;
function reverse(arr:any[]):any[];
function reverse(
  stringOrArray:string|any[]
):string|any[] {
  if (typeof stringOrArray === 'string')
    return stringOrArray.split('').reverse().join('');
  else
    return stringOrArray.slice().reverse();
}
```

上面示例中，前两行类型声明列举了重载的各种情况。第三行是函数本身的类型声明，它必须与前面已有的重载声明兼容。

有一些编程语言允许不同的函数参数，对应不同的函数实现。但是，JavaScript 函数只能有一个实现，必须在这个实现当中，处理不同的参数。因此，函数体内部就需要判断参数的类型及个数，并根据判断结果执行不同的操作。

```
function add(
  x:number,
  y:number
):number;
```

```
function add(
  x:any[],
  y:any[]
):any[];
function add(
  x:number|any[],
  y:number|any[]
):number|any[] {
  if (typeof x === 'number' && typeof y === 'number') {
    return x + y;
  } else if (Array.isArray(x) && Array.isArray(y)) {
    return [...x, ...y];
  }

  throw new Error('wrong parameters');
}
```

上面示例中，函数`add()`内部使用`if`代码块，分别处理参数的两种情况。

注意，重载的各个类型描述与函数的具体实现之间，不能有其他代码，否则报错。

另外，虽然函数的具体实现里面，有完整的类型声明。但是，函数实际调用的类型，以前面的类型声明为准。比如，上例的函数实现，参数类型和返回值类型都是`number|any[]`，但不意味着参数类型为`number`时返回值类型为`any[]`。

函数重载的每个类型声明之间，以及类型声明与函数实现的类型之间，不能有冲突。

```
// 报错
function fn(x:boolean):void;
function fn(x:string):void;
function fn(x:number|string) {
  console.log(x);
}
```

上面示例中，函数重载的类型声明与函数实现是冲突的，导致报错。

重载声明的排序很重要，因为 TypeScript 是按照顺序进行检查的，一旦发现符合某个类型声明，就不再往下检查了，所以类型最宽的声明应该放在最后面，防止覆盖其他类型声明。

```
function f(x:any):number;
function f(x:string): 0|1;
function f(x:any):any {
  // ...
}

const a:0|1 = f('hi'); // 报错
```

上面声明中，第一行类型声明`x:any`范围最宽，导致函数`f()`的调用都会匹配这行声明，无法匹配第二行类型声明，所以最后一行调用就报错了，因为等号两侧类型不匹配，左侧类型是`0|1`，右侧类型是`number`。这个函数重载的正确顺序是，第二行类型声明放到第一行的位置。

对象的方法也可以使用重载。

```
class StringBuilder {
  #data = '';

  add(num:number): this;
  add(bool:boolean): this;
  add(str:string): this;
  add(value:any): this {
    this.#data += String(value);
    return this;
  }

  toString() {
    return this.#data;
  }
}
```

上面示例中，方法`add()`也使用了函数重载。

函数重载也可以用来精确描述函数参数与返回值之间的对应关系。

```
function createElement(
  tag:'a'
):HTMLAnchorElement;
function createElement(
  tag:'canvas'
):HTMLCanvasElement;
function createElement(
  tag:'table'
):HTMLTableElement;
function createElement(
  tag:string
):HTMLElement {
  // ...
}
```

上面示例中，函数重载精确描述了参数`tag`的三个值，所对应的不同的函数返回值。

这个示例的函数重载，也可以用对象表示。

```
type CreateElement = {
  (tag:'a'): HTMLAnchorElement;
  (tag:'canvas'): HTMLCanvasElement;
  (tag:'table'): HTMLTableElement;
  (tag:string): HTMLElement;
}
```

由于重载是一种比较复杂的类型声明方法，为了降低复杂性，一般来说，如果可以的话，应该优先使用联合类型替代函数重载，除非多个参数之间、或者某个参数与返回值之间，存在对应关系。

```
// 写法一
function len(s:string):number;
function len(arr:any[]):number;
function len(x:any):number {
  return x.length;
}

// 写法二
function len(x:any[]|string):number {
  return x.length;
}
```

上面示例中，写法二使用联合类型，要比写法一的函数重载简单很多。

构造函数

JavaScript 语言使用构造函数，生成对象的实例。

构造函数的最大特点，就是必须使用`new`命令调用。

```
const d = new Date();
```

上面示例中，`Date()`就是一个构造函数，使用`new`命令调用，返回 Date 对象的实例。

构造函数的类型写法，就是在参数列表前面加上`new`命令。

```
class Animal {
  numLegs:number = 4;
}

type AnimalConstructor = new () => Animal;

function create(c:AnimalConstructor):Animal {
  return new c();
}

const a = create(Animal);
```

上面示例中，类型`AnimalConstructor`就是一个构造函数，而函数`create()`需要传入一个构造函数。在 JavaScript 中，类（class）本质上是构造函数，所以`Animal`这个类可以传入`create()`。

构造函数还有另一种类型写法，就是采用对象形式。

```
type F = {
  new (s:string): object;
};
```

上面示例中，类型 F 就是一个构造函数。类型写成一个可执行对象的形式，并且在参数列表前面要加上`new`命令。

某些函数既是构造函数，又可以当作普通函数使用，比如`Date()`。这时，类型声明可以写成下面这样。

```
type F = {
  new (s:string): object;
  (n?:number): number;
}
```

上面示例中，F 既可以当作普通函数执行，也可以当作构造函数使用。

TypeScript 的对象类型

简介

除了原始类型，对象是 JavaScript 最基本的数据结构。TypeScript 对于对象类型有很多规则。

对象类型的最简单声明方法，就是使用大括号表示对象，在大括号内部声明每个属性和方法的类型。

```
const obj:{
  x:number;
  y:number;
} = { x: 1, y: 1 };
```

上面示例中，对象`obj`的类型就写在变量名后面，使用大括号描述，内部声明每个属性的属性名和类型。

属性的类型可以用分号结尾，也可以用逗号结尾。

```
// 属性类型以分号结尾
type MyObj = {
  x:number;
  y:number;
};

// 属性类型以逗号结尾
type MyObj = {
  x:number,
  y:number,
};
```

最后一个属性后面，可以写分号或逗号，也可以不写。

一旦声明了类型，对象赋值时，就不能缺少指定的属性，也不能有多余的属性。

```
type MyObj = {
  x:number;
  y:number;
};

const o1:MyObj = { x: 1 }; // 报错
const o2:MyObj = { x: 1, y: 1, z: 1 }; // 报错
```

上面示例中，变量`o1`缺少了属性`y`，变量`o2`多出了属性`z`，都会报错。

读写不存在的属性也会报错。

```
const obj:{
  x:number;
  y:number;
```

```
} = { x: 1, y: 1 };

console.log(obj.z); // 报错
obj.z = 1; // 报错
```

上面示例中，读写不存在的属性`z`都会报错。

同样地，也不能删除类型声明中存在的属性，修改属性值是可以的。

```
const myUser = {
  name: "Sabrina",
};

delete myUser.name // 报错
myUser.name = "Cynthia"; // 正确
```

上面声明中，删除类型声明中存在的属性`name`会报错，但是可以修改它的值。

对象的方法使用函数类型描述。

```
const obj:{
  x: number;
  y: number;
  add(x:number, y:number): number;
  // 或者写成
  // add: (x:number, y:number) => number;
} = {
  x: 1,
  y: 1,
  add(x, y) {
    return x + y;
  }
};
```

上面示例中，对象`obj`有一个方法`add()`，需要定义它的参数类型和返回值类型。

对象类型可以使用方括号读取属性的类型。

```
type User = {
  name: string,
  age: number
};
type Name = User['name']; // string
```

上面示例中，对象类型`User`使用方括号，读取了属性`name`的类型（`string`）。

除了`type`命令可以为对象类型声明一个别名，TypeScript 还提供了`interface`命令，可以把对象类型提炼为一个接口。

```
// 写法一
type MyObj = {
```

```
  x:number;
  y:number;
};

const obj:MyObj = { x: 1, y: 1 };

// 写法二
interface MyObj {
  x: number;
  y: number;
}

const obj:MyObj = { x: 1, y: 1 };
```

上面示例中，写法一是`type`命令的用法，写法二是`interface`命令的用法。`interface`命令的详细解释，以及与`type`命令的区别，详见《Interface》一章。

注意，TypeScript 不区分对象自身的属性和继承的属性，一律视为对象的属性。

```
interface MyInterface {
  toString(): string; // 继承的属性
  prop: number; // 自身的属性
}

const obj:MyInterface = { // 正确
  prop: 123,
};
```

上面示例中，`obj`只写了`prop`属性，但是不报错。因为它可以继承原型上面的`toString()`方法。

可选属性

如果某个属性是可选的（即可以忽略），需要在属性名后面加一个问号。

```
const obj: {
  x: number;
  y?: number;
} = { x: 1 };
```

上面示例中，属性`y`是可选的。

可选属性等同于允许赋值为`undefined`，下面两种写法是等效的。

```
type User = {
  firstName: string;
  lastName?: string;
};

// 等同于
type User = {
  firstName: string;
```

```
  lastName?: string|undefined;
};
```

上面示例中，类型`User`的可选属性`lastName`可以是字符串，也可以是`undefined`，即可选属性可以赋值为`undefined`。

```
const obj: {
  x: number;
  y?: number;
} = { x: 1, y: undefined };
```

上面示例中，可选属性`y`赋值为`undefined`，不会报错。

同样地，读取一个没有赋值的可选属性时，返回`undefined`。

```
type MyObj = {
  x: string,
  y?: string
};

const obj:MyObj = { x: 'hello' };
obj.y.toLowerCase() // 报错
```

上面示例中，最后一行会报错，因为`obj.y`返回`undefined`，无法对其调用`toLowerCase()`。

所以，读取可选属性之前，必须检查一下是否为`undefined`。

```
const user:{
  firstName: string;
  lastName?: string;
} = { firstName: 'Foo'};

if (user.lastName !== undefined) {
  console.log(`hello ${user.firstName} ${user.lastName}`)
}
```

上面示例中，`lastName`是可选属性，需要判断是否为`undefined`以后，才能使用。建议使用下面的写法。

```
// 写法一
let firstName = (user.firstName === undefined)
  ? 'Foo' : user.firstName;
let lastName = (user.lastName === undefined)
  ? 'Bar' : user.lastName;

// 写法二
let firstName = user.firstName ?? 'Foo';
let lastName = user.lastName ?? 'Bar';
```

上面示例中，写法一使用三元运算符`?:`，判断是否为`undefined`，并设置默认值。写法二使用 Null 判断运算符`??`，与写法一的作用完全相同。

TypeScript 提供编译设置`exactOptionalPropertyTypes`，只要同时打开这个设置和`strictNullChecks`，可选属性就不能设为`undefined`。

```
// 打开 exactOptionalPropertyTypes 和 strictNullChecks
const obj: {
  x: number;
  y?: number;
} = { x: 1, y: undefined }; // 报错
```

上面示例中，打开了这两个设置以后，可选属性就不能设为`undefined`了。

注意，可选属性与允许设为`undefined`的必选属性是不等价的。

```
type A = { x:number, y?:number };
type B = { x:number, y:number|undefined };

const ObjA:A = { x: 1 }; // 正确
const ObjB:B = { x: 1 }; // 报错
```

上面示例中，属性`y`如果是一个可选属性，那就可以省略不写；如果是允许设为`undefined`的必选属性，一旦省略就会报错，必须显式写成`{ x: 1, y: undefined }`。

只读属性

属性名前面加上`readonly`关键字，表示这个属性是只读属性，不能修改。

```
interface MyInterface {
  readonly prop: number;
}
```

上面示例中，`prop`属性是只读属性，不能修改它的值。

```
const person:{
  readonly age: number
} = { age: 20 };

person.age = 21; // 报错
```

上面示例中，最后一行修改了只读属性`age`，就报错了。

只读属性只能在对象初始化期间赋值，此后就不能修改该属性。

```
type Point = {
  readonly x: number;
  readonly y: number;
};

const p:Point = { x: 0, y: 0 };
```

```
p.x = 100; // 报错
```

上面示例中，类型`Point`的属性`x`和`y`都带有修饰符`readonly`，表示这两个属性只能在初始化期间赋值，后面再修改就会报错。

注意，如果属性值是一个对象，`readonly`修饰符并不禁止修改该对象的属性，只是禁止完全替换掉该对象。

```
interface Home {
  readonly resident: {
    name: string;
    age: number
  };
}

const h:Home = {
  resident: {
    name: 'Vicky',
    age: 42
  }
};

h.resident.age = 32; // 正确
h.resident = {
  name: 'Kate',
  age: 23
} // 报错
```

上面示例中，`h.resident`是只读属性，它的值是一个对象。修改这个对象的`age`属性是可以的，但是整个替换掉`h.resident`属性会报错。

另一个需要注意的地方是，如果一个对象有两个引用，即两个变量对应同一个对象，其中一个变量是可写的，另一个变量是只读的，那么从可写变量修改属性，会影响到只读变量。

```
interface Person {
  name: string;
  age: number;
}

interface ReadonlyPerson {
  readonly name: string;
  readonly age: number;
}

let w:Person = {
  name: 'Vicky',
  age: 42,
};

let r:ReadonlyPerson = w;
```

```
w.age += 1;
r.age // 43
```

上面示例中，变量`w`和`r`指向同一个对象，其中`w`是可写的，`r`是只读的。那么，对`w`的属性修改，会影响到`r`。

如果希望属性值是只读的，除了声明时加上`readonly`关键字，还有一种方法，就是在赋值时，在对象后面加上只读断言`as const`。

```
const myUser = {
  name: "Sabrina",
} as const;

myUser.name = "Cynthia"; // 报错
```

上面示例中，对象后面加了只读断言`as const`，就变成只读对象了，不能修改属性了。

注意，上面的`as const`属于 TypeScript 的类型推断，如果变量明确地声明了类型，那么 TypeScript 会以声明的类型为准。

```
const myUser:{ name: string } = {
  name: "Sabrina",
} as const;

myUser.name = "Cynthia"; // 正确
```

上面示例中，根据变量`myUser`的类型声明，`name`不是只读属性，但是赋值时又使用只读断言`as const`。这时会以声明的类型为准，因为`name`属性可以修改。

属性名的索引类型

如果对象的属性非常多，一个个声明类型就很麻烦，而且有些时候，无法事前知道对象会有多少属性，比如外部 API 返回的对象。这时 TypeScript 允许采用属性名表达式的写法来描述类型，称为“属性名的索引类型”。

索引类型里面，最常见的就是属性名的字符串索引。

```
type MyObj = {
  [property: string]: string
};

const obj:MyObj = {
  foo: 'a',
  bar: 'b',
  baz: 'c',
};
```

上面示例中，类型`MyObj`的属性名类型就采用了表达式形式，写在方括号里面。`[property: string]`的`property`表示属性名，这个是可以随便起的，它的类型是`string`，即属性名类型为`string`。也就是说，不管这个对象有多少属性，只要属性名为字符串，且属性值也是字符串，就符合这个类型声明。

JavaScript 对象的属性名（即上例的`property`）的类型有三种可能，除了上例的`string`，还有`number`和`symbol`。

```
type T1 = {
  [property: number]: string
};

type T2 = {
  [property: symbol]: string
};
```

上面示例中，对象属性名的类型分别为`number`和`symbol`。

```
type MyArr = {
  [n:number]: number;
};

const arr:MyArr = [1, 2, 3];
// 或者
const arr:MyArr = {
  0: 1,
  1: 2,
  2: 3,
};
```

上面示例中，对象类型`MyArr`的属性名是`[n:number]`，就表示它的属性名都是数值，比如`0`、`1`、`2`。

对象可以同时有多种类型的属性名索引，比如同时有数值索引和字符串索引。但是，数值索引不能与字符串索引发生冲突，必须服从后者，这是因为在 JavaScript 语言内部，所有的数值属性名都会自动转为字符串属性名。

```
type MyType = {
  [x: number]: boolean; // 报错
  [x: string]: string;
}
```

上面示例中，类型`MyType`同时有两种属性名索引，但是数值索引与字符串索引冲突了，所以报错了。由于字符属性名的值类型是`string`，数值属性名的值类型只有同样为`string`，才不会报错。

同样地，可以既声明属性名索引，也声明具体的单个属性名。如果单个属性名不符合属性名索引的范围，两者发生冲突，就会报错。

```
type MyType = {
  foo: boolean; // 报错
```

```
  [x: string]: string;
}
```

上面示例中，属性名`foo`符合属性名的字符串索引，但是两者的属性值类型不一样，所以报错了。

属性的索引类型写法，建议谨慎使用，因为属性名的声明太宽泛，约束太少。另外，属性名的数值索引不宜用来声明数组，因为采用这种方式声明数组，就不能使用各种数组方法以及`length`属性，因为类型里面没有定义这些东西。

```
type MyArr = {
  [n:number]: number;
};

const arr:MyArr = [1, 2, 3];
arr.length // 报错
```

上面示例中，读取`arr.length`属性会报错，因为类型`MyArr`没有这个属性。

解构赋值

解构赋值用于直接从对象中提取属性。

```
const {id, name, price} = product;
```

上面语句从对象`product`提取了三个属性，并声明属性名的同名变量。

解构赋值的类型写法，跟为对象声明类型是一样的。

```
const {id, name, price}:{
  id: string;
  name: string;
  price: number
} = product;
```

注意，目前没法为解构变量指定类型，因为对象解构里面的冒号，JavaScript 指定了其他用途。

```
let { x: foo, y: bar } = obj;

// 等同于
let foo = obj.x;
let bar = obj.y;
```

上面示例中，冒号不是表示属性`x`和`y`的类型，而是为这两个属性指定新的变量名。如果要为`x`和`y`指定类型，不得不写成下面这样。

```
let { x: foo, y: bar }
  : { x: string; y: number } = obj;
```

这一点要特别小心，TypeScript 里面很容易搞糊涂。

```
function draw({
  shape: Shape,
  xPos: number = 100
}) {
  let myShape = shape; // 报错
  let x = xPos; // 报错
}
```

上面示例中，函数`draw()`的参数是一个对象解构，里面的冒号很像是为变量指定类型，其实是为对应的属性指定新的变量名。所以，TypeScript 就会解读成，函数体内不存在变量`shape`，而是属性`shape`的值被赋值给了变量`Shape`。

结构类型原则

只要对象 B 满足 对象 A 的结构特征，TypeScript 就认为对象 B 兼容对象 A 的类型，这称为“结构类型”原则（structural typing）。

```
type A = {
  x: number;
};

type B = {
  x: number;
  y: number;
};
```

上面示例中，对象`A`只有一个属性`x`，类型为`number`。对象`B`满足这个特征，因此兼容对象`A`，只要可以使用`A`的地方，就可以使用`B`。

```
const B = {
  x: 1,
  y: 1
};

const A:{ x: number } = B; // 正确
```

上面示例中，`A`和`B`并不是同一个类型，但是`B`可以赋值给`A`，因为`B`满足`A`的结构特征。

根据“结构类型”原则，TypeScript 检查某个值是否符合指定类型时，并不是检查这个值的类型名（即“名义类型”），而是检查这个值的结构是否符合要求（即“结构类型”）。

TypeScript 之所以这样设计，是为了符合 JavaScript 的行为。JavaScript 并不关心对象是否严格相似，只要某个对象具有所要求的属性，就可以正确运行。

如果类型 B 可以赋值给类型 A，TypeScript 就认为 B 是 A 的子类型（subtyping），A 是 B 的父类型。子类型满足父类型的所有结构特征，同时还具有自己的特征。凡是可以使用父类型的地方，都可以使用子类型，即

子类型兼容父类型。

这种设计有时会导致令人惊讶的结果。

```
type myObj = {
  x: number,
  y: number,
};

function getSum(obj:myObj) {
  let sum = 0;

  for (const n of Object.keys(obj)) {
    const v = obj[n]; // 报错
    sum += Math.abs(v);
  }

  return sum;
}
```

上面示例中，函数`getSum()`要求传入参数的类型是`myObj`，但是实际上所有与`myObj`兼容的对象都可以传入。这会导致`const v = obj[n]`这一行报错，原因是`obj[n]`取出的属性值不一定是数值（`number`），使得变量`v`的类型被推断为`any`。如果项目设置为不允许变量类型推断为`any`，代码就会报错。写成下面这样，就不会报错。

```
type MyObj = {
  x: number,
  y: number,
};

function getSum(obj:MyObj) {
  return Math.abs(obj.x) + Math.abs(obj.y);
}
```

上面示例就不会报错，因为函数体内部只使用了属性`x`和`y`，这两个属性有明确的类型声明，保证`obj.x`和`obj.y`肯定是数值。虽然与`MyObj`兼容的任何对象都可以传入函数`getSum()`，但是只要不使用其他属性，就不会有类型报错。

严格字面量检查

如果对象使用字面量表示，会触发 TypeScript 的严格字面量检查（strict object literal checking）。如果字面量的结构跟类型定义的不一样（比如多出了未定义的属性），就会报错。

```
const point:{
  x:number;
  y:number;
} = {
  x: 1,
  y: 1,
```

```
  z: 1 // 报错
};
```

上面示例中，等号右边是一个对象的字面量，这时会触发严格字面量检查。只要有类型声明中不存在的属性（本例是`z`），就会导致报错。

如果等号右边不是字面量，而是一个变量，根据结构类型原则，是不会报错的。

```
const myPoint = {
  x: 1,
  y: 1,
  z: 1
};

const point:{
  x:number;
  y:number;
} = myPoint; // 正确
```

上面示例中，等号右边是一个变量，就不会触发严格字面量检查，从而不报错。

TypeScript 对字面量进行严格检查的目的，主要是防止拼写错误。一般来说，字面量大多数来自手写，容易出现拼写错误，或者误用 API。

```
type Options = {
  title:string;
  darkMode?:boolean;
};

const obj:Options = {
  title: '我的网页',
  darkmode: true, // 报错
};
```

上面示例中，属性`darkMode`拼写错了，成了`darkmode`。如果没有严格字面量规则，就不会报错，因为`darkMode`是可选属性，根据结构类型原则，任何对象只要有`title`属性，都认为符合`Options`类型。

规避严格字面量检查，可以使用中间变量。

```
let myOptions = {
  title: '我的网页',
  darkmode: true,
};

const obj:Options = myOptions;
```

上面示例中，创建了一个中间变量`myOptions`，就不会触发严格字面量规则，因为这时变量`obj`的赋值，不属于直接字面量赋值。

如果你确认字面量没有错误，也可以使用类型断言规避严格字面量检查。

```
const obj:Options = {
  title: '我的网页',
  darkmode: true,
} as Options;
```

上面示例使用类型断言`as Options`，告诉编译器，字面量符合 Options 类型，就能规避这条规则。

如果允许字面量有多余属性，可以像下面这样在类型里面定义一个通用属性。

```
let x: {
  foo: number,
  [x: string]: any
};

x = { foo: 1, baz: 2 };  // Ok
```

上面示例中，变量`x`的类型声明里面，有一个属性的字符串索引（`[x: string]`），导致任何字符串属性名都是合法的。

由于严格字面量检查，字面量对象传入函数必须很小心，不能有多余的属性。

```
interface Point {
  x: number;
  y: number;
}

function computeDistance(point: Point) { /*...*/ }

computeDistance({ x: 1, y: 2, z: 3 }); // 报错
computeDistance({x: 1, y: 2}); // 正确
```

上面示例中，对象字面量传入函数`computeDistance()`时，不能有多余的属性，否则就通不过严格字面量检查。

编译器选项`suppressExcessPropertyErrors`，可以关闭多余属性检查。下面是它在 tsconfig.json 文件里面的写法。

```
{
  "compilerOptions": {
    "suppressExcessPropertyErrors": true
  }
}
```

最小可选属性规则

根据“结构类型”原则，如果一个对象的所有属性都是可选的，那么其他对象跟它都是结构类似的。

```
type Options = {
  a?:number;
```

```
  b?:number;
  c?:number;
};
```

上面示例中，类型`Options`的所有属性都是可选的，所以它可以是一个空对象，也就意味着任意对象都满足`Options`的结构。

为了避免这种情况，TypeScript 2.4 引入了一个“最小可选属性规则”，也称为“弱类型检测”（weak type detection）。

```
type Options = {
  a?:number;
  b?:number;
  c?:number;
};

const opts = { d: 123 };

const obj:Options = opts; // 报错
```

上面示例中，对象`opts`与类型`Options`没有共同属性，赋值给该类型的变量就会报错。

报错原因是，如果某个类型的所有属性都是可选的，那么该类型的对象必须至少存在一个可选属性，不能所有可选属性都不存在。这就叫做“最小可选属性规则”。

如果想规避这条规则，要么在类型里面增加一条索引属性（`[propName: string]: someType`），要么使用类型断言（`opts as Options`）。

空对象

空对象是 TypeScript 的一种特殊值，也是一种特殊类型。

```
const obj = {};
obj.prop = 123; // 报错
```

上面示例中，变量`obj`的值是一个空对象，然后对`obj.prop`赋值就会报错。

原因是这时 TypeScript 会推断变量`obj`的类型为空对象，实际执行的是下面的代码。

```
const obj:{} = {};
```

空对象没有自定义属性，所以对自定义属性赋值就会报错。空对象只能使用继承的属性，即继承自原型对象`Object.prototype`的属性。

```
obj.toString() // 正确
```

上面示例中，`toString()`方法是一个继承自原型对象的方法，TypeScript 允许在空对象上使用。

回到本节开始的例子，这种写法其实在 JavaScript 很常见：先声明一个空对象，然后向空对象添加属性。但是，TypeScript 不允许动态添加属性，所以对象不能分步生成，必须生成时一次性声明所有属性。

```
// 错误
const pt = {};
pt.x = 3;
pt.y = 4;

// 正确
const pt = {
  x: 3,
  y: 4
};
```

如果确实需要分步声明，一个比较好的方法是，使用扩展运算符（`...`）合成一个新对象。

```
const pt0 = {};
const pt1 = { x: 3 };
const pt2 = { y: 4 };

const pt = {
  ...pt0, ...pt1, ...pt2
};
```

上面示例中，对象`pt`是三个部分合成的，这样既可以分步声明，也符合 TypeScript 静态声明的要求。

空对象作为类型，其实是`Object`类型的简写形式。

```
let d:{};
// 等同于
// let d:Object;

d = {};
d = { x: 1 };
d = 'hello';
d = 2;
```

上面示例中，各种类型的值（除了`null`和`undefined`）都可以赋值给空对象类型，跟`Object`类型的行为是一样的。

因为`Object`可以接受各种类型的值，而空对象是`Object`类型的简写，所以它不会有严格字面量检查，赋值时总是允许多余的属性，只是不能读取这些属性。

```
interface Empty { }
const b:Empty = {myProp: 1, anotherProp: 2}; // 正确
b.myProp // 报错
```

上面示例中，变量`b`的类型是空对象，视同`Object`类型，不会有严格字面量检查，但是读取多余的属性会报错。

如果想强制使用没有任何属性的对象，可以采用下面的写法。

```
interface WithoutProperties {
  [key: string]: never;
}

// 报错
const a:WithoutProperties = { prop: 1 };
```

上面的示例中，`[key: string]: never`表示字符串属性名是不存在的，因此其他对象进行赋值时就会报错。

TypeScript 的 interface 接口

简介

interface 是对象的模板，可以看作是一种类型约定，中文译为“接口”。使用了某个模板的对象，就拥有了指定的类型结构。

```
interface Person {
  firstName: string;
  lastName: string;
  age: number;
}
```

上面示例中，定义了一个接口`Person`，它指定一个对象模板，拥有三个属性`firstName`、`lastName`和`age`。任何实现这个接口的对象，都必须部署这三个属性，并且必须符合规定的类型。

实现该接口很简单，只要指定它作为对象的类型即可。

```
const p:Person = {
  firstName: 'John',
  lastName: 'Smith',
  age: 25
};
```

上面示例中，变量`p`的类型就是接口`Person`，所以必须符合`Person`指定的结构。

方括号运算符可以取出 interface 某个属性的类型。

```
interface Foo {
  a: string;
}

type A = Foo['a']; // string
```

上面示例中，`Foo['a']`返回属性`a`的类型，所以类型`A`就是`string`。

interface 可以表示对象的各种语法，它的成员有5种形式。

- 对象属性
- 对象的属性索引
- 对象方法
- 函数
- 构造函数

（1）对象属性

```
interface Point {
  x: number;
  y: number;
}
```

上面示例中，`x`和`y`都是对象的属性，分别使用冒号指定每个属性的类型。

属性之间使用分号或逗号分隔，最后一个属性结尾的分号或逗号可以省略。

如果属性是可选的，就在属性名后面加一个问号。

```
interface Foo {
  x?: string;
}
```

如果属性是只读的，需要加上`readonly`修饰符。

```
interface A {
  readonly a: string;
}
```

（2）对象的属性索引

```
interface A {
  [prop: string]: number;
}
```

上面示例中，`[prop: string]`就是属性的字符串索引，表示属性名只要是字符串，都符合类型要求。

属性索引共有`string`、`number`和`symbol`三种类型。

一个接口中，最多只能定义一个字符串索引。字符串索引会约束该类型中所有名字为字符串的属性。

```
interface MyObj {
  [prop: string]: number;

  a: boolean;      // 编译错误
}
```

上面示例中，属性索引指定所有名称为字符串的属性，它们的属性值必须是数值（`number`）。属性`a`的值为布尔值就报错了。

属性的数值索引，其实是指定数组的类型。

```
interface A {
  [prop: number]: string;
}

const obj:A = ['a', 'b', 'c'];
```

上面示例中，`[prop: number]`表示属性名的类型是数值，所以可以用数组对变量`obj`赋值。

同样的，一个接口中最多只能定义一个数值索引。数值索引会约束所有名称为数值的属性。

如果一个 interface 同时定义了字符串索引和数值索引，那么数值索引必须服从于字符串索引。因为在 JavaScript 中，数值属性名最终是自动转换成字符串属性名。

```
interface A {
  [prop: string]: number;
  [prop: number]: string; // 报错
}

interface B {
  [prop: string]: number;
  [prop: number]: number; // 正确
}
```

上面示例中，数值索引的属性值类型与字符串索引不一致，就会报错。数值索引必须兼容字符串索引的类型声明。

（3）对象的方法

对象的方法共有三种写法。

```
// 写法一
interface A {
  f(x: boolean): string;
}

// 写法二
interface B {
  f: (x: boolean) => string;
}

// 写法三
interface C {
  f: { (x: boolean): string };
}
```

属性名可以采用表达式，所以下面的写法也是可以的。

```
const f = 'f';

interface A {
  [f](x: boolean): string;
}
```

类型方法可以重载。

```
interface A {
  f(): number;
```

```
  f(x: boolean): boolean;
  f(x: string, y: string): string;
}
```

interface 里面的函数重载，不需要给出实现。但是，由于对象内部定义方法时，无法使用函数重载的语法，所以需要额外在对象外部给出函数方法的实现。

```
interface A {
  f(): number;
  f(x: boolean): boolean;
  f(x: string, y: string): string;
}

function MyFunc(): number;
function MyFunc(x: boolean): boolean;
function MyFunc(x: string, y: string): string;
function MyFunc(
  x?:boolean|string, y?:string
):number|boolean|string {
  if (x === undefined && y === undefined) return 1;
  if (typeof x === 'boolean' && y === undefined) return true;
  if (typeof x === 'string' && typeof y === 'string') return 'hello';
  throw new Error('wrong parameters');
}

const a:A = {
  f: MyFunc
}
```

上面示例中，接口`A`的方法`f()`有函数重载，需要额外定义一个函数`MyFunc()`实现这个重载，然后部署接口`A`的对象`a`的属性`f`等于函数`MyFunc()`就可以了。

（4）函数

interface 也可以用来声明独立的函数。

```
interface Add {
  (x:number, y:number): number;
}

const myAdd:Add = (x,y) => x + y;
```

上面示例中，接口`Add`声明了一个函数类型。

（5）构造函数

interface 内部可以使用`new`关键字，表示构造函数。

```
interface ErrorConstructor {
  new (message?: string): Error;
}
```

上面示例中，接口`ErrorConstructor`内部有`new`命令，表示它是一个构造函数。

TypeScript 里面，构造函数特指具有`constructor`属性的类，详见《Class》一章。

interface 的继承

interface 可以继承其他类型，主要有下面几种情况。

interface 继承 interface

interface 可以使用`extends`关键字，继承其他 interface。

```
interface Shape {
  name: string;
}

interface Circle extends Shape {
  radius: number;
}
```

上面示例中，`Circle`继承了`Shape`，所以`Circle`其实有两个属性`name`和`radius`。这时，`Circle`是子接口，`Shape`是父接口。

`extends`关键字会从继承的接口里面拷贝属性类型，这样就不必书写重复的属性。

interface 允许多重继承。

```
interface Style {
  color: string;
}

interface Shape {
  name: string;
}

interface Circle extends Style, Shape {
  radius: number;
}
```

上面示例中，`Circle`同时继承了`Style`和`Shape`，所以拥有三个属性`color`、`name`和`radius`。

多重接口继承，实际上相当于多个父接口的合并。

如果子接口与父接口存在同名属性，那么子接口的属性会覆盖父接口的属性。注意，子接口与父接口的同名属性必须是类型兼容的，不能有冲突，否则会报错。

```
interface Foo {
  id: string;
}
```

```
interface Bar extends Foo {
  id: number; // 报错
}
```

上面示例中，`Bar`继承了`Foo`，但是两者的同名属性`id`的类型不兼容，导致报错。

多重继承时，如果多个父接口存在同名属性，那么这些同名属性不能有类型冲突，否则会报错。

```
interface Foo {
  id: string;
}

interface Bar {
  id: number;
}

// 报错
interface Baz extends Foo, Bar {
  type: string;
}
```

上面示例中，`Baz`同时继承了`Foo`和`Bar`，但是后两者的同名属性`id`有类型冲突，导致报错。

interface 继承 type

interface 可以继承`type`命令定义的对象类型。

```
type Country = {
  name: string;
  capital: string;
}

interface CountryWithPop extends Country {
  population: number;
}
```

上面示例中，`CountryWithPop`继承了`type`命令定义的`Country`对象，并且新增了一个`population`属性。

注意，如果`type`命令定义的类型不是对象，interface 就无法继承。

interface 继承 class

interface 还可以继承 class，即继承该类的所有成员。关于 class 的详细解释，参见下一章。

```
class A {
  x:string = '';

  y():boolean {
    return true;
  }
}
```

```
interface B extends A {
  z: number
}
```

上面示例中，`B`继承了`A`，因此`B`就具有属性`x`、`y()`和`z`。

实现`B`接口的对象就需要实现这些属性。

```
const b:B = {
  x: '',
  y: function(){ return true },
  z: 123
}
```

上面示例中，对象`b`就实现了接口`B`，而接口`B`又继承了类`A`。

某些类拥有私有成员和保护成员，interface 可以继承这样的类，但是意义不大。

```
class A {
  private x: string = '';
  protected y: string = '';
}

interface B extends A {
  z: number
}

// 报错
const b:B = { /* ... */ }

// 报错
class C implements B {
  // ...
}
```

上面示例中，`A`有私有成员和保护成员，`B`继承了`A`，但无法用于对象，因为对象不能实现这些成员。这导致`B`只能用于其他 class，而这时其他 class 与`A`之间不构成父类和子类的关系，使得`x`与`y`无法部署。

接口合并

多个同名接口会合并成一个接口。

```
interface Box {
  height: number;
  width: number;
}

interface Box {
```

```
  length: number;
}
```

上面示例中，两个`Box`接口会合并成一个接口，同时有`height`、`width`和`length`三个属性。

这样的设计主要是为了兼容 JavaScript 的行为。JavaScript 开发者常常对全局对象或者外部库，添加自己的属性和方法。那么，只要使用 interface 给出这些自定义属性和方法的类型，就能自动跟原始的 interface 合并，使得扩展外部类型非常方便。

举例来说，Web 网页开发经常会对`window`对象和`document`对象添加自定义属性，但是 TypeScript 会报错，因为原始定义没有这些属性。解决方法就是把自定义属性写成 interface，合并进原始定义。

```
interface Document {
  foo: string;
}

document.foo = 'hello';
```

上面示例中，接口`Document`增加了一个自定义属性`foo`，从而就可以在`document`对象上使用自定义属性。

同名接口合并时，同一个属性如果有多个类型声明，彼此不能有类型冲突。

```
interface A {
  a: number;
}

interface A {
  a: string; // 报错
}
```

上面示例中，接口`A`的属性`a`有两个类型声明，彼此是冲突的，导致报错。

同名接口合并时，如果同名方法有不同的类型声明，那么会发生函数重载。而且，后面的定义比前面的定义具有更高的优先级。

```
interface Cloner {
  clone(animal: Animal): Animal;
}

interface Cloner {
  clone(animal: Sheep): Sheep;
}

interface Cloner {
  clone(animal: Dog): Dog;
  clone(animal: Cat): Cat;
}

// 等同于
interface Cloner {
  clone(animal: Dog): Dog;
```

```
  clone(animal: Cat): Cat;
  clone(animal: Sheep): Sheep;
  clone(animal: Animal): Animal;
}
```

上面示例中，`clone()`方法有不同的类型声明，会发生函数重载。这时，越靠后的定义，优先级越高，排在函数重载的越前面。比如，`clone(animal: Animal)`是最先出现的类型声明，就排在函数重载的最后，属于`clone()`函数最后匹配的类型。

这个规则有一个例外。同名方法之中，如果有一个参数是字面量类型，字面量类型有更高的优先级。

```
interface A {
  f(x:'foo'): boolean;
}

interface A {
  f(x:any): void;
}

// 等同于
interface A {
  f(x:'foo'): boolean;
  f(x:any): void;
}
```

上面示例中，`f()`方法有一个类型声明的参数`x`是字面量类型，这个类型声明的优先级最高，会排在函数重载的最前面。

一个实际的例子是 Document 对象的`createElement()`方法，它会根据参数的不同，而生成不同的 HTML 节点对象。

```
interface Document {
  createElement(tagName: any): Element;
}
interface Document {
  createElement(tagName: "div"): HTMLDivElement;
  createElement(tagName: "span"): HTMLSpanElement;
}
interface Document {
  createElement(tagName: string): HTMLElement;
  createElement(tagName: "canvas"): HTMLCanvasElement;
}

// 等同于
interface Document {
  createElement(tagName: "canvas"): HTMLCanvasElement;
  createElement(tagName: "div"): HTMLDivElement;
  createElement(tagName: "span"): HTMLSpanElement;
  createElement(tagName: string): HTMLElement;
  createElement(tagName: any): Element;
}
```

上面示例中，`createElement()`方法的函数重载，参数为字面量的类型声明会排到最前面，返回具体的 HTML 节点对象。类型越不具体的参数，排在越后面，返回通用的 HTML 节点对象。

如果两个 interface 组成的联合类型存在同名属性，那么该属性的类型也是联合类型。

```
interface Circle {
  area: bigint;
}

interface Rectangle {
  area: number;
}

declare const s: Circle | Rectangle;

s.area;  // bigint | number
```

上面示例中，接口`Circle`和`Rectangle`组成一个联合类型`Circle | Rectangle`。因此，这个联合类型的同名属性`area`，也是一个联合类型。本例中的`declare`命令表示变量`s`的具体定义，由其他脚本文件给出，详见《declare 命令》一章。

interface 与 type 的异同

`interface`命令与`type`命令作用类似，都可以表示对象类型。

很多对象类型既可以用 interface 表示，也可以用 type 表示。而且，两者往往可以换用，几乎所有的 interface 命令都可以改写为 type 命令。

它们的相似之处，首先表现在都能为对象类型起名。

```
type Country = {
  name: string;
  capital: string;
}

interface Country {
  name: string;
  capital: string;
}
```

上面示例是`type`命令和`interface`命令，分别定义同一个类型。

`class`命令也有类似作用，通过定义一个类，同时定义一个对象类型。但是，它会创造一个值，编译后依然存在。如果只是单纯想要一个类型，应该使用`type`或`interface`。

interface 与 type 的区别有下面几点。

（1）`type`能够表示非对象类型，而`interface`只能表示对象类型（包括数组、函数等）。

（2）`interface`可以继承其他类型，`type`不支持继承。

继承的主要作用是添加属性，`type`定义的对象类型如果想要添加属性，只能使用`&`运算符，重新定义一个类型。

```
type Animal = {
  name: string
}

type Bear = Animal & {
  honey: boolean
}
```

上面示例中，类型`Bear`在`Animal`的基础上添加了一个属性`honey`。

上例的`&`运算符，表示同时具备两个类型的特征，因此可以起到两个对象类型合并的作用。

作为比较，`interface`添加属性，采用的是继承的写法。

```
interface Animal {
  name: string
}

interface Bear extends Animal {
  honey: boolean
}
```

继承时，type 和 interface 是可以换用的。interface 可以继承 type。

```
type Foo = { x: number; };

interface Bar extends Foo {
  y: number;
}
```

type 也可以继承 interface。

```
interface Foo {
  x: number;
}

type Bar = Foo & { y: number; };
```

（3）同名`interface`会自动合并，同名`type`则会报错。也就是说，TypeScript 不允许使用`type`多次定义同一个类型。

```
type A = { foo:number }; // 报错
type A = { bar:number }; // 报错
```

上面示例中，`type`两次定义了类型`A`，导致两行都会报错。

作为比较，`interface`则会自动合并。

```
interface A { foo:number };
interface A { bar:number };

const obj:A = {
  foo: 1,
  bar: 1
};
```

上面示例中，`interface`把类型`A`的两个定义合并在一起。

这表明，interface 是开放的，可以添加属性，type 是封闭的，不能添加属性，只能定义新的 type。

（4）`interface`不能包含属性映射（mapping），`type`可以，详见《映射》一章。

```
interface Point {
  x: number;
  y: number;
}

// 正确
type PointCopy1 = {
  [Key in keyof Point]: Point[Key];
};

// 报错
interface PointCopy2 {
  [Key in keyof Point]: Point[Key];
};
```

（5）`this`关键字只能用于`interface`。

```
// 正确
interface Foo {
  add(num:number): this;
};

// 报错
type Foo = {
  add(num:number): this;
};
```

上面示例中，type 命令声明的方法`add()`，返回`this`就报错了。interface 命令没有这个问题。

下面是返回`this`的实际对象的例子。

```
class Calculator implements Foo {
  result = 0;
  add(num:number) {
    this.result += num;
    return this;
```

```
  }
}
```

（6）type 可以扩展原始数据类型，interface 不行。

```
// 正确
type MyStr = string & {
  type: 'new'
};

// 报错
interface MyStr extends string {
  type: 'new'
}
```

上面示例中，type 可以扩展原始数据类型 string，interface 就不行。

（7）`interface`无法表达某些复杂类型（比如交叉类型和联合类型），但是`type`可以。

```
type A = { /* ... */ };
type B = { /* ... */ };

type AorB = A | B;
type AorBwithName = AorB & {
  name: string
};
```

上面示例中，类型`AorB`是一个联合类型，`AorBwithName`则是为`AorB`添加一个属性。这两种运算，`interface`都没法表达。

综上所述，如果有复杂的类型运算，那么没有其他选择只能使用`type`；一般情况下，`interface`灵活性比较高，便于扩充类型或自动合并，建议优先使用。

TypeScript 的 class 类型

简介

类（class）是面向对象编程的基本构件，封装了属性和方法，TypeScript 给予了全面支持。

属性的类型

类的属性可以在顶层声明，也可以在构造方法内部声明。

对于顶层声明的属性，可以在声明时同时给出类型。

```
class Point {
  x:number;
  y:number;
}
```

上面声明中，属性`x`和`y`的类型都是`number`。

如果不给出类型，TypeScript 会认为`x`和`y`的类型都是`any`。

```
class Point {
  x;
  y;
}
```

上面示例中，`x`和`y`的类型都是`any`。

如果声明时给出初值，可以不写类型，TypeScript 会自行推断属性的类型。

```
class Point {
  x = 0;
  y = 0;
}
```

上面示例中，属性`x`和`y`的类型都会被推断为 number。

TypeScript 有一个配置项`strictPropertyInitialization`，只要打开（默认是打开的），就会检查属性是否设置了初值，如果没有就报错。

```
// 打开 strictPropertyInitialization
class Point {
  x: number; // 报错
  y: number; // 报错
}
```

上面示例中，如果类的顶层属性不赋值，就会报错。如果不希望出现报错，可以使用非空断言。

```
class Point {
  x!: number;
  y!: number;
}
```

上面示例中，属性`x`和`y`没有初值，但是属性名后面添加了感叹号，表示这两个属性肯定不会为空，所以 TypeScript 就不报错了，详见《类型断言》一章。

readonly 修饰符

属性名前面加上 readonly 修饰符，就表示该属性是只读的。实例对象不能修改这个属性。

```
class A {
  readonly id = 'foo';
}

const a = new A();
a.id = 'bar'; // 报错
```

上面示例中，`id`属性前面有 readonly 修饰符，实例对象修改这个属性就会报错。

readonly 属性的初始值，可以写在顶层属性，也可以写在构造方法里面。

```
class A {
  readonly id:string;

  constructor() {
    this.id = 'bar'; // 正确
  }
}
```

上面示例中，构造方法内部设置只读属性的初值，这是可以的。

```
class A {
  readonly id:string = 'foo';

  constructor() {
    this.id = 'bar'; // 正确
  }
}
```

上面示例中，构造方法修改只读属性的值也是可以的。或者说，如果两个地方都设置了只读属性的值，以构造方法为准。在其他方法修改只读属性都会报错。

方法的类型

类的方法就是普通函数，类型声明方式与函数一致。

```
class Point {
  x:number;
  y:number;

  constructor(x:number, y:number) {
    this.x = x;
    this.y = y;
  }

  add(point:Point) {
    return new Point(
      this.x + point.x,
      this.y + point.y
    );
  }
}
```

上面示例中，构造方法`constructor()`和普通方法`add()`都注明了参数类型，但是省略了返回值类型，因为 TypeScript 可以自己推断出来。

类的方法跟普通函数一样，可以使用参数默认值，以及函数重载。

下面是参数默认值的例子。

```
class Point {
  x: number;
  y: number;

  constructor(x = 0, y = 0) {
    this.x = x;
    this.y = y;
  }
}
```

上面示例中，如果新建实例时，不提供属性`x`和`y`的值，它们都等于默认值`0`。

下面是函数重载的例子。

```
class Point {
  constructor(x:number, y:string);
  constructor(s:string);
  constructor(xs:number|string, y?:string) {
    // ...
  }
}
```

上面示例中，构造方法可以接受一个参数，也可以接受两个参数，采用函数重载进行类型声明。

另外，构造方法不能声明返回值类型，否则报错，因为它总是返回实例对象。

```
class B {
  constructor():object { // 报错
```

```
    // ...
  }
}
```

上面示例中，构造方法声明了返回值类型`object`，导致报错。

存取器方法

存取器（accessor）是特殊的类方法，包括取值器（getter）和存值器（setter）两种方法。

它们用于读写某个属性，取值器用来读取属性，存值器用来写入属性。

```
class C {
  _name = '';
  get name() {
    return this._name;
  }
  set name(value) {
    this._name = value;
  }
}
```

上面示例中，`get name()`是取值器，其中`get`是关键词，`name`是属性名。外部读取`name`属性时，实例对象会自动调用这个方法，该方法的返回值就是`name`属性的值。

`set name()`是存值器，其中`set`是关键词，`name`是属性名。外部写入`name`属性时，实例对象会自动调用这个方法，并将所赋的值作为函数参数传入。

TypeScript 对存取器有以下规则。

（1）如果某个属性只有`get`方法，没有`set`方法，那么该属性自动成为只读属性。

```
class C {
  _name = 'foo';

  get name() {
    return this._name;
  }
}

const c = new C();
c.name = 'bar'; // 报错
```

上面示例中，`name`属性没有`set`方法，对该属性赋值就会报错。

（2）TypeScript 5.1 版之前，`set`方法的参数类型，必须兼容`get`方法的返回值类型，否则报错。

```
// TypeScript 5.1 版之前
class C {
  _name = '';
  get name():string {  // 报错
```

```
    return this._name;
  }
  set name(value:number) {
    this._name = String(value);
  }
}
```

上面示例中，`get`方法的返回值类型是字符串，与`set`方法的参数类型`number`不兼容，导致报错。改成下面这样，就不会报错。

```
class C {
  _name = '';
  get name():string {
    return this._name;
  }
  set name(value:number|string) {
    this._name = String(value);
  }
}
```

上面示例中，`set`方法的参数类型（`number|string`）兼容`get`方法的返回值类型（`string`），这是允许的。

TypeScript 5.1 版做出了改变，现在两者可以不兼容。

（3）`get`方法与`set`方法的可访问性必须一致，要么都为公开方法，要么都为私有方法。

属性索引

类允许定义属性索引。

```
class MyClass {
  [s:string]: boolean |
    ((s:string) => boolean);

  get(s:string) {
    return this[s] as boolean;
  }
}
```

上面示例中，`[s:string]`表示所有属性名类型为字符串的属性，它们的属性值要么是布尔值，要么是返回布尔值的函数。

注意，由于类的方法是一种特殊属性（属性值为函数的属性），所以属性索引的类型定义也涵盖了方法。如果一个对象同时定义了属性索引和方法，那么前者必须包含后者的类型。

```
class MyClass {
  [s:string]: boolean;
  f() { // 报错
    return true;
```

```
  }
}
```

上面示例中，属性索引的类型里面不包括方法，导致后面的方法`f()`定义直接报错。正确的写法是下面这样。

```
class MyClass {
  [s:string]: boolean | (() => boolean);
  f() {
    return true;
  }
}
```

属性存取器视同属性。

```
class MyClass {
  [s:string]: boolean;

  get isInstance() {
    return true;
  }
}
```

上面示例中，属性`isInstance`的读取器虽然是一个函数方法，但是视同属性，所以属性索引虽然没有涉及方法类型，但是不会报错。

类的 interface 接口

implements 关键字

interface 接口或 type 别名，可以用对象的形式，为 class 指定一组检查条件。然后，类使用 implements 关键字，表示当前类满足这些外部类型条件的限制。

```
interface Country {
  name:string;
  capital:string;
}
// 或者
type Country = {
  name:string;
  capital:string;
}

class MyCountry implements Country {
  name = '';
  capital = '';
}
```

上面示例中，`interface`或`type`都可以定义一个对象类型。类`MyCountry`使用`implements`关键字，表示该类的实例对象满足这个外部类型。

interface 只是指定检查条件，如果不满足这些条件就会报错。它并不能代替 class 自身的类型声明。

```
interface A {
  get(name:string): boolean;
}

class B implements A {
  get(s) { // s 的类型是 any
    return true;
  }
}
```

上面示例中，类`B`实现了接口`A`，但是后者并不能代替`B`的类型声明。因此，`B`的`get()`方法的参数`s`的类型是`any`，而不是`string`。`B`类依然需要声明参数`s`的类型。

```
class B implements A {
  get(s:string) {
    return true;
  }
}
```

下面是另一个例子。

```
interface A {
  x: number;
  y?: number;
}

class B implements A {
  x = 0;
}

const b = new B();
b.y = 10; // 报错
```

上面示例中，接口`A`有一个可选属性`y`，类`B`没有声明这个属性，所以可以通过类型检查。但是，如果给`B`的实例对象的属性`y`赋值，就会报错。所以，`B`类还是需要声明可选属性`y`。

```
class B implements A {
  x = 0;
  y?: number;
}
```

同理，类可以定义接口没有声明的方法和属性。

```
interface Point {
  x: number;
  y: number;
}

class MyPoint implements Point {
```

```
  x = 1;
  y = 1;
  z:number = 1;
}
```

上面示例中，`MyPoint`类实现了`Point`接口，但是内部还定义了一个额外的属性`z`，这是允许的，表示除了满足接口给出的条件，类还有额外的条件。

`implements`关键字后面，不仅可以是接口，也可以是另一个类。这时，后面的类将被当作接口。

```
class Car {
  id:number = 1;
  move():void {};
}

class MyCar implements Car {
  id = 2; // 不可省略
  move():void {};   // 不可省略
}
```

上面示例中，`implements`后面是类`Car`，这时 TypeScript 就把`Car`视为一个接口，要求`MyCar`实现`Car`里面的每一个属性和方法，否则就会报错。所以，这时不能因为`Car`类已经实现过一次，而在`MyCar`类省略属性或方法。

注意，interface 描述的是类的对外接口，也就是实例的公开属性和公开方法，不能定义私有的属性和方法。这是因为 TypeScript 设计者认为，私有属性是类的内部实现，接口作为模板，不应该涉及类的内部代码写法。

```
interface Foo {
  private member:{}; // 报错
}
```

上面示例中，接口`Foo`有一个私有属性，结果就报错了。

实现多个接口

类可以实现多个接口（其实是接受多重限制），每个接口之间使用逗号分隔。

```
class Car implements MotorVehicle, Flyable, Swimmable {
  // ...
}
```

上面示例中，`Car`类同时实现了`MotorVehicle`、`Flyable`、`Swimmable`三个接口。这意味着，它必须部署这三个接口声明的所有属性和方法，满足它们的所有条件。

但是，同时实现多个接口并不是一个好的写法，容易使得代码难以管理，可以使用两种方法替代。

第一种方法是类的继承。

```
class Car implements MotorVehicle {
}

class SecretCar extends Car implements Flyable, Swimmable {
}
```

上面示例中，`Car`类实现了`MotorVehicle`，而`SecretCar`类继承了`Car`类，然后再实现`Flyable`和`Swimmable`两个接口，相当于`SecretCar`类同时实现了三个接口。

第二种方法是接口的继承。

```
interface A {
  a:number;
}

interface B extends A {
  b:number;
}
```

上面示例中，接口`B`继承了接口`A`，类只要实现接口`B`，就相当于实现`A`和`B`两个接口。

前一个例子可以用接口继承改写。

```
interface MotorVehicle {
  // ...
}
interface Flyable {
  // ...
}
interface Swimmable {
  // ...
}

interface SuperCar extends MotorVehicle,Flyable, Swimmable {
  // ...
}

class SecretCar implements SuperCar {
  // ...
}
```

上面示例中，类`SecretCar`通过`SuperCar`接口，就间接实现了多个接口。

注意，发生多重实现时（即一个接口同时实现多个接口），不同接口不能有互相冲突的属性。

```
interface Flyable {
  foo:number;
}

interface Swimmable {
```

```
  foo:string;
}
```

上面示例中，属性`foo`在两个接口里面的类型不同，如果同时实现这两个接口，就会报错。

类与接口的合并

TypeScript 不允许两个同名的类，但是如果一个类和一个接口同名，那么接口会被合并进类。

```
class A {
  x:number = 1;
}

interface A {
  y:number;
}

let a = new A();
a.y = 10;

a.x // 1
a.y // 10
```

上面示例中，类`A`与接口`A`同名，后者会被合并进前者的类型定义。

注意，合并进类的非空属性（上例的`y`），如果在赋值之前读取，会返回`undefined`。

```
class A {
  x:number = 1;
}

interface A {
  y:number;
}

let a = new A();
a.y // undefined
```

上面示例中，根据类型定义，`y`应该是一个非空属性。但是合并后，`y`有可能是`undefined`。

Class 类型

实例类型

TypeScript 的类本身就是一种类型，但是它代表该类的实例类型，而不是 class 的自身类型。

```
class Color {
  name:string;

  constructor(name:string) {
```

```
    this.name = name;
  }
}

const green:Color = new Color('green');
```

上面示例中，定义了一个类`Color`。它的类名就代表一种类型，实例对象`green`就属于该类型。

对于引用实例对象的变量来说，既可以声明类型为 Class，也可以声明类型为 Interface，因为两者都代表实例对象的类型。

```
interface MotorVehicle {
}

class Car implements MotorVehicle {
}

// 写法一
const c1:Car = new Car();
// 写法二
const c2:MotorVehicle = new Car();
```

上面示例中，变量的类型可以写成类`Car`，也可以写成接口`MotorVehicle`。它们的区别是，如果类`Car`有接口`MotorVehicle`没有的属性和方法，那么只有变量`c1`可以调用这些属性和方法。

作为类型使用时，类名只能表示实例的类型，不能表示类的自身类型。

```
class Point {
  x:number;
  y:number;

  constructor(x:number, y:number) {
    this.x = x;
    this.y = y;
  }
}

// 错误
function createPoint(
  PointClass:Point,
  x: number,
  y: number
) {
  return new PointClass(x, y);
}
```

上面示例中，函数`createPoint()`的第一个参数`PointClass`，需要传入 Point 这个类，但是如果把参数的类型写成`Point`就会报错，因为`Point`描述的是实例类型，而不是 Class 的自身类型。

由于类名作为类型使用，实际上代表一个对象，因此可以把类看作为对象类型起名。事实上，TypeScript 有三种方法可以为对象类型起名：type、interface 和 class。

类的自身类型

要获得一个类的自身类型，一个简便的方法就是使用 typeof 运算符。

```
function createPoint(
  PointClass:typeof Point,
  x:number,
  y:number
):Point {
  return new PointClass(x, y);
}
```

上面示例中，`createPoint()`的第一个参数`PointClass`是`Point`类自身，要声明这个参数的类型，简便的方法就是使用`typeof Point`。因为`Point`类是一个值，`typeof Point`返回这个值的类型。注意，`createPoint()`的返回值类型是`Point`，代表实例类型。

JavaScript 语言中，类只是构造函数的一种语法糖，本质上是构造函数的另一种写法。所以，类的自身类型可以写成构造函数的形式。

```
function createPoint(
  PointClass: new (x:number, y:number) => Point,
  x: number,
  y: number
):Point {
  return new PointClass(x, y);
}
```

上面示例中，参数`PointClass`的类型写成了一个构造函数，这时就可以把`Point`类传入。

构造函数也可以写成对象形式，所以参数`PointClass`的类型还有另一种写法。

```
function createPoint(
  PointClass: {
    new (x:number, y:number): Point
  },
  x: number,
  y: number
):Point {
  return new PointClass(x, y);
}
```

根据上面的写法，可以把构造函数提取出来，单独定义一个接口（interface），这样可以大大提高代码的通用性。

```
interface PointConstructor {
  new(x:number, y:number):Point;
}

function createPoint(
  PointClass: PointConstructor,
  x: number,
```

```
  y: number
):Point {
  return new PointClass(x, y);
}
```

总结一下，类的自身类型就是一个构造函数，可以单独定义一个接口来表示。

结构类型原则

Class 也遵循“结构类型原则”。一个对象只要满足 Class 的实例结构，就跟该 Class 属于同一个类型。

```
class Foo {
  id!:number;
}

function fn(arg:Foo) {
  // ...
}

const bar = {
  id: 10,
  amount: 100,
};

fn(bar); // 正确
```

上面示例中，对象`bar`满足类`Foo`的实例结构，只是多了一个属性`amount`。所以，它可以当作参数，传入函数`fn()`。

如果两个类的实例结构相同，那么这两个类就是兼容的，可以用在对方的使用场合。

```
class Person {
  name: string;
}

class Customer {
  name: string;
}

// 正确
const cust:Customer = new Person();
```

上面示例中，`Person`和`Customer`是两个结构相同的类，TypeScript 将它们视为相同类型，因此`Person`可以用在类型为`Customer`的场合。

现在修改一下代码，`Person`类添加一个属性。

```
class Person {
  name: string;
  age: number;
}
```

```
class Customer {
  name: string;
}

// 正确
const cust:Customer = new Person();
```

上面示例中，`Person`类添加了一个属性`age`，跟`Customer`类的结构不再相同。但是这种情况下，TypeScript 依然认为，`Person`属于`Customer`类型。

这是因为根据“结构类型原则”，只要`Person`类具有`name`属性，就满足`Customer`类型的实例结构，所以可以代替它。反过来就不行，如果`Customer`类多出一个属性，就会报错。

```
class Person {
  name: string;
}

class Customer {
  name: string;
  age: number;
}

// 报错
const cust:Customer = new Person();
```

上面示例中，`Person`类比`Customer`类少一个属性`age`，它就不满足`Customer`类型的实例结构，就报错了。因为在使用`Customer`类型的情况下，可能会用到它的`age`属性，而`Person`类就没有这个属性。

总之，只要 A 类具有 B 类的结构，哪怕还有额外的属性和方法，TypeScript 也认为 A 兼容 B 的类型。

不仅是类，如果某个对象跟某个 class 的实例结构相同，TypeScript 也认为两者的类型相同。

```
class Person {
  name: string;
}

const obj = { name: 'John' };
const p:Person = obj; // 正确
```

上面示例中，对象`obj`并不是`Person`的实例，但是赋值给变量`p`不会报错，TypeScript 认为`obj`也属于`Person`类型，因为它们的属性相同。

由于这种情况，运算符`instanceof`不适用于判断某个对象是否跟某个 class 属于同一类型。

```
obj instanceof Person // false
```

上面示例中，运算符`instanceof`确认变量`obj`不是 Person 的实例，但是两者的类型是相同的。

空类不包含任何成员，任何其他类都可以看作与空类结构相同。因此，凡是类型为空类的地方，所有类（包括对象）都可以使用。

```
class Empty {}

function fn(x:Empty) {
  // ...
}

fn({});
fn(window);
fn(fn);
```

上面示例中，函数`fn()`的参数是一个空类，这意味着任何对象都可以用作`fn()`的参数。

注意，确定两个类的兼容关系时，只检查实例成员，不考虑静态成员和构造方法。

```
class Point {
  x: number;
  y: number;
  static t: number;
  constructor(x:number) {}
}

class Position {
  x: number;
  y: number;
  z: number;
  constructor(x:string) {}
}

const point:Point = new Position('');
```

上面示例中，`Point`与`Position`的静态属性和构造方法都不一样，但因为`Point`的实例成员与`Position`相同，所以`Position`兼容`Point`。

如果类中存在私有成员（private）或保护成员（protected），那么确定兼容关系时，TypeScript 要求私有成员和保护成员来自同一个类，这意味着两个类需要存在继承关系。

```
// 情况一
class A {
  private name = 'a';
}

class B extends A {
}

const a:A = new B();

// 情况二
class A {
  protected name = 'a';
```

```
}

class B extends A {
  protected name = 'b';
}

const a:A = new B();
```

上面示例中，`A`和`B`都有私有成员（或保护成员）`name`，这时只有在`B`继承`A`的情况下（`class B extends A`），`B`才兼容`A`。

类的继承

类（这里又称“子类”）可以使用 extends 关键字继承另一个类（这里又称“基类”）的所有属性和方法。

```
class A {
  greet() {
    console.log('Hello, world!');
  }
}

class B extends A {
}

const b = new B();
b.greet() // "Hello, world!"
```

上面示例中，子类`B`继承了基类`A`，因此就拥有了`greet()`方法，不需要再次在类的内部定义这个方法了。

根据结构类型原则，子类也可以用于类型为基类的场合。

```
const a:A = b;
a.greet()
```

上面示例中，变量`a`的类型是基类，但是可以赋值为子类的实例。

子类可以覆盖基类的同名方法。

```
class B extends A {
  greet(name?: string) {
    if (name === undefined) {
      super.greet();
    } else {
      console.log(`Hello, ${name}`);
    }
  }
}
```

上面示例中，子类`B`定义了一个方法`greet()`，覆盖了基类`A`的同名方法。其中，参数`name`省略时，就调用基类`A`的`greet()`方法，这里可以写成`super.greet()`，使用`super`关键字指代基类是常见做法。

但是，子类的同名方法不能与基类的类型定义相冲突。

```
class A {
  greet() {
    console.log('Hello, world!');
  }
}

class B extends A {
  // 报错
  greet(name:string) {
    console.log(`Hello, ${name}`);
  }
}
```

上面示例中，子类`B`的`greet()`有一个`name`参数，跟基类`A`的`greet()`定义不兼容，因此就报错了。

如果基类包括保护成员（`protected`修饰符），子类可以将该成员的可访问性设置为公开（`public`修饰符），也可以保持保护成员不变，但是不能改用私有成员（`private`修饰符），详见后文。

```
class A {
  protected x: string = '';
  protected y: string = '';
  protected z: string = '';
}

class B extends A {
  // 正确
  public x:string = '';

  // 正确
  protected y:string = '';

  // 报错
  private z: string = '';
}
```

上面示例中，子类`B`将基类`A`的受保护成员改成私有成员，就会报错。

注意，`extends`关键字后面不一定是类名，可以是一个表达式，只要它的类型是构造函数就可以了。

```
// 例一
class MyArray extends Array<number> {}

// 例二
class MyError extends Error {}

// 例三
class A {
  greeting() {
    return 'Hello from A';
  }
}
```

```
class B {
  greeting() {
    return 'Hello from B';
  }
}

interface Greeter {
  greeting(): string;
}

interface GreeterConstructor {
  new (): Greeter;
}

function getGreeterBase():GreeterConstructor {
  return Math.random() >= 0.5 ? A : B;
}

class Test extends getGreeterBase() {
  sayHello() {
    console.log(this.greeting());
  }
}
```

上面示例中，例一和例二的`extends`关键字后面都是构造函数，例三的`extends`关键字后面是一个表达式，执行后得到的也是一个构造函数。

override 关键字

子类继承父类时，可以覆盖父类的同名方法。

```
class A {
  show() {
    // ...
  }
  hide() {
    // ...
  }
}
class B extends A {
  show() {
    // ...
  }
  hide() {
    // ...
  }
}
```

上面示例中，B 类定义了自己的`show()`方法和`hide()`方法，覆盖了 A 类的同名方法。

但是有些时候，我们继承他人的类，可能会在不知不觉中，就覆盖了他人的方法。为了防止这种情况，TypeScript 4.3 引入了 override 关键字。

```
class B extends A {
  override show() {
    // ...
  }
  override hide() {
    // ...
  }
}
```

上面示例中，B 类的`show()`方法和`hide()`方法前面加了 override 关键字，明确表明作者的意图，就是要覆盖 A 类里面的这两个同名方法。这时，如果 A 类没有定义自己的`show()`方法和`hide()`方法，就会报错。

但是，这依然没有解决，子类无意中覆盖父类同名方法的问题。因此，TypeScript 又提供了一个编译参数`noImplicitOverride`。一旦打开这个参数，子类覆盖父类的同名方法就会报错，除非使用了 override 关键字。

可访问性修饰符

类的内部成员的外部可访问性，由三个可访问性修饰符（access modifiers）控制：`public`、`private`和`protected`。

这三个修饰符的位置，都写在属性或方法的最前面。

public

`public`修饰符表示这是公开成员，外部可以自由访问。

```
class Greeter {
  public greet() {
    console.log("hi!");
  }
}

const g = new Greeter();
g.greet();
```

上面示例中，`greet()`方法前面的`public`修饰符，表示该方法可以在类的外部调用，即外部实例可以调用。

`public`修饰符是默认修饰符，如果省略不写，实际上就带有该修饰符。因此，类的属性和方法默认都是外部可访问的。

正常情况下，除非为了醒目和代码可读性，`public`都是省略不写的。

private

`private`修饰符表示私有成员，只能用在当前类的内部，类的实例和子类都不能使用该成员。

```
class A {
  private x:number = 0;
}

const a = new A();
a.x // 报错

class B extends A {
  showX() {
    console.log(this.x); // 报错
  }
}
```

上面示例中，属性`x`前面有`private`修饰符，表示这是私有成员。因此，实例对象和子类使用该成员，都会报错。

注意，子类不能定义父类私有成员的同名成员。

```
class A {
  private x = 0;
}

class B extends A {
  x = 1; // 报错
}
```

上面示例中，`A`类有一个私有属性`x`，子类`B`就不能定义自己的属性`x`了。

如果在类的内部，当前类的实例可以获取私有成员。

```
class A {
  private x = 10;

  f(obj:A) {
    console.log(obj.x);
  }
}

const a = new A();
a.f(a) // 10
```

上面示例中，在类`A`内部，`A`的实例对象可以获取私有成员`x`。

严格地说，`private`定义的私有成员，并不是真正意义的私有成员。一方面，编译成 JavaScript 后，`private`关键字就被剥离了，这时外部访问该成员就不会报错。另一方面，由于前一个原因，TypeScript 对于访问`private`成员没有严格禁止，使用方括号写法（`[]`）或者`in`运算符，实例对象就能访问该成员。

```
class A {
  private x = 1;
}
```

```
const a = new A();
a['x'] // 1

if ('x' in a) { // 正确
  // ...
}
```

上面示例中，`A`类的属性`x`是私有属性，但是实例使用方括号，就可以读取这个属性，或者使用`in`运算符检查这个属性是否存在，都可以正确执行。

由于`private`存在这些问题，加上它是 ES2022 标准发布前出台的，而 ES2022 引入了自己的私有成员写法`#propName`。因此建议不使用`private`，改用 ES2022 的写法，获得真正意义的私有成员。

```
class A {
  #x = 1;
}

const a = new A();
a['x'] // 报错
```

上面示例中，采用了 ES2022 的私有成员写法（属性名前加`#`），TypeScript 就正确识别了实例对象没有属性`x`，从而报错。

构造方法也可以是私有的，这就直接防止了使用`new`命令生成实例对象，只能在类的内部创建实例对象。

这时一般会有一个静态方法，充当工厂函数，强制所有实例都通过该方法生成。

```
class Singleton {
  private static instance?: Singleton;

  private constructor() {}

  static getInstance() {
    if (!Singleton.instance) {
      Singleton.instance = new Singleton();
    }
    return Singleton.instance;
  }
}

const s = Singleton.getInstance();
```

上面示例使用私有构造方法，实现了单例模式。想要获得 Singleton 的实例，不能使用`new`命令，只能使用`getInstance()`方法。

protected

`protected`修饰符表示该成员是保护成员，只能在类的内部使用该成员，实例无法使用该成员，但是子类内部可以使用。

```
class A {
  protected x = 1;
}

class B extends A {
  getX() {
    return this.x;
  }
}

const a = new A();
const b = new B();

a.x // 报错
b.getX() // 1
```

上面示例中，类`A`的属性`x`是保护成员，直接从实例读取该属性（`a.x`）会报错，但是子类`B`内部可以读取该属性。

子类不仅可以拿到父类的保护成员，还可以定义同名成员。

```
class A {
  protected x = 1;
}

class B extends A {
  x = 2;
}
```

上面示例中，子类`B`定义了父类`A`的同名成员`x`，并且父类的`x`是保护成员，子类将其改成了公开成员。`B`类的`x`属性前面没有修饰符，等同于修饰符是`public`，外界可以读取这个属性。

在类的外部，实例对象不能读取保护成员，但是在类的内部可以。

```
class A {
  protected x = 1;

  f(obj:A) {
    console.log(obj.x);
  }
}

const a = new A();

a.x // 报错
a.f(a) // 1
```

上面示例中，属性`x`是类`A`的保护成员，在类的外部，实例对象`a`拿不到这个属性。但是，实例对象`a`传入类`A`的内部，就可以从`a`拿到`x`。

实例属性的简写形式

实际开发中，很多实例属性的值，是通过构造方法传入的。

```
class Point {
  x:number;
  y:number;

  constructor(x:number, y:number) {
    this.x = x;
    this.y = y;
  }
}
```

上面实例中，属性`x`和`y`的值是通过构造方法的参数传入的。

这样的写法等于对同一个属性要声明两次类型，一次在类的头部，另一次在构造方法的参数里面。这有些累赘，TypeScript 就提供了一种简写形式。

```
class Point {
  constructor(
    public x:number,
    public y:number
  ) {}
}

const p = new Point(10, 10);
p.x // 10
p.y // 10
```

上面示例中，构造方法的参数`x`前面有`public`修饰符，这时 TypeScript 就会自动声明一个公开属性`x`，不必在构造方法里面写任何代码，同时还会设置`x`的值为构造方法的参数值。注意，这里的`public`不能省略。

除了`public`修饰符，构造方法的参数名只要有`private`、`protected`、`readonly`修饰符，都会自动声明对应修饰符的实例属性。

```
class A {
  constructor(
    public a: number,
    protected b: number,
    private c: number,
    readonly d: number
  ) {}
}

// 编译结果
class A {
    a;
    b;
    c;
    d;
    constructor(a, b, c, d) {
```

```
      this.a = a;
      this.b = b;
      this.c = c;
      this.d = d;
    }
}
```

上面示例中，从编译结果可以看到，构造方法的`a`、`b`、`c`、`d`会生成对应的实例属性。

`readonly`还可以与其他三个可访问性修饰符，一起使用。

```
class A {
  constructor(
    public readonly x:number,
    protected readonly y:number,
    private readonly z:number
  ) {}
}
```

顶层属性的处理方法

对于类的顶层属性，TypeScript 早期的处理方法，与后来的 ES2022 标准不一致。这会导致某些代码的运行结果不一样。

类的顶层属性在 TypeScript 里面，有两种写法。

```
class User {
  // 写法一
  age = 25;

  // 写法二
  constructor(private currentYear: number) {}
}
```

上面示例中，写法一是直接声明一个实例属性`age`，并初始化；写法二是顶层属性的简写形式，直接将构造方法的参数`currentYear`声明为实例属性。

TypeScript 早期的处理方法是，先在顶层声明属性，但不进行初始化，等到运行构造方法时，再完成所有初始化。

```
class User {
  age = 25;
}

// TypeScript 的早期处理方法
class User {
  age: number;

  constructor() {
    this.age = 25;
```

```
  }
}
```

上面示例中，TypeScript 早期会先声明顶层属性`age`，然后等到运行构造函数时，再将其初始化为`25`。

ES2022 标准里面的处理方法是，先进行顶层属性的初始化，再运行构造方法。这在某些情况下，会使得同一段代码在 TypeScript 和 JavaScript 下运行结果不一致。

这种不一致一般发生在两种情况。第一种情况是，顶层属性的初始化依赖于其他实例属性。

```
class User {
  age = this.currentYear - 1998;

  constructor(private currentYear: number) {
    // 输出结果将不一致
    console.log('Current age:', this.age);
  }
}

const user = new User(2023);
```

上面示例中，顶层属性`age`的初始化值依赖于实例属性`this.currentYear`。按照 TypeScript 的处理方法，初始化是在构造方法里面完成的，会输出结果为`25`。但是，按照 ES2022 标准的处理方法，初始化在声明顶层属性时就会完成，这时`this.currentYear`还等于`undefined`，所以`age`的初始化结果为`NaN`，因此最后输出的也是`NaN`。

第二种情况与类的继承有关，子类声明的顶层属性在父类完成初始化。

```
interface Animal {
  animalStuff: any;
}

interface Dog extends Animal {
  dogStuff: any;
}

class AnimalHouse {
  resident: Animal;

  constructor(animal:Animal) {
    this.resident = animal;
  }
}

class DogHouse extends AnimalHouse {
  resident: Dog;

  constructor(dog:Dog) {
    super(dog);
  }
}
```

上面示例中，类`DogHouse`继承自`AnimalHouse`。它声明了顶层属性`resident`，但是该属性的初始化是在父类`AnimalHouse`完成的。不同的设置运行下面的代码，结果将不一致。

```
const dog = {
  animalStuff: 'animal',
  dogStuff: 'dog'
};

const dogHouse = new DogHouse(dog);

console.log(dogHouse.resident) // 输出结果将不一致
```

上面示例中，TypeScript 的处理方法，会使得`resident`属性能够初始化，所以输出参数对象的值。但是，ES2022 标准的处理方法是，顶层属性的初始化先于构造方法的运行。这使得`resident`属性不会得到赋值，因此输出为`undefined`。

为了解决这个问题，同时保证以前代码的行为一致，TypeScript 从3.7版开始，引入了编译设置`useDefineForClassFields`。这个设置设为`true`，则采用 ES2022 标准的处理方法，否则采用 TypeScript 早期的处理方法。

它的默认值与`target`属性有关，如果输出目标设为`ES2022`或者更高，那么`useDefineForClassFields`的默认值为`true`，否则为`false`。关于这个设置的详细说明，参见官方 3.7 版本的发布说明。

如果希望避免这种不一致，让代码在不同设置下的行为都一样，那么可以将所有顶层属性的初始化，都放到构造方法里面。

```
class User  {
  age: number;

  constructor(private currentYear: number) {
    this.age = this.currentYear - 1998;
    console.log('Current age:', this.age);
  }
}

const user = new User(2023);
```

上面示例中，顶层属性`age`的初始化就放在构造方法里面，那么任何情况下，代码行为都是一致的。

对于类的继承，还有另一种解决方法，就是使用`declare`命令，去声明子类顶层属性的类型，告诉 TypeScript 这些属性的初始化由父类实现。

```
class DogHouse extends AnimalHouse {
  declare resident: Dog;

  constructor(dog:Dog) {
    super(dog);
  }
}
```

上面示例中，`resident`属性的类型声明前面用了`declare`命令。这种情况下，这一行代码在编译成 JavaScript 后就不存在，那么也就不会有行为不一致，无论是否设置`useDefineForClassFields`，输出结果都是一样的。

静态成员

类的内部可以使用`static`关键字，定义静态成员。

静态成员是只能通过类本身使用的成员，不能通过实例对象使用。

```
class MyClass {
  static x = 0;
  static printX() {
    console.log(MyClass.x);
  }
}

MyClass.x // 0
MyClass.printX() // 0
```

上面示例中，`x`是静态属性，`printX()`是静态方法。它们都必须通过`MyClass`获取，而不能通过实例对象调用。

`static`关键字前面可以使用 public、private、protected 修饰符。

```
class MyClass {
  private static x = 0;
}

MyClass.x // 报错
```

上面示例中，静态属性`x`前面有`private`修饰符，表示只能在`MyClass`内部使用，如果在外部调用这个属性就会报错。

静态私有属性也可以用 ES6 语法的`#`前缀表示，上面示例可以改写如下。

```
class MyClass {
  static #x = 0;
}
```

`public`和`protected`的静态成员可以被继承。

```
class A {
  public static x = 1;
  protected static y = 1;
}

class B extends A {
  static getY() {
```

```
    return B.y;
  }
}

B.x // 1
B.getY() // 1
```

上面示例中，类`A`的静态属性`x`和`y`都被`B`继承，公开成员`x`可以在`B`的外部获取，保护成员`y`只能在`B`的内部获取。

泛型类

类也可以写成泛型，使用类型参数。关于泛型的详细介绍，请看《泛型》一章。

```
class Box<Type> {
  contents: Type;

  constructor(value:Type) {
    this.contents = value;
  }
}

const b:Box<string> = new Box('hello!');
```

上面示例中，类`Box`有类型参数`Type`，因此属于泛型类。新建实例时，变量的类型声明需要带有类型参数的值，不过本例等号左边的`Box<string>`可以省略不写，因为可以从等号右边推断得到。

注意，静态成员不能使用泛型的类型参数。

```
class Box<Type> {
  static defaultContents: Type; // 报错
}
```

上面示例中，静态属性`defaultContents`的类型写成类型参数`Type`会报错。因为这意味着调用时必须给出类型参数（即写成`Box<string>.defaultContents`），并且类型参数发生变化，这个属性也会跟着变，这并不是好的做法。

抽象类，抽象成员

TypeScript 允许在类的定义前面，加上关键字`abstract`，表示该类不能被实例化，只能当作其他类的模板。这种类就叫做“抽象类”（abstract class）。

```
abstract class A {
  id = 1;
}

const a = new A(); // 报错
```

上面示例中，直接新建抽象类的实例，会报错。

抽象类只能当作基类使用，用来在它的基础上定义子类。

```
abstract class A {
  id = 1;
}

class B extends A {
  amount = 100;
}

const b = new B();

b.id // 1
b.amount // 100
```

上面示例中，`A`是一个抽象类，`B`是`A`的子类，继承了`A`的所有成员，并且可以定义自己的成员和实例化。

抽象类的子类也可以是抽象类，也就是说，抽象类可以继承其他抽象类。

```
abstract class A {
  foo:number;
}

abstract class B extends A {
  bar:string;
}
```

抽象类的内部可以有已经实现好的属性和方法，也可以有还未实现的属性和方法。后者就叫做“抽象成员”（abstract member），即属性名和方法名有`abstract`关键字，表示该方法需要子类实现。如果子类没有实现抽象成员，就会报错。

```
abstract class A {
  abstract foo:string;
  bar:string = '';
}

class B extends A {
  foo = 'b';
}
```

上面示例中，抽象类`A`定义了抽象属性`foo`，子类`B`必须实现这个属性，否则会报错。

下面是抽象方法的例子。如果抽象类的方法前面加上`abstract`，就表明子类必须给出该方法的实现。

```
abstract class A {
  abstract execute():string;
}

class B extends A {
```

```
  execute() {
    return `B executed`;
  }
}
```

这里有几个注意点。

（1）抽象成员只能存在于抽象类，不能存在于普通类。

（2）抽象成员不能有具体实现的代码。也就是说，已经实现好的成员前面不能加`abstract`关键字。

（3）抽象成员前也不能有`private`修饰符，否则无法在子类中实现该成员。

（4）一个子类最多只能继承一个抽象类。

总之，抽象类的作用是，确保各种相关的子类都拥有跟基类相同的接口，可以看作是模板。其中的抽象成员都是必须由子类实现的成员，非抽象成员则表示基类已经实现的、由所有子类共享的成员。

this 问题

类的方法经常用到`this`关键字，它表示该方法当前所在的对象。

```
class A {
  name = 'A';

  getName() {
    return this.name;
  }
}

const a = new A();
a.getName() // 'A'

const b = {
  name: 'b',
  getName: a.getName
};
b.getName() // 'b'
```

上面示例中，变量`a`和`b`的`getName()`是同一个方法，但是执行结果不一样，原因就是它们内部的`this`指向不一样的对象。如果`getName()`在变量`a`上运行，`this`指向`a`；如果在`b`上运行，`this`指向`b`。

有些场合需要给出`this`类型，但是 JavaScript 函数通常不带有`this`参数，这时 TypeScript 允许函数增加一个名为`this`的参数，放在参数列表的第一位，用来描述函数内部的`this`关键字的类型。

```
// 编译前
function fn(
  this: SomeType,
  x: number
) {
```

```
  /* ... */
}

// 编译后
function fn(x) {
  /* ... */
}
```

上面示例中，函数`fn()`的第一个参数是`this`，用来声明函数内部的`this`的类型。编译时，TypeScript 一旦发现函数的第一个参数名为`this`，则会去除这个参数，即编译结果不会带有该参数。

```
class A {
  name = 'A';

  getName(this: A) {
    return this.name;
  }
}

const a = new A();
const b = a.getName;

b() // 报错
```

上面示例中，类`A`的`getName()`添加了`this`参数，如果直接调用这个方法，`this`的类型就会跟声明的类型不一致，从而报错。

`this`参数的类型可以声明为各种对象。

```
function foo(
  this: { name: string }
) {
  this.name = 'Jack';
  this.name = 0; // 报错
}

foo.call({ name: 123 }); // 报错
```

上面示例中，参数`this`的类型是一个带有`name`属性的对象，不符合这个条件的`this`都会报错。

TypeScript 提供了一个`noImplicitThis`编译选项。如果打开了这个设置项，如果`this`的值推断为`any`类型，就会报错。

```
// noImplicitThis 打开

class Rectangle {
  constructor(
    public width:number,
    public height:number
  ) {}

  getAreaFunction() {
```

```
    return function () {
      return this.width * this.height; // 报错
    };
  }
}
```

上面示例中，`getAreaFunction()`方法返回一个函数，这个函数里面用到了`this`，但是这个`this`跟`Rectangle`这个类没关系，它的类型推断为`any`，所以就报错了。

在类的内部，`this`本身也可以当作类型使用，表示当前类的实例对象。

```
class Box {
  contents:string = '';

  set(value:string):this {
    this.contents = value;
    return this;
  }
}
```

上面示例中，`set()`方法的返回值类型就是`this`，表示当前的实例对象。

注意，`this`类型不允许应用于静态成员。

```
class A {
  static a:this; // 报错
}
```

上面示例中，静态属性`a`的返回值类型是`this`，就报错了。原因是`this`类型表示实例对象，但是静态成员拿不到实例对象。

有些方法返回一个布尔值，表示当前的`this`是否属于某种类型。这时，这些方法的返回值类型可以写成`this is Type`的形式，其中用到了`is`运算符。

```
class FileSystemObject {
  isFile(): this is FileRep {
    return this instanceof FileRep;
  }

  isDirectory(): this is Directory {
    return this instanceof Directory;
  }

  // ...
}
```

上面示例中，两个方法的返回值类型都是布尔值，写成`this is Type`的形式，可以精确表示返回值。`is`运算符的介绍详见《类型断言》一章。

参考链接

- TypeScript Constructor in Interface
- TypeScript: useDefineForClassFields – How to avoid future Breaking Changes

TypeScript 泛型

简介

有些时候，函数返回值的类型与参数类型是相关的。

```
function getFirst(arr) {
  return arr[0];
}
```

上面示例中，函数`getFirst()`总是返回参数数组的第一个成员。参数数组是什么类型，返回值就是什么类型。

这个函数的类型声明只能写成下面这样。

```
function f(arr:any[]):any {
  return arr[0];
}
```

上面的类型声明，就反映不出参数与返回值之间的类型关系。

为了解决这个问题，TypeScript 就引入了“泛型”（generics）。泛型的特点就是带有“类型参数”（type parameter）。

```
function getFirst<T>(arr:T[]):T {
  return arr[0];
}
```

上面示例中，函数`getFirst()`的函数名后面尖括号的部分`<T>`，就是类型参数，参数要放在一对尖括号（`<>`）里面。本例只有一个类型参数`T`，可以将其理解为类型声明需要的变量，需要在调用时传入具体的参数类型。

上例的函数`getFirst()`的参数类型是`T[]`，返回值类型是`T`，就清楚地表示了两者之间的关系。比如，输入的参数类型是`number[]`，那么 T 的值就是`number`，因此返回值类型也是`number`。

函数调用时，需要提供类型参数。

```
getFirst<number>([1, 2, 3])
```

上面示例中，调用函数`getFirst()`时，需要在函数名后面使用尖括号，给出类型参数`T`的值，本例是`<number>`。

不过为了方便，函数调用时，往往省略不写类型参数的值，让 TypeScript 自己推断。

```
getFirst([1, 2, 3])
```

上面示例中，TypeScript 会从实际参数`[1, 2, 3]`，推断出类型参数 T 的值为`number`。

有些复杂的使用场景，TypeScript 可能推断不出类型参数的值，这时就必须显式给出了。

```
function comb<T>(arr1:T[], arr2:T[]):T[] {
  return arr1.concat(arr2);
}
```

上面示例中，两个参数`arr1`、`arr2`和返回值都是同一个类型。如果不给出类型参数的值，下面的调用会报错。

```
comb([1, 2], ['a', 'b']) // 报错
```

上面示例会报错，TypeScript 认为两个参数不是同一个类型。但是，如果类型参数是一个联合类型，就不会报错。

```
comb<number|string>([1, 2], ['a', 'b']) // 正确
```

上面示例中，类型参数是一个联合类型，使得两个参数都符合类型参数，就不报错了。这种情况下，类型参数是不能省略不写的。

类型参数的名字，可以随便取，但是必须为合法的标识符。习惯上，类型参数的第一个字符往往采用大写字母。一般会使用`T`（type 的第一个字母）作为类型参数的名字。如果有多个类型参数，则使用 T 后面的 U、V 等字母命名，各个参数之间使用逗号（“,”）分隔。

下面是多个类型参数的例子。

```
function map<T, U>(
  arr:T[],
  f:(arg:T) => U
):U[] {
  return arr.map(f);
}

// 用法实例
map<string, number>(
  ['1', '2', '3'],
  (n) => parseInt(n)
); // 返回 [1, 2, 3]
```

上面示例将数组的实例方法`map()`改写成全局函数，它有两个类型参数`T`和`U`。含义是，原始数组的类型为`T[]`，对该数组的每个成员执行一个处理函数`f`，将类型`T`转成类型`U`，那么就会得到一个类型为`U[]`的数组。

总之，泛型可以理解成一段类型逻辑，需要类型参数来表达。有了类型参数以后，可以在输入类型与输出类型之间，建立一一对应关系。

泛型的写法

泛型主要用在四个场合：函数、接口、类和别名。

函数的泛型写法

上一节提到，`function`关键字定义的泛型函数，类型参数放在尖括号中，写在函数名后面。

```
function id<T>(arg:T):T {
  return arg;
}
```

那么对于变量形式定义的函数，泛型有下面两种写法。

```
// 写法一
let myId:<T>(arg:T) => T = id;

// 写法二
let myId:{ <T>(arg:T): T } = id;
```

接口的泛型写法

interface 也可以采用泛型的写法。

```
interface Box<Type> {
  contents: Type;
}

let box:Box<string>;
```

上面示例中，使用泛型接口时，需要给出类型参数的值（本例是`string`）。

下面是另一个例子。

```
interface Comparator<T> {
  compareTo(value:T): number;
}

class Rectangle implements Comparator<Rectangle> {

  compareTo(value:Rectangle): number {
    // ...
  }
}
```

上面示例中，先定义了一个泛型接口，然后将这个接口用于一个类。

泛型接口还有第二种写法。

```
interface Fn {
  <Type>(arg:Type): Type;
}

function id<Type>(arg:Type): Type {
  return arg;
}

let myId:Fn = id;
```

上面示例中，`Fn`的类型参数`Type`的具体类型，需要函数`id`在使用时提供。所以，最后一行的赋值语句不需要给出`Type`的具体类型。

此外，第二种写法还有一个差异之处。那就是它的类型参数定义在某个方法之中，其他属性和方法不能使用该类型参数。前面的第一种写法，类型参数定义在整个接口，接口内部的所有属性和方法都可以使用该类型参数。

类的泛型写法

泛型类的类型参数写在类名后面。

```
class Pair<K, V> {
  key: K;
  value: V;
}
```

下面是继承泛型类的例子。

```
class A<T> {
  value: T;
}

class B extends A<any> {
}
```

上面示例中，类`A`有一个类型参数`T`，使用时必须给出`T`的类型，所以类`B`继承时要写成`A<any>`。

泛型也可以用在类表达式。

```
const Container = class<T> {
  constructor(private readonly data:T) {}
};

const a = new Container<boolean>(true);
const b = new Container<number>(0);
```

上面示例中，新建实例时，需要同时给出类型参数`T`和类参数`data`的值。

下面是另一个例子。

```
class C<NumType> {
  value!: NumType;
  add!: (x: NumType, y: NumType) => NumType;
}

let foo = new C<number>();

foo.value = 0;
foo.add = function (x, y) {
  return x + y;
};
```

上面示例中，先新建类`C`的实例`foo`，然后再定义实例的`value`属性和`add()`方法。类的定义中，属性和方法后面的感叹号是非空断言，告诉 TypeScript 它们都是非空的，后面会赋值。

JavaScript 的类本质上是一个构造函数，因此也可以把泛型类写成构造函数。

```
type MyClass<T> = new (...args: any[]) => T;

// 或者
interface MyClass<T> {
  new(...args: any[]): T;
}

// 用法实例
function createInstance<T>(
  AnyClass: MyClass<T>,
  ...args: any[]
):T {
  return new AnyClass(...args);
}
```

上面示例中，函数`createInstance()`的第一个参数`AnyClass`是构造函数（也可以是一个类），它的类型是`MyClass<T>`，这里的`T`是`createInstance()`的类型参数，在该函数调用时再指定具体类型。

注意，泛型类描述的是类的实例，不包括静态属性和静态方法，因为这两者定义在类的本身。因此，它们不能引用类型参数。

```
class C<T> {
  static data: T;  // 报错
  constructor(public value:T) {}
}
```

上面示例中，静态属性`data`引用了类型参数`T`，这是不可以的，因为类型参数只能用于实例属性和实例方法，所以报错了。

类型别名的泛型写法

type 命令定义的类型别名，也可以使用泛型。

```
type Nullable<T> = T | undefined | null;
```

上面示例中，`Nullable<T>`是一个泛型，只要传入一个类型，就可以得到这个类型与`undefined`和`null`的一个联合类型。

下面是另一个例子。

```
type Container<T> = { value: T };

const a: Container<number> = { value: 0 };
const b: Container<string> = { value: 'b' };
```

下面是定义树形结构的例子。

```
type Tree<T> = {
  value: T;
  left: Tree<T> | null;
  right: Tree<T> | null;
};
```

上面示例中，类型别名`Tree`内部递归引用了`Tree`自身。

类型参数的默认值

类型参数可以设置默认值。使用时，如果没有给出类型参数的值，就会使用默认值。

```
function getFirst<T = string>(
  arr:T[]
):T {
  return arr[0];
}
```

上面示例中，`T = string`表示类型参数的默认值是`string`。调用`getFirst()`时，如果不给出`T`的值，TypeScript 就认为`T`等于`string`。

但是，因为 TypeScript 会从实际参数推断出`T`的值，从而覆盖掉默认值，所以下面的代码不会报错。

```
getFirst([1, 2, 3]) // 正确
```

上面示例中，实际参数是`[1, 2, 3]`，TypeScript 推断 T 等于`number`，从而覆盖掉默认值`string`。

类型参数的默认值，往往用在类中。

```
class Generic<T = string> {
  list:T[] = []

  add(t:T) {
    this.list.push(t)
```

```
  }
}
```

上面示例中，类`Generic`有一个类型参数`T`，默认值为`string`。这意味着，属性`list`默认是一个字符串数组，方法`add()`的默认参数是一个字符串。

```
const g = new Generic();

g.add(4) // 报错
g.add('hello') // 正确
```

上面示例中，新建`Generic`的实例`g`时，没有给出类型参数`T`的值，所以`T`就等于`string`。因此，向`add()`方法传入一个数值会报错，传入字符串就不会。

```
const g = new Generic<number>();

g.add(4) // 正确
g.add('hello') // 报错
```

上面示例中，新建实例`g`时，给出了类型参数`T`的值是`number`，因此`add()`方法传入数值不会报错，传入字符串会报错。

一旦类型参数有默认值，就表示它是可选参数。如果有多个类型参数，可选参数必须在必选参数之后。

```
<T = boolean, U> // 错误

<T, U = boolean> // 正确
```

上面示例中，依次有两个类型参数`T`和`U`。如果`T`是可选参数，`U`不是，就会报错。

数组的泛型表示

《数组》一章提到过，数组类型有一种表示方法是`Array<T>`。这就是泛型的写法，`Array`是 TypeScript 原生的一个类型接口，`T`是它的类型参数。声明数组时，需要提供`T`的值。

```
let arr:Array<number> = [1, 2, 3];
```

上面的示例中，`Array<number>`就是一个泛型，类型参数的值是`number`，表示该数组的全部成员都是数值。

同样的，如果数组成员都是字符串，那么类型就写成`Array<string>`。事实上，在 TypeScript 内部，数组类型的另一种写法`number[]`、`string[]`，只是`Array<number>`、`Array<string>`的简写形式。

在 TypeScript 内部，`Array`是一个泛型接口，类型定义基本是下面的样子。

```
interface Array<Type> {

  length: number;
```

```
  pop(): Type|undefined;

  push(...items:Type[]): number;

  // ...
}
```

上面代码中，`push()`方法的参数`item`的类型是`Type[]`，跟`Array()`的参数类型`Type`保持一致，表示只能添加同类型的成员。调用`push()`的时候，TypeScript 就会检查两者是否一致。

其他的 TypeScript 内部数据结构，比如`Map`、`Set`和`Promise`，其实也是泛型接口，完整的写法是`Map<K, V>`、`Set<T>`和`Promise<T>`。

TypeScript 默认还提供一个`ReadonlyArray<T>`接口，表示只读数组。

```
function doStuff(
  values:ReadonlyArray<string>
) {
  values.push('hello!');  // 报错
}
```

上面示例中，参数`values`的类型是`ReadonlyArray<string>`，表示不能修改这个数组，所以函数体内部新增数组成员就会报错。因此，如果不希望函数内部改动参数数组，就可以将该参数数组声明为`ReadonlyArray<T>`类型。

类型参数的约束条件

很多类型参数并不是无限制的，对于传入的类型存在约束条件。

```
function comp<Type>(a:Type, b:Type) {
  if (a.length >= b.length) {
    return a;
  }
  return b;
}
```

上面示例中，类型参数 Type 有一个隐藏的约束条件：它必须存在`length`属性。如果不满足这个条件，就会报错。

TypeScript 提供了一种语法，允许在类型参数上面写明约束条件，如果不满足条件，编译时就会报错。这样也可以有良好的语义，对类型参数进行说明。

```
function comp<T extends { length: number }>(
  a: T,
  b: T
) {
  if (a.length >= b.length) {
    return a;
```

```
  }
  return b;
}
```

上面示例中，`T extends { length: number }`就是约束条件，表示类型参数 T 必须满足`{ length: number }`，否则就会报错。

```
comp([1, 2], [1, 2, 3]) // 正确
comp('ab', 'abc') // 正确
comp(1, 2) // 报错
```

上面示例中，只要传入的参数类型不满足约束条件，就会报错。

类型参数的约束条件采用下面的形式。

```
<TypeParameter extends ConstraintType>
```

上面语法中，`TypeParameter`表示类型参数，`extends`是关键字，这是必须的，`ConstraintType`表示类型参数要满足的条件，即类型参数应该是`ConstraintType`的子类型。

类型参数可以同时设置约束条件和默认值，前提是默认值必须满足约束条件。

```
type Fn<A extends string, B extends string = 'world'>
  =  [A, B];

type Result = Fn<'hello'> // ["hello", "world"]
```

上面示例中，类型参数`A`和`B`都有约束条件，并且`B`还有默认值。所以，调用`Fn`的时候，可以只给出`A`的值，不给出`B`的值。

另外，上例也可以看出，泛型本质上是一个类型函数，通过输入参数，获得结果，两者是一一对应关系。

如果有多个类型参数，一个类型参数的约束条件，可以引用其他参数。

```
<T, U extends T>
// 或者
<T extends U, U>
```

上面示例中，`U`的约束条件引用`T`，或者`T`的约束条件引用`U`，都是正确的。

但是，约束条件不能引用类型参数自身。

```
<T extends T>               // 报错
<T extends U, U extends T>  // 报错
```

上面示例中，`T`的约束条件不能是`T`自身。同理，多个类型参数也不能互相约束（即`T`的约束条件是`U`、`U`的约束条件是`T`），因为互相约束就意味着约束条件就是类型参数自身。

使用注意点

泛型有一些使用注意点。

（1）尽量少用泛型。

泛型虽然灵活，但是会加大代码的复杂性，使其变得难读难写。一般来说，只要使用了泛型，类型声明通常都不太易读，容易写得很复杂。因此，可以不用泛型就不要用。

（2）类型参数越少越好。

多一个类型参数，多一道替换步骤，加大复杂性。因此，类型参数越少越好。

```
function filter<
  T,
  Fn extends (arg:T) => boolean
>(
  arr:T[],
  func:Fn
): T[] {
  return arr.filter(func);
}
```

上面示例有两个类型参数，但是第二个类型参数`Fn`是不必要的，完全可以直接写在函数参数的类型声明里面。

```
function filter<T>(
  arr:T[],
  func:(arg:T) => boolean
): T[] {
  return arr.filter(func);
}
```

上面示例中，类型参数简化成了一个，效果与前一个示例是一样的。

（3）类型参数需要出现两次。

如果类型参数在定义后只出现一次，那么很可能是不必要的。

```
function greet<Str extends string>(
  s:Str
) {
  console.log('Hello, ' + s);
}
```

上面示例中，类型参数`Str`只在函数声明中出现一次（除了它的定义部分），这往往表明这个类型参数是不必要。

```
function greet(s:string) {
  console.log('Hello, ' + s);
```

```
}
```

上面示例把前面的类型参数省略了，效果与前一个示例是一样的。

也就是说，只有当类型参数用到两次或两次以上，才是泛型的适用场合。

（4）泛型可以嵌套。

类型参数可以是另一个泛型。

```
type OrNull<Type> = Type|null;

type OneOrMany<Type> = Type|Type[];

type OneOrManyOrNull<Type> = OrNull<OneOrMany<Type>>;
```

上面示例中，最后一行的泛型`OrNull`的类型参数，就是另一个泛型`OneOrMany`。

TypeScript 的 Enum 类型

Enum 是 TypeScript 新增的一种数据结构和类型，中文译为“枚举”。

简介

实际开发中，经常需要定义一组相关的常量。

```
const RED = 1;
const GREEN = 2;
const BLUE = 3;

let color = userInput();

if (color === RED) {/* */}
if (color === GREEN) {/* */}
if (color === BLUE) {/* */}

throw new Error('wrong color');
```

上面示例中，常量`RED`、`GREEN`、`BLUE`是相关的，意为变量`color`的三个可能的取值。它们具体等于什么值其实并不重要，只要不相等就可以了。

TypeScript 就设计了 Enum 结构，用来将相关常量放在一个容器里面，方便使用。

```
enum Color {
  Red,     // 0
  Green,   // 1
  Blue     // 2
}
```

上面示例声明了一个 Enum 结构`Color`，里面包含三个成员`Red`、`Green`和`Blue`。第一个成员的值默认为整数`0`，第二个为`1`，第三个为`2`，以此类推。

使用时，调用 Enum 的某个成员，与调用对象属性的写法一样，可以使用点运算符，也可以使用方括号运算符。

```
let c = Color.Green; // 1
// 等同于
let c = Color['Green']; // 1
```

Enum 结构本身也是一种类型。比如，上例的变量`c`等于`1`，它的类型可以是 Color，也可以是`number`。

```
let c:Color = Color.Green; // 正确
let c:number = Color.Green; // 正确
```

上面示例中，变量`c`的类型写成`Color`或`number`都可以。但是，`Color`类型的语义更好。

Enum 结构的特别之处在于，它既是一种类型，也是一个值。绝大多数 TypeScript 语法都是类型语法，编译后会全部去除，但是 Enum 结构是一个值，编译后会变成 JavaScript 对象，留在代码中。

```
// 编译前
enum Color {
  Red,     // 0
  Green,   // 1
  Blue     // 2
}

// 编译后
let Color = {
  Red: 0,
  Green: 1,
  Blue: 2
};
```

上面示例是 Enum 结构编译前后的对比。

由于 TypeScript 的定位是 JavaScript 语言的类型增强，所以官方建议谨慎使用 Enum 结构，因为它不仅仅是类型，还会为编译后的代码加入一个对象。

Enum 结构比较适合的场景是，成员的值不重要，名字更重要，从而增加代码的可读性和可维护性。

```
enum Operator {
  ADD,
  DIV,
  MUL,
  SUB
}

function compute(
  op:Operator,
  a:number,
  b:number
) {
  switch (op) {
    case Operator.ADD:
      return a + b;
    case Operator.DIV:
      return a / b;
    case Operator.MUL:
      return a * b;
    case Operator.SUB:
      return a - b;
    default:
      throw new Error('wrong operator');
  }
}

compute(Operator.ADD, 1, 3) // 4
```

上面示例中，Enum 结构`Operator`的四个成员表示四则运算“加减乘除”。代码根本不需要用到这四个成员的值，只用成员名就够了。

TypeScript 5.0 之前，Enum 有一个 Bug，就是 Enum 类型的变量可以赋值为任何数值。

```
enum Bool {
  No,
  Yes
}

function foo(noYes:Bool) {
  // ...
}

foo(33);  // TypeScript 5.0 之前不报错
```

上面示例中，函数`foo`的参数`noYes`是 Enum 类型，只有两个可用的值。但是，TypeScript 5.0 之前，任何数值作为函数`foo`的参数，编译都不会报错，TypeScript 5.0 纠正了这个问题。

另外，由于 Enum 结构编译后是一个对象，所以不能有与它同名的变量（包括对象、函数、类等）。

```
enum Color {
  Red,
  Green,
  Blue
}

const Color = 'red'; // 报错
```

上面示例，Enum 结构与变量同名，导致报错。

很大程度上，Enum 结构可以被对象的`as const`断言替代。

```
enum Foo {
  A,
  B,
  C,
}

const Bar = {
  A: 0,
  B: 1,
  C: 2,
} as const;

if (x === Foo.A) {}
// 等同于
if (x === Bar.A) {}
```

上面示例中，对象`Bar`使用了`as const`断言，作用就是使得它的属性无法修改。这样的话，`Foo`和`Bar`的行为就很类似了，前者完全可以用后者替代，而且后者还是 JavaScript 的原生数据结构。

Enum 成员的值

Enum 成员默认不必赋值，系统会从零开始逐一递增，按照顺序为每个成员赋值，比如0、1、2……

但是，也可以为 Enum 成员显式赋值。

```
enum Color {
  Red,
  Green,
  Blue
}

// 等同于
enum Color {
  Red = 0,
  Green = 1,
  Blue = 2
}
```

上面示例中，Enum 每个成员的值都是显式赋值。

成员的值可以是任意数值，但不能是大整数（Bigint）。

```
enum Color {
  Red = 90,
  Green = 0.5,
  Blue = 7n // 报错
}
```

上面示例中，Enum 成员的值可以是小数，但不能是 Bigint。

成员的值甚至可以相同。

```
enum Color {
  Red = 0,
  Green = 0,
  Blue = 0
}
```

如果只设定第一个成员的值，后面成员的值就会从这个值开始递增。

```
enum Color {
  Red = 7,
  Green,  // 8
  Blue   // 9
}

// 或者
enum Color {
  Red, // 0
  Green = 7,
```

```
  Blue // 8
}
```

Enum 成员的值也可以使用计算式。

```
enum Permission {
  UserRead     = 1 << 8,
  UserWrite    = 1 << 7,
  UserExecute  = 1 << 6,
  GroupRead    = 1 << 5,
  GroupWrite   = 1 << 4,
  GroupExecute = 1 << 3,
  AllRead      = 1 << 2,
  AllWrite     = 1 << 1,
  AllExecute   = 1 << 0,
}

enum Bool {
  No = 123,
  Yes = Math.random(),
}
```

上面示例中，Enum 成员的值等于一个计算式，或者等于函数的返回值，都是正确的。

Enum 成员值都是只读的，不能重新赋值。

```
enum Color {
  Red,
  Green,
  Blue
}

Color.Red = 4; // 报错
```

上面示例中，重新为 Enum 成员赋值就会报错。

为了让这一点更醒目，通常会在 enum 关键字前面加上`const`修饰，表示这是常量，不能再次赋值。

```
const enum Color {
  Red,
  Green,
  Blue
}
```

加上`const`还有一个好处，就是编译为 JavaScript 代码后，代码中 Enum 成员会被替换成对应的值，这样能提高性能表现。

```
const enum Color {
  Red,
  Green,
  Blue
}
```

```
const x = Color.Red;
const y = Color.Green;
const z = Color.Blue;

// 编译后
const x = 0 /* Color.Red */;
const y = 1 /* Color.Green */;
const z = 2 /* Color.Blue */;
```

上面示例中，由于 Enum 结构前面加了`const`关键字，所以编译产物里面就没有生成对应的对象，而是把所有 Enum 成员出现的场合，都替换成对应的常量。

如果希望加上`const`关键词后，运行时还能访问 Enum 结构（即编译后依然将 Enum 转成对象），需要在编译时打开`preserveConstEnums`编译选项。

同名 Enum 的合并

多个同名的 Enum 结构会自动合并。

```
enum Foo {
  A,
}

enum Foo {
  B = 1,
}

enum Foo {
  C = 2,
}

// 等同于
enum Foo {
  A,
  B = 1,
  C = 2
}
```

上面示例中，`Foo`分成三段定义，系统会自动把它们合并。

Enum 结构合并时，只允许其中一个的首成员省略初始值，否则报错。

```
enum Foo {
  A,
}

enum Foo {
  B, // 报错
}
```

上面示例中，`Foo`的两段定义的第一个成员，都没有设置初始值，导致报错。

同名 Enum 合并时，不能有同名成员，否则报错。

```
enum Foo {
  A,
  B
}

enum Foo {
  B = 1, // 报错
  C
}
```

上面示例中，`Foo`的两段定义有一个同名成员`B`，导致报错。

同名 Enum 合并的另一个限制是，所有定义必须同为 const 枚举或者非 const 枚举，不允许混合使用。

```
// 正确
enum E {
  A,
}
enum E {
  B = 1,
}

// 正确
const enum E {
  A,
}
const enum E {
  B = 1,
}

// 报错
enum E {
  A,
}
const enum E {
  B = 1,
}
```

同名 Enum 的合并，最大用处就是补充外部定义的 Enum 结构。

字符串 Enum

Enum 成员的值除了设为数值，还可以设为字符串。也就是说，Enum 也可以用作一组相关字符串的集合。

```
enum Direction {
  Up = 'UP',
  Down = 'DOWN',
```

```
  Left = 'LEFT',
  Right = 'RIGHT',
}
```

上面示例中，`Direction`就是字符串枚举，每个成员的值都是字符串。

注意，字符串枚举的所有成员值，都必须显式设置。如果没有设置，成员值默认为数值，且位置必须在字符串成员之前。

```
enum Foo {
  A, // 0
  B = 'hello',
  C // 报错
}
```

上面示例中，`A`之前没有其他成员，所以可以不设置初始值，默认等于`0`；`C`之前有一个字符串成员，所以`C`必须有初始值，不赋值就报错了。

Enum 成员可以是字符串和数值混合赋值。

```
enum Enum {
  One = 'One',
  Two = 'Two',
  Three = 3,
  Four = 4,
}
```

除了数值和字符串，Enum 成员不允许使用其他值（比如 Symbol 值）。

变量类型如果是字符串 Enum，就不能再赋值为字符串，这跟数值 Enum 不一样。

```
enum MyEnum {
  One = 'One',
  Two = 'Two',
}

let s = MyEnum.One;
s = 'One'; // 报错
```

上面示例中，变量`s`的类型是`MyEnum`，再赋值为字符串就报错。

由于这个原因，如果函数的参数类型是字符串 Enum，传参时就不能直接传入字符串，而要传入 Enum 成员。

```
enum MyEnum {
  One = 'One',
  Two = 'Two',
}

function f(arg:MyEnum) {
  return 'arg is ' + arg;
```

```
}

f('One') // 报错
```

上面示例中，参数类型是`MyEnum`，直接传入字符串会报错。

所以，字符串 Enum 作为一种类型，有限定函数参数的作用。

前面说过，数值 Enum 的成员值往往不重要。但是有些场合，开发者可能希望 Enum 成员值可以保存一些有用的信息，所以 TypeScript 才设计了字符串 Enum。

```
const enum MediaTypes {
  JSON = 'application/json',
  XML = 'application/xml',
}

const url = 'localhost';

fetch(url, {
  headers: {
    Accept: MediaTypes.JSON,
  },
}).then(response => {
  // ...
});
```

上面示例中，函数`fetch()`的参数对象的属性`Accept`，只能接受一些指定的字符串。这时就很适合把字符串放进一个 Enum 结构，通过成员值来引用这些字符串。

字符串 Enum 可以使用联合类型（union）代替。

```
function move(
  where:'Up'|'Down'|'Left'|'Right'
) {
  // ...
 }
```

上面示例中，函数参数`where`属于联合类型，效果跟指定为字符串 Enum 是一样的。

注意，字符串 Enum 的成员值，不能使用表达式赋值。

```
enum MyEnum {
  A = 'one',
  B = ['T', 'w', 'o'].join('') // 报错
}
```

上面示例中，成员`B`的值是一个字符串表达式，导致报错。

keyof 运算符

keyof 运算符可以取出 Enum 结构的所有成员名，作为联合类型返回。

```
enum MyEnum {
  A = 'a',
  B = 'b'
}

// 'A'|'B'
type Foo = keyof typeof MyEnum;
```

上面示例中，`keyof typeof MyEnum`可以取出`MyEnum`的所有成员名，所以类型`Foo`等同于联合类型`'A'|'B'`。

注意，这里的`typeof`是必需的，否则`keyof MyEnum`相当于`keyof string`。

```
type Foo = keyof MyEnum;
// number | typeof Symbol.iterator | "toString" | "charAt" | "charCodeAt" | ...
```

上面示例中，类型`Foo`等于类型`string`的所有原生属性名组成的联合类型。这是`MyEnum`为字符串 Enum 的结果，如果`MyEnum`是数值 Enum，那么`keyof MyEnum`相当于`keyof number`。

这是因为 Enum 作为类型，本质上属于`number`或`string`的一种变体，而`typeof MyEnum`会将`MyEnum`当作一个值处理，从而先将其转为对象类型，就可以再用`keyof`运算符返回该对象的所有属性名。

如果要返回 Enum 所有的成员值，可以使用`in`运算符。

```
enum MyEnum {
  A = 'a',
  B = 'b'
}

// { a: any, b: any }
type Foo = { [key in MyEnum]: any };
```

上面示例中，采用属性索引可以取出`MyEnum`的所有成员值。

反向映射

数值 Enum 存在反向映射，即可以通过成员值获得成员名。

```
enum Weekdays {
  Monday = 1,
  Tuesday,
  Wednesday,
  Thursday,
  Friday,
  Saturday,
  Sunday
```

```
}

console.log(Weekdays[3]) // Wednesday
```

上面示例中，Enum 成员`Wednesday`的值等于3，从而可以从成员值`3`取到对应的成员名`Wednesday`，这就叫反向映射。

这是因为 TypeScript 会将上面的 Enum 结构，编译成下面的 JavaScript 代码。

```
var Weekdays;
(function (Weekdays) {
    Weekdays[Weekdays["Monday"] = 1] = "Monday";
    Weekdays[Weekdays["Tuesday"] = 2] = "Tuesday";
    Weekdays[Weekdays["Wednesday"] = 3] = "Wednesday";
    Weekdays[Weekdays["Thursday"] = 4] = "Thursday";
    Weekdays[Weekdays["Friday"] = 5] = "Friday";
    Weekdays[Weekdays["Saturday"] = 6] = "Saturday";
    Weekdays[Weekdays["Sunday"] = 7] = "Sunday";
})(Weekdays || (Weekdays = {}));
```

上面代码中，实际进行了两组赋值，以第一个成员为例。

```
Weekdays[
  Weekdays["Monday"] = 1
] = "Monday";
```

上面代码有两个赋值运算符（`=`），实际上等同于下面的代码。

```
Weekdays["Monday"] = 1;
Weekdays[1] = "Monday";
```

注意，这种情况只发生在数值 Enum，对于字符串 Enum，不存在反向映射。这是因为字符串 Enum 编译后只有一组赋值。

```
enum MyEnum {
  A = 'a',
  B = 'b'
}

// 编译后
var MyEnum;
(function (MyEnum) {
    MyEnum["A"] = "a";
    MyEnum["B"] = "b";
})(MyEnum || (MyEnum = {}));
```

TypeScript 的类型断言

简介

对于没有类型声明的值，TypeScript 会进行类型推断，很多时候得到的结果，未必是开发者想要的。

```
type T = 'a'|'b'|'c';
let foo = 'a';

let bar:T = foo; // 报错
```

上面示例中，最后一行报错，原因是 TypeScript 推断变量`foo`的类型是`string`，而变量`bar`的类型是`'a'|'b'|'c'`，前者是后者的父类型。父类型不能赋值给子类型，所以就报错了。

TypeScript 提供了“类型断言”这样一种手段，允许开发者在代码中“断言”某个值的类型，告诉编译器此处的值是什么类型。TypeScript 一旦发现存在类型断言，就不再对该值进行类型推断，而是直接采用断言给出的类型。

这种做法的实质是，允许开发者在某个位置“绕过”编译器的类型推断，让本来通不过类型检查的代码能够通过，避免编译器报错。这样虽然削弱了 TypeScript 类型系统的严格性，但是为开发者带来了方便，毕竟开发者比编译器更了解自己的代码。

回到上面的例子，解决方法就是进行类型断言，在赋值时断言变量`foo`的类型。

```
type T = 'a'|'b'|'c';

let foo = 'a';
let bar:T = foo as T; // 正确
```

上面示例中，最后一行的`foo as T`表示告诉编译器，变量`foo`的类型断言为`T`，所以这一行不再需要类型推断了，编译器直接把`foo`的类型当作`T`，就不会报错了。

总之，类型断言并不是真的改变一个值的类型，而是提示编译器，应该如何处理这个值。

类型断言有两种语法。

```
// 语法一：<类型>值
<Type>value

// 语法二：值 as 类型
value as Type
```

上面两种语法是等价的，`value`表示值，`Type`表示类型。早期只有语法一，后来因为 TypeScript 开始支持 React 的 JSX 语法（尖括号表示 HTML 元素），为了避免两者冲突，就引入了语法二。目前，推荐使用语法二。

```
// 语法一
let bar:T = <T>foo;

// 语法二
let bar:T = foo as T;
```

上面示例是两种类型断言的语法，其中的语法一因为跟 JSX 语法冲突，使用时必须关闭 TypeScript 的 React 支持，否则会无法识别。由于这个原因，现在一般都使用语法二。

下面看一个例子。《对象》一章提到过，对象类型有严格字面量检查，如果存在额外的属性会报错。

```
// 报错
const p:{ x: number } = { x: 0, y: 0 };
```

上面示例中，等号右侧是一个对象字面量，多出了属性`y`，导致报错。解决方法就是使用类型断言，可以用两种不同的断言。

```
// 正确
const p0:{ x: number } =
  { x: 0, y: 0 } as { x: number };

// 正确
const p1:{ x: number } =
  { x: 0, y: 0 } as { x: number; y: number };
```

上面示例中，两种类型断言都是正确的。第一种断言将类型改成与等号左边一致，第二种断言使得等号右边的类型是左边类型的子类型，子类型可以赋值给父类型，同时因为存在类型断言，就没有严格字面量检查了，所以不报错。

下面是一个网页编程的实际例子。

```
const username = document.getElementById('username');

if (username) {
  (username as HTMLInputElement).value; // 正确
}
```

上面示例中，变量`username`的类型是`HTMLElement | null`，排除了`null`的情况以后，HTMLElement 类型是没有`value`属性的。如果`username`是一个输入框，那么就可以通过类型断言，将它的类型改成`HTMLInputElement`，就可以读取`value`属性。

注意，上例的类型断言的圆括号是必需的，否则`username`会被断言成`HTMLInputElement.value`，从而报错。

类型断言不应滥用，因为它改变了 TypeScript 的类型检查，很可能埋下错误的隐患。

```
const data:object = {
  a: 1,
  b: 2,
  c: 3
```

```
};

data.length; // 报错

(data as Array<string>).length; // 正确
```

上面示例中，变量`data`是一个对象，没有`length`属性。但是通过类型断言，可以将它的类型断言为数组，这样使用`length`属性就能通过类型检查。但是，编译后的代码在运行时依然会报错，所以类型断言可以让错误的代码通过编译。

类型断言的一大用处是，指定 unknown 类型的变量的具体类型。

```
const value:unknown = 'Hello World';

const s1:string = value; // 报错
const s2:string = value as string; // 正确
```

上面示例中，unknown 类型的变量`value`不能直接赋值给其他类型的变量，但是可以将它断言为其他类型，这样就可以赋值给别的变量了。

类型断言的条件

类型断言并不意味着，可以把某个值断言为任意类型。

```
const n = 1;
const m:string = n as string; // 报错
```

上面示例中，变量`n`是数值，无法把它断言成字符串，TypeScript 会报错。

类型断言的使用前提是，值的实际类型与断言的类型必须满足一个条件。

```
expr as T
```

上面代码中，`expr`是实际的值，`T`是类型断言，它们必须满足下面的条件：`expr`是`T`的子类型，或者`T`是`expr`的子类型。

也就是说，类型断言要求实际的类型与断言的类型兼容，实际类型可以断言为一个更加宽泛的类型（父类型），也可以断言为一个更加精确的类型（子类型），但不能断言为一个完全无关的类型。

但是，如果真的要断言成一个完全无关的类型，也是可以做到的。那就是连续进行两次类型断言，先断言成 unknown 类型或 any 类型，然后再断言为目标类型。因为`any`类型和`unknown`类型是所有其他类型的父类型，所以可以作为两种完全无关的类型的中介。

```
// 或者写成 <T><unknown>expr
expr as unknown as T
```

上面代码中，`expr`连续进行了两次类型断言，第一次断言为`unknown`类型，第二次断言为`T`类型。这样的话，`expr`就可以断言成任意类型`T`，而不报错。

下面是本小节开头那个例子的改写。

```
const n = 1;
const m:string = n as unknown as string; // 正确
```

上面示例中，通过两次类型断言，变量`n`的类型就从数值，变成了完全无关的字符串，从而赋值时不会报错。

as const 断言

如果没有声明变量类型，let 命令声明的变量，会被类型推断为 TypeScript 内置的基本类型之一；const 命令声明的变量，则被推断为值类型常量。

```
// 类型推断为基本类型 string
let s1 = 'JavaScript';

// 类型推断为字符串 “JavaScript”
const s2 = 'JavaScript';
```

上面示例中，变量`s1`的类型被推断为`string`，变量`s2`的类型推断为值类型`JavaScript`。后者是前者的子类型，相当于 const 命令有更强的限定作用，可以缩小变量的类型范围。

有些时候，let 变量会出现一些意想不到的报错，变更成 const 变量就能消除报错。

```
let s = 'JavaScript';

type Lang =
  |'JavaScript'
  |'TypeScript'
  |'Python';

function setLang(language:Lang) {
  /* ... */
}

setLang(s); // 报错
```

上面示例中，最后一行报错，原因是函数`setLang()`的参数`language`类型是`Lang`，这是一个联合类型。但是，传入的字符串`s`的类型被推断为`string`，属于`Lang`的父类型。父类型不能替代子类型，导致报错。

一种解决方法就是把 let 命令改成 const 命令。

```
const s = 'JavaScript';
```

这样的话，变量`s`的类型就是值类型`JavaScript`，它是联合类型`Lang`的子类型，传入函数`setLang()`就不会报错。

另一种解决方法是使用类型断言。TypeScript 提供了一种特殊的类型断言`as const`，用于告诉编译器，推断类型时，可以将这个值推断为常量，即把 let 变量断言为 const 变量，从而把内置的基本类型变更为值类型。

```
let s = 'JavaScript' as const;
setLang(s);  // 正确
```

上面示例中，变量`s`虽然是用 let 命令声明的，但是使用了`as const`断言以后，就等同于是用 const 命令声明的，变量`s`的类型会被推断为值类型`JavaScript`。

使用了`as const`断言以后，let 变量就不能再改变值了。

```
let s = 'JavaScript' as const;
s = 'Python'; // 报错
```

上面示例中，let 命令声明的变量`s`，使用`as const`断言以后，就不能改变值了，否则报错。

注意，`as const`断言只能用于字面量，不能用于变量。

```
let s = 'JavaScript';
setLang(s as const); // 报错
```

上面示例中，`as const`断言用于变量`s`，就报错了。下面的写法可以更清晰地看出这一点。

```
let s1 = 'JavaScript';
let s2 = s1 as const; // 报错
```

另外，`as const`也不能用于表达式。

```
let s = ('Java' + 'Script') as const; // 报错
```

上面示例中，`as const`用于表达式，导致报错。

`as const`也可以写成前置的形式。

```
// 后置形式
expr as const

// 前置形式
<const>expr
```

`as const`断言可以用于整个对象，也可以用于对象的单个属性，这时它的类型缩小效果是不一样的。

```
const v1 = {
  x: 1,
  y: 2,
}; // 类型是 { x: number; y: number; }

const v2 = {
```

```
  x: 1 as const,
  y: 2,
}; // 类型是 { x: 1; y: number; }

const v3 = {
  x: 1,
  y: 2,
} as const; // 类型是 { readonly x: 1; readonly y: 2; }
```

上面示例中，第二种写法是对属性`x`缩小类型，第三种写法是对整个对象缩小类型。

总之，`as const`会将字面量的类型断言为不可变类型，缩小成 TypeScript 允许的最小类型。

下面是数组的例子。

```
// a1 的类型推断为 number[]
const a1 = [1, 2, 3];

// a2 的类型推断为 readonly [1, 2, 3]
const a2 = [1, 2, 3] as const;
```

上面示例中，数组字面量使用`as const`断言后，类型推断就变成了只读元组。

由于`as const`会将数组变成只读元组，所以很适合用于函数的 rest 参数。

```
function add(x:number, y:number) {
  return x + y;
}

const nums = [1, 2];
const total = add(...nums); // 报错
```

上面示例中，变量`nums`的类型推断为`number[]`，导致使用扩展运算符`...`传入函数`add()`会报错，因为`add()`只能接受两个参数，而`...nums`并不能保证参数的个数。

事实上，对于固定参数个数的函数，如果传入的参数包含扩展运算符，那么扩展运算符只能用于元组。只有当函数定义使用了 rest 参数，扩展运算符才能用于数组。

解决方法就是使用`as const`断言，将数组变成元组。

```
const nums = [1, 2] as const;
const total = add(...nums); // 正确
```

上面示例中，使用`as const`断言后，变量`nums`的类型会被推断为`readonly [1, 2]`，使用扩展运算符展开后，正好符合函数`add()`的参数类型。

Enum 成员也可以使用`as const`断言。

```
enum Foo {
  X,
```

```
  Y,
}
let e1 = Foo.X;            // Foo
let e2 = Foo.X as const;   // Foo.X
```

上面示例中，如果不使用`as const`断言，变量`e1`的类型被推断为整个 Enum 类型；使用了`as const`断言以后，变量`e2`的类型被推断为 Enum 的某个成员，这意味着它不能变更为其他成员。

非空断言

对于那些可能为空的变量（即可能等于`undefined`或`null`），TypeScript 提供了非空断言，保证这些变量不会为空，写法是在变量名后面加上感叹号`!`。

```
function f(x?:number|null) {
  validateNumber(x); // 自定义函数，确保 x 是数值
  console.log(x!.toFixed());
}

function validateNumber(e?:number|null) {
  if (typeof e !== 'number')
    throw new Error('Not a number');
}
```

上面示例中，函数`f()`的参数`x`的类型是`number|null`，即可能为空。如果为空，就不存在`x.toFixed()`方法，这样写会报错。但是，开发者可以确认，经过`validateNumber()`的前置检验，变量`x`肯定不会为空，这时就可以使用非空断言，为函数体内部的变量`x`加上后缀`!`，`x!.toFixed()`编译就不会报错了。

非空断言在实际编程中很有用，有时可以省去一些额外的判断。

```
const root = document.getElementById('root');

// 报错
root.addEventListener('click', e => {
  /* ... */
});
```

上面示例中，`getElementById()`有可能返回空值`null`，即变量`root`可能为空，这时对它调用`addEventListener()`方法就会报错，通不过编译。但是，开发者如果可以确认`root`元素肯定会在网页中存在，这时就可以使用非空断言。

```
const root = document.getElementById('root')!;
```

上面示例中，`getElementById()`方法加上后缀`!`，表示这个方法肯定返回非空结果。

不过，非空断言会造成安全隐患，只有在确定一个表达式的值不为空时才能使用。比较保险的做法还是手动检查一下是否为空。

```
const root = document.getElementById('root');

if (root === null) {
  throw new Error('Unable to find DOM element #root');
}

root.addEventListener('click', e => {
  /* ... */
});
```

上面示例中，如果`root`为空会抛错，比非空断言更保险一点。

非空断言还可以用于赋值断言。TypeScript 有一个编译设置，要求类的属性必须初始化（即有初始值），如果不对属性赋值就会报错。

```
class Point {
  x:number; // 报错
  y:number; // 报错

  constructor(x:number, y:number) {
    // ...
  }
}
```

上面示例中，属性`x`和`y`会报错，因为 TypeScript 认为它们没有初始化。

这时就可以使用非空断言，表示这两个属性肯定会有值，这样就不会报错了。

```
class Point {
  x!:number; // 正确
  y!:number; // 正确

  constructor(x:number, y:number) {
    // ...
  }
}
```

另外，非空断言只有在打开编译选项`strictNullChecks`时才有意义。如果不打开这个选项，编译器就不会检查某个变量是否可能为`undefined`或`null`。

断言函数

断言函数是一种特殊函数，用于保证函数参数符合某种类型。如果函数参数达不到要求，就会抛出错误，中断程序执行；如果达到要求，就不进行任何操作，让代码按照正常流程运行。

```
function isString(value:unknown):void {
  if (typeof value !== 'string')
    throw new Error('Not a string');
}
```

上面示例中，函数`isString()`就是一个断言函数，用来保证参数`value`是一个字符串，否则就会抛出错误，中断程序的执行。

下面是它的用法。

```
function toUpper(x: string|number) {
  isString(x);
  return x.toUpperCase();
}
```

上面示例中，函数`toUpper()`的参数`x`，可能是字符串，也可能是数值。但是，函数体的最后一行调用`toUpperCase()`方法，必须保证`x`是字符串，否则报错。所以，这一行前面调用断言函数`isString()`，调用以后 TypeScript 就能确定，变量`x`一定是字符串，不是数值，也就不报错了。

传统的断言函数`isString()`的写法有一个缺点，它的参数类型是`unknown`，返回值类型是`void`（即没有返回值）。单单从这样的类型声明，很难看出`isString()`是一个断言函数。

为了更清晰地表达断言函数，TypeScript 3.7 引入了新的类型写法。

```
function isString(value:unknown):asserts value is string {
  if (typeof value !== 'string')
    throw new Error('Not a string');
}
```

上面示例中，函数`isString()`的返回值类型写成`asserts value is string`，其中`asserts`和`is`都是关键词，`value`是函数的参数名，`string`是函数参数的预期类型。它的意思是，该函数用来断言参数`value`的类型是`string`，如果达不到要求，程序就会在这里中断。

使用了断言函数的新写法以后，TypeScript 就会自动识别，只要执行了该函数，对应的变量都为断言的类型。

注意，函数返回值的断言写法，只是用来更清晰地表达函数意图，真正的检查是需要开发者自己部署的。而且，如果内部的检查与断言不一致，TypeScript 也不会报错。

```
function isString(value:unknown):asserts value is string {
  if (typeof value !== 'number')
    throw new Error('Not a number');
}
```

上面示例中，函数的断言是参数`value`类型为字符串，但是实际上，内部检查的却是它是否为数值，如果不是就抛错。这段代码能够正常通过编译，表示 TypeScript 并不会检查断言与实际的类型检查是否一致。

另外，断言函数的`asserts`语句等同于`void`类型，所以如果返回除了`undefined`和`null`以外的值，都会报错。

```
function isString(value:unknown):asserts value is string {
  if (typeof value !== 'string')
    throw new Error('Not a string');
```

```
  return true; // 报错
}
```

上面示例中，断言函数返回了`true`，导致报错。

下面是另一个例子。

```
type AccessLevel = 'r' | 'w' | 'rw';

function allowsReadAccess(
  level:AccessLevel
):asserts level is 'r' | 'rw' {
  if (!level.includes('r'))
    throw new Error('Read not allowed');
}
```

上面示例中，函数`allowsReadAccess()`用来断言参数`level`一定等于`r`或`rw`。

如果要断言参数非空，可以使用工具类型`NonNullable<T>`。

```
function assertIsDefined<T>(
  value:T
):asserts value is NonNullable<T> {
  if (value === undefined || value === null) {
    throw new Error(`${value} is not defined`)
  }
}
```

上面示例中，工具类型`NonNullable<T>`对应类型`T`去除空类型后的剩余类型。

如果要将断言函数用于函数表达式，可以采用下面的写法。根据 TypeScript 的要求，这时函数表达式所赋予的变量，必须有明确的类型声明。

```
type AssertIsNumber =
  (value:unknown) => asserts value is number;

const assertIsNumber:AssertIsNumber = (value) => {
  if (typeof value !== 'number')
    throw Error('Not a number');
};
```

注意，断言函数与类型保护函数（type guard）是两种不同的函数。它们的区别是，断言函数不返回值，而类型保护函数总是返回一个布尔值。

```
function isString(
  value:unknown
):value is string {
  return typeof value === 'string';
}
```

上面示例就是一个类型保护函数`isString()`，作用是检查参数`value`是否为字符串。如果是的，返回`true`，否则返回`false`。该函数的返回值类型是`value is string`，其中的`is`是一个类型运算符，如果左侧的值符合右侧的类型，则返回`true`，否则返回`false`。

如果要断言某个参数保证为真（即不等于`false`、`undefined`和`null`），TypeScript 提供了断言函数的一种简写形式。

```
function assert(x:unknown):asserts x {
  // ...
}
```

上面示例中，函数`assert()`的断言部分，`asserts x`省略了谓语和宾语，表示参数`x`保证为真（`true`）。

同样的，参数为真的实际检查需要开发者自己实现。

```
function assert(x:unknown):asserts x {
  if (!x) {
    throw new Error(`${x} should be a truthy value.`);
  }
}
```

这种断言函数的简写形式，通常用来检查某个操作是否成功。

```
type Person = {
  name: string;
  email?: string;
};

function loadPerson(): Person | null {
  return null;
}

let person = loadPerson();

function assert(
  condition: unknown,
  message: string
):asserts condition {
  if (!condition) throw new Error(message);
}

// Error: Person is not defined
assert(person, 'Person is not defined');
console.log(person.name);
```

上面示例中，只有`loadPerson()`返回结果为真（即操作成功），`assert()`才不会报错。

参考链接

- Const Assertions in Literal Expressions in TypeScript, Marius Schulz

- Assertion Functions in TypeScript, Marius Schulz
- Assertion functions in TypeScript, Matteo Di Pirro

TypeScript 模块

简介

任何包含 import 或 export 语句的文件，就是一个模块（module）。相应地，如果文件不包含 export 语句，就是一个全局的脚本文件。

模块本身就是一个作用域，不属于全局作用域。模块内部的变量、函数、类只在内部可见，对于模块外部是不可见的。暴露给外部的接口，必须用 export 命令声明；如果其他文件要使用模块的接口，必须用 import 命令来输入。

如果一个文件不包含 export 语句，但是希望把它当作一个模块（即内部变量对外不可见），可以在脚本头部添加一行语句。

```
export {};
```

上面这行语句不产生任何实际作用，但会让当前文件被当作模块处理，所有它的代码都变成了内部代码。

ES 模块的详细介绍，请参考 ES6 教程，这里就不重复了。本章主要介绍 TypeScript 的模块处理。

TypeScript 模块除了支持所有 ES 模块的语法，特别之处在于允许输出和输入类型。

```
export type Bool = true | false;
```

上面示例中，当前脚本输出一个类型别名`Bool`。这行语句把类型定义和接口输出写在一行，也可以写成两行。

```
type Bool = true | false;

export { Bool };
```

假定上面的模块文件为`a.ts`，另一个文件`b.ts`就可以使用 import 语句，输入这个类型。

```
import { Bool } from './a';

let foo:Bool = true;
```

上面示例中，import 语句加载的是一个类型。注意，加载文件写成`./a`，没有写脚本文件的后缀名。TypeScript 允许加载模块时，省略模块文件的后缀名，它会自动定位，将`./a`定位到`./a.ts`。

编译时，可以两个脚本同时编译。

```
$ tsc a.ts b.ts
```

上面命令会将`a.ts`和`b.ts`分别编译成`a.js`和`b.js`。

也可以只编译`b.ts`，因为它是入口脚本，tsc 会自动编译它依赖的所有脚本。

```
$ tsc b.ts
```

上面命令发现`b.ts`依赖`a.ts`，就会自动寻找`a.ts`，也将其同时编译，因此编译产物还是`a.js`和`b.js`两个文件。

import type 语句

import 在一条语句中，可以同时输入类型和正常接口。

```
// a.ts
export interface A {
  foo: string;
}

export let a = 123;

// b.ts
import { A, a } from './a';
```

上面示例中，文件`a.ts`的 export 语句输出了一个类型`A`和一个正常接口`a`，另一个文件`b.ts`则在同一条语句中输入了类型和正常接口。

这样很不利于区分类型和正常接口，容易造成混淆。为了解决这个问题，TypeScript 引入了两个解决方法。

第一个方法是在 import 语句输入的类型前面加上`type`关键字。

```
import { type A, a } from './a';
```

上面示例中，import 语句输入的类型`A`前面有`type`关键字，表示这是一个类型。

第二个方法是使用 import type 语句，这个语句只用来输入类型，不用来输入正常接口。

```
// 正确
import type { A } from './a';
let b:A = 'hello';

// 报错
import type { a } from './a';
let b = a;
```

上面示例中，import type 输入类型`A`是正确的，可以把`A`当作类型使用。但是，输入正常接口`a`，并把`a`当作一个值使用，就会报错。这就是说，看到`import type`，你就知道它输入的肯定是类型。

import type 语句也可以输入默认类型。

```
import type DefaultType from 'moduleA';
```

import type 在一个名称空间下，输入所有类型的写法如下。

```
import type * as TypeNS from 'moduleA';
```

同样的，export 语句也有两种方法，表示输出的是类型。

```
type A = 'a';
type B = 'b';

// 方法一
export {type A, type B};

// 方法二
export type {A, B};
```

上面示例中，方法一是使用`type`关键字作为前缀，表示输出的是类型；方法二是使用 export type 语句，表示整行输出的都是类型。

下面是 export type 将一个类作为类型输出的例子。

```
class Point {
  x: number;
  y: number;
}

export type { Point };
```

上面示例中，由于使用了 export type 语句，输出的并不是 Point 这个类，而是 Point 代表的实例类型。输入时，只能作为类型输入。

```
import type { Point } from './module';

const p:Point = { x: 0, y: 0 };
```

上面示例中，`Point`只能作为类型输入，不能当作正常接口使用。

importsNotUsedAsValues 编译设置

TypeScript 特有的输入类型（type）的 import 语句，编译成 JavaScript 时怎么处理呢?

TypeScript 提供了`importsNotUsedAsValues`编译设置项，有三个可能的值。

（1）`remove`：这是默认值，自动删除输入类型的 import 语句。

（2）`preserve`：保留输入类型的 import 语句。

（3）`error`：保留输入类型的 import 语句（与`preserve`相同），但是必须写成`import type`的形式，否则报错。

请看示例，下面是一个输入类型的 import 语句。

```
import { TypeA } from './a';
```

上面示例中，`TypeA`是一个类型。

`remove`的编译结果会将该语句删掉。

`preserve`的编译结果会保留该语句，但会删掉其中涉及类型的部分。

```
import './a';
```

上面就是`preserve`的编译结果，可以看到编译后的`import`语句不从`a.js`输入任何接口（包括类型），但是会引发`a.js`的执行，因此会保留`a.js`里面的副作用。

`error`的编译结果与`preserve`相同，但在编译过程中会报错，因为它要求输入类型的`import`语句必须写成`import type`的形式。原始语句改成下面的形式，就不会报错。

```
import type { TypeA } from './a';
```

CommonJS 模块

CommonJS 是 Node.js 的专用模块格式，与 ES 模块格式不兼容。

import = 语句

TypeScript 使用`import =`语句输入 CommonJS 模块。

```
import fs = require('fs');
const code = fs.readFileSync('hello.ts', 'utf8');
```

上面示例中，使用`import =`语句和`require()`命令输入了一个 CommonJS 模块。模块本身的用法跟 Node.js 是一样的。

除了使用`import =`语句，TypeScript 还允许使用`import * as [接口名] from "模块文件"`输入 CommonJS 模块。

```
import * as fs from 'fs';
// 等同于
import fs = require('fs');
```

export = 语句

TypeScript 使用`export =`语句，输出 CommonJS 模块的对象，等同于 CommonJS 的`module.exports`对象。

```
let obj = { foo: 123 };

export = obj;
```

`export =`语句输出的对象，只能使用`import =`语句加载。

```
import obj = require('./a');

console.log(obj.foo); // 123
```

模块定位

模块定位（module resolution）指的是一种算法，用来确定 import 语句和 export 语句里面的模块文件位置。

```
// 相对模块
import { TypeA } from './a';

// 非相对模块
import * as $ from "jquery";
```

上面示例中，TypeScript 怎么确定`./a`或`jquery`到底是指哪一个模块，具体位置在哪里，用到的算法就叫做“模块定位”。

编译参数`moduleResolution`，用来指定具体使用哪一种定位算法。常用的算法有两种：`Classic`和`Node`。

如果没有指定`moduleResolution`，它的默认值与编译参数`module`有关。`module`设为`commonjs`时（项目脚本采用 CommonJS 模块格式），`moduleResolution`的默认值为`Node`，即采用 Node.js 的模块定位算法。其他情况下（`module`设为 es2015、 esnext、amd, system, umd 等等），就采用`Classic`定位算法。

相对模块，非相对模块

加载模块时，目标模块分为相对模块（relative import）和非相对模块两种（non-relative import）。

相对模块指的是路径以`/`、`./`、`../`开头的模块。下面 import 语句加载的模块，都是相对模块。

- `import Entry from "./components/Entry";`
- `import { DefaultHeaders } from "../constants/http";`
- `import "/mod";`

相对模块的定位，是根据当前脚本的位置进行计算的，一般用于保存在当前项目目录结构中的模块脚本。

非相对模块指的是不带有路径信息的模块。下面 import 语句加载的模块，都是非相对模块。

- `import * as $ from "jquery";`
- `import { Component } from "@angular/core";`

非相对模块的定位，是由`baseUrl`属性或模块映射而确定的，通常用于加载外部模块。

Classic 方法

Classic 方法以当前脚本的路径作为“基准路径”，计算相对模块的位置。比如，脚本`a.ts`里面有一行代码`import { b } from "./b"`，那么 TypeScript 就会在`a.ts`所在的目录，查找`b.ts`和`b.d.ts`。

至于非相对模块，也是以当前脚本的路径作为起点，一层层查找上级目录。比如，脚本`a.ts`里面有一行代码`import { b } from "b"`，那么就会依次在每一级上层目录里面，查找`b.ts`和`b.d.ts`。

Node 方法

Node 方法就是模拟 Node.js 的模块加载方法，也就是`require()`的实现方法。

相对模块依然是以当前脚本的路径作为“基准路径”。比如，脚本文件`a.ts`里面有一行代码`let x = require("./b");`，TypeScript 按照以下顺序查找。

1. 当前目录是否包含`b.ts`、`b.tsx`、`b.d.ts`。如果不存在就执行下一步。
2. 当前目录是否存在子目录`b`，该子目录里面的`package.json`文件是否有`types`字段指定了模块入口文件。如果不存在就执行下一步。
3. 当前目录的子目录`b`是否包含`index.ts`、`index.tsx`、`index.d.ts`。如果不存在就报错。

非相对模块则是以当前脚本的路径作为起点，逐级向上层目录查找是否存在子目录`node_modules`。比如，脚本文件`a.js`有一行`let x = require("b");`，TypeScript 按照以下顺序进行查找。

1. 当前目录的子目录`node_modules`是否包含`b.ts`、`b.tsx`、`b.d.ts`。
2. 当前目录的子目录`node_modules`，是否存在文件`package.json`，该文件的`types`字段是否指定了入口文件，如果是的就加载该文件。
3. 当前目录的子目录`node_modules`里面，是否包含子目录`@types`，在该目录中查找文件`b.d.ts`。
4. 当前目录的子目录`node_modules`里面，是否包含子目录`b`，在该目录中查找`index.ts`、`index.tsx`、`index.d.ts`。
5. 进入上一层目录，重复上面4步，直到找到为止。

路径映射

TypeScript 允许开发者在`tsconfig.json`文件里面，手动指定脚本模块的路径。

（1）baseUrl

`baseUrl`字段可以手动指定脚本模块的基准目录。

```
{
  "compilerOptions": {
    "baseUrl": "."
  }
}
```

上面示例中，`baseUrl`是一个点，表示基准目录就是`tsconfig.json`所在的目录。

（2）paths

`paths`字段指定非相对路径的模块与实际脚本的映射。

```
{
  "compilerOptions": {
    "baseUrl": ".",
    "paths": {
      "jquery": ["node_modules/jquery/dist/jquery"]
    }
  }
}
```

上面示例中，加载模块`jquery`时，实际加载的脚本是`node_modules/jquery/dist/jquery`，它的位置要根据`baseUrl`字段计算得到。

注意，上例的`jquery`属性的值是一个数组，可以指定多个路径。如果第一个脚本路径不存在，那么就加载第二个路径，以此类推。

（3）rootDirs

`rootDirs`字段指定模块定位时必须查找的其他目录。

```
{
  "compilerOptions": {
    "rootDirs": ["src/zh", "src/de", "src/#{locale}"]
  }
}
```

上面示例中，`rootDirs`指定了模块定位时，需要查找的不同的国际化目录。

tsc 的`--traceResolution`参数

由于模块定位的过程很复杂，tsc 命令有一个`--traceResolution`参数，能够在编译时在命令行显示模块定位的每一步。

```
$ tsc --traceResolution
```

上面示例中，`traceResolution`会输出模块定位的判断过程。

tsc 的--noResolve参数

tsc 命令的`--noResolve`参数，表示模块定位时，只考虑在命令行传入的模块。

举例来说，`app.ts`包含如下两行代码。

```
import * as A from "moduleA";
import * as B from "moduleB";
```

使用下面的命令进行编译。

```
$ tsc app.ts moduleA.ts --noResolve
```

上面命令使用`--noResolve`参数，因此可以定位到`moduleA.ts`，因为它从命令行传入了；无法定位到`moduleB`，因为它没有传入，因此会报错。

参考链接

- tsconfig 之 importsNotUsedAsValues 属性

TypeScript namespace

namespace 是一种将相关代码组织在一起的方式，中文译为“命名空间”。

它出现在 ES 模块诞生之前，作为 TypeScript 自己的模块格式而发明的。但是，自从有了 ES 模块，官方已经不推荐使用 namespace 了。

基本用法

namespace 用来建立一个容器，内部的所有变量和函数，都必须在这个容器里面使用。

```
namespace Utils {
  function isString(value:any) {
    return typeof value === 'string';
  }

  // 正确
  isString('yes');
}

Utils.isString('no'); // 报错
```

上面示例中，命名空间`Utils`里面定义了一个函数`isString()`，它只能在`Utils`里面使用，如果用于外部就会报错。

如果要在命名空间以外使用内部成员，就必须为该成员加上`export`前缀，表示对外输出该成员。

```
namespace Utility {
  export function log(msg:string) {
    console.log(msg);
  }
  export function error(msg:string) {
    console.error(msg);
  }
}

Utility.log('Call me');
Utility.error('maybe!');
```

上面示例中，只要加上`export`前缀，就可以在命名空间外部使用内部成员。

编译出来的 JavaScript 代码如下。

```
var Utility;

(function (Utility) {
  function log(msg) {
    console.log(msg);
```

```
  }
  Utility.log = log;
  function error(msg) {
    console.error(msg);
  }
  Utility.error = error;
})(Utility || (Utility = {}));
```

上面代码中，命名空间`Utility`变成了 JavaScript 的一个对象，凡是`export`的内部成员，都成了该对象的属性。

这就是说，namespace 会变成一个值，保留在编译后的代码中。这一点要小心，它不是纯的类型代码。

namespace 内部还可以使用`import`命令输入外部成员，相当于为外部成员起别名。当外部成员的名字比较长时，别名能够简化代码。

```
namespace Utils {
  export function isString(value:any) {
    return typeof value === 'string';
  }
}

namespace App {
  import isString = Utils.isString;

  isString('yes');
  // 等同于
  Utils.isString('yes');
}
```

上面示例中，`import`命令指定在命名空间`App`里面，外部成员`Utils.isString`的别名为`isString`。

`import`命令也可以在 namespace 外部，指定别名。

```
namespace Shapes {
  export namespace Polygons {
    export class Triangle {}
    export class Square {}
  }
}

import polygons = Shapes.Polygons;

// 等同于 new Shapes.Polygons.Square()
let sq = new polygons.Square();
```

上面示例中，`import`命令在命名空间`Shapes`的外部，指定`Shapes.Polygons`的别名为`polygons`。

namespace 可以嵌套。

```
namespace Utils {
  export namespace Messaging {
```

```
    export function log(msg:string) {
      console.log(msg);
    }
  }
}

Utils.Messaging.log('hello') // "hello"
```

上面示例中，命名空间`Utils`内部还有一个命名空间`Messaging`。注意，如果要在外部使用`Messaging`，必须在它前面加上`export`命令。

使用嵌套的命名空间，必须从最外层开始引用，比如`Utils.Messaging.log()`。

namespace 不仅可以包含实义代码，还可以包括类型代码。

```
namespace N {
  export interface MyInterface{}
  export class MyClass{}
}
```

上面代码中，命令空间`N`不仅对外输出类，还对外输出一个接口，它们都可以用作类型。

namespace 与模块的作用是一致的，都是把相关代码组织在一起，对外输出接口。区别是一个文件只能有一个模块，但可以有多个 namespace。由于模块可以取代 namespace，而且是 JavaScript 的标准语法，还不需要编译转换，所以建议总是使用模块，替代 namespace。

如果 namespace 代码放在一个单独的文件里，那么引入这个文件需要使用三斜杠的语法。

```
/// <reference path = "SomeFileName.ts" />
```

namespace 的输出

namespace 本身也可以使用`export`命令输出，供其他文件使用。

```
// shapes.ts
export namespace Shapes {
  export class Triangle {
    // ...
  }
  export class Square {
    // ...
  }
}
```

上面示例是一个文件`shapes.ts`，里面使用`export`命令，输出了一个命名空间`Shapes`。

其他脚本文件使用`import`命令，加载这个命名空间。

```
// 写法一
import { Shapes } from './shapes';
let t = new Shapes.Triangle();

// 写法二
import * as shapes from "./shapes";
let t = new shapes.Shapes.Triangle();
```

不过，更好的方法还是建议使用模块，采用模块的输出和输入。

```
// shapes.ts
export class Triangle {
  /* ... */
}
export class Square {
  /* ... */
}

// shapeConsumer.ts
import * as shapes from "./shapes";
let t = new shapes.Triangle();
```

上面示例中，使用模块的输出和输入，改写了前面的例子。

namespace 的合并

多个同名的 namespace 会自动合并，这一点跟 interface 一样。

```
namespace Animals {
  export class Cat {}
}
namespace Animals {
  export interface Legged {
    numberOfLegs: number;
  }
  export class Dog {}
}

// 等同于
namespace Animals {
  export interface Legged {
    numberOfLegs: number;
  }
  export class Cat {}
  export class Dog {}
}
```

这样做的目的是，如果同名的命名空间分布在不同的文件中，TypeScript 最终会将它们合并在一起。这样就比较方便扩展别人的代码。

合并命名空间时，命名空间中的非`export`的成员不会被合并，但是它们只能在各自的命名空间中使用。

```
namespace N {
  const a = 0;

  export function foo() {
    console.log(a);  // 正确
  }
}

namespace N {
  export function bar() {
    foo(); // 正确
    console.log(a);  // 报错
  }
}
```

上面示例中，变量`a`是第一个名称空间`N`的非对外成员，它只在第一个名称空间可用。

命名空间还可以跟同名函数合并，但是要求同名函数必须在命名空间之前声明。这样做是为了确保先创建出一个函数对象，然后同名的命名空间就相当于给这个函数对象添加额外的属性。

```
function f() {
  return f.version;
}

namespace f {
  export const version = '1.0';
}

f()   // '1.0'
f.version // '1.0'
```

上面示例中，函数`f()`与命名空间`f`合并，相当于命名空间为函数对象`f`添加属性。

命名空间也能与同名 class 合并，同样要求class 必须在命名空间之前声明，原因同上。

```
class C {
  foo = 1;
}

namespace C {
  export const bar = 2;
}

C.bar // 2
```

上面示例中，名称空间`C`为类`C`添加了一个静态属性`bar`。

命名空间还能与同名 Enum 合并。

```
enum E {
  A,
  B,
```

```
  C,
}

namespace E {
  export function foo() {
    console.log(E.C);
  }
}

E.foo() // 2
```

上面示例中，命名空间`E`为枚举`E`添加了一个`foo()`方法。

注意，Enum 成员与命名空间导出成员不允许同名。

```
enum E {
  A, // 报错
  B,
}

namespace E {
  export function A() {} // 报错
}
```

上面示例中，同名 Enum 与命名空间有同名成员，结果报错。

TypeScript 装饰器

简介

装饰器（Decorator）是一种语法结构，用来在定义时修改类（class）的行为。

在语法上，装饰器有如下几个特征。

（1）第一个字符（或者说前缀）是`@`，后面是一个表达式。

（2）`@`后面的表达式，必须是一个函数（或者执行后可以得到一个函数）。

（3）这个函数接受所修饰对象的一些相关值作为参数。

（4）这个函数要么不返回值，要么返回一个新对象取代所修饰的目标对象。

举例来说，有一个函数`Injectable()`当作装饰器使用，那么需要写成`@Injectable`，然后放在某个类的前面。

```
@Injectable class A {
  // ...
}
```

上面示例中，由于有了装饰器`@Injectable`，类`A`的行为在运行时就会发生改变。

下面就是一个最简单的装饰器。

```
function simpleDecorator() {
  console.log('hi');
}

@simpleDecorator
class A {} // "hi"
```

上面示例中，函数`simpleDecorator()`用作装饰器，附加在类`A`之上，后者在代码解析时就会打印一行日志。

编译上面的代码会报错，提示没有用到装饰器的参数。现在就为装饰器加上参数，让它更像正式运行的代码。

```
function simpleDecorator(
  value:any,
  context:any
) {
  console.log(`hi, this is ${context.kind} ${context.name}`);
  return value;
}
```

```
@simpleDecorator
class A {} // "hi, this is class A"
```

上面的代码就可以顺利通过编译了，代码含义这里先不解释。大家只要理解，类`A`在执行前会先执行装饰器`simpleDecorator()`，并且会向装饰器自动传入参数就可以了。

装饰器有多种形式，基本上只要在`@`符号后面添加表达式都是可以的。下面都是合法的装饰器。

```
@myFunc
@myFuncFactory(arg1, arg2)

@libraryModule.prop
@someObj.method(123)

@(wrap(dict['prop']))
```

注意，`@`后面的表达式，最终执行后得到的应该是一个函数。

相比使用子类改变父类，装饰器更加简洁优雅，缺点是不那么直观，功能也受到一些限制。所以，装饰器一般只用来为类添加某种特定行为。

```
@frozen class Foo {

  @configurable(false)
  @enumerable(true)
  method() {}

  @throttle(500)
  expensiveMethod() {}
}
```

上面示例中，一共有四个装饰器，一个用在类本身（`@frozen`），另外三个用在类的方法（`@configurable`、`@enumerable`、`@throttle`）。它们不仅增加了代码的可读性，清晰地表达了意图，而且提供一种方便的手段，增加或修改类的功能。

装饰器的版本

TypeScript 从早期开始，就支持装饰器。但是，装饰器的语法后来发生了变化。ECMAScript 标准委员会最终通过的语法标准，与 TypeScript 早期使用的语法有很大差异。

目前，TypeScript 5.0 同时支持两种装饰器语法。标准语法可以直接使用，传统语法需要打开`--experimentalDecorators`编译参数。

```
$ tsc --target ES5 --experimentalDecorators
```

本章介绍装饰器的标准语法，下一章介绍传统语法。

装饰器的结构

装饰器函数的类型定义如下。

```
type Decorator = (
  value: DecoratedValue,
  context: {
    kind: string;
    name: string | symbol;
    addInitializer?(initializer: () => void): void;
    static?: boolean;
    private?: boolean;
    access: {
      get?(): unknown;
      set?(value: unknown): void;
    };
  }
) => void | ReplacementValue;
```

上面代码中，`Decorator`是装饰器的类型定义。它是一个函数，使用时会接收到`value`和`context`两个参数。

- `value`：所装饰的对象。
- `context`：上下文对象，TypeScript 提供一个原生接口`ClassMethodDecoratorContext`，描述这个对象。

```
function decorator(
  value:any,
  context:ClassMethodDecoratorContext
) {
  // ...
}
```

上面是一个装饰器函数，其中第二个参数`context`的类型就可以写成`ClassMethodDecoratorContext`。

`context`对象的属性，根据所装饰对象的不同而不同，其中只有两个属性（`kind`和`name`）是必有的，其他都是可选的。

（1）`kind`：字符串，表示所装饰对象的类型，可能取以下的值。

- 'class'
- 'method'
- 'getter'
- 'setter'
- 'field'
- 'accessor'

这表示一共有六种类型的装饰器。

（2）`name`：字符串或者 Symbol 值，所装饰对象的名字，比如类名、属性名等。

（3）`addInitializer()`：函数，用来添加类的初始化逻辑。以前，这些逻辑通常放在构造函数里面，对方法进行初始化，现在改成以函数形式传入`addInitializer()`方法。注意，`addInitializer()`没有返回值。

（4）`private`：布尔值，表示所装饰的对象是否为类的私有成员。

（5）`static`：布尔值，表示所装饰的对象是否为类的静态成员。

（6）`access`：一个对象，包含了某个值的 get 和 set 方法。

类装饰器

类装饰器的类型描述如下。

```
type ClassDecorator = (
  value: Function,
  context: {
    kind: 'class';
    name: string | undefined;
    addInitializer(initializer: () => void): void;
  }
) => Function | void;
```

类装饰器接受两个参数：`value`（当前类本身）和`context`（上下文对象）。其中，`context`对象的`kind`属性固定为字符串`class`。

类装饰器一般用来对类进行操作，可以不返回任何值，请看下面的例子。

```
function Greeter(value, context) {
  if (context.kind === 'class') {
    value.prototype.greet = function () {
      console.log('你好');
    };
  }
}

@Greeter
class User {}

let u = new User();
u.greet(); // "你好"
```

上面示例中，类装饰器`@Greeter`在类`User`的原型对象上，添加了一个`greet()`方法，实例就可以直接使用该方法。

类装饰器可以返回一个函数，替代当前类的构造方法。

```
function countInstances(value:any, context:any) {
  let instanceCount = 0;

  const wrapper = function (...args:any[]) {
```

```
    instanceCount++;
    const instance = new value(...args);
    instance.count = instanceCount;
    return instance;
  } as unknown as typeof MyClass;

  wrapper.prototype = value.prototype; // A
  return wrapper;
}

@countInstances
class MyClass {}

const inst1 = new MyClass();
inst1 instanceof MyClass // true
inst1.count // 1
```

上面示例中，类装饰器`@countInstances`返回一个函数，替换了类`MyClass`的构造方法。新的构造方法实现了实例的计数，每新建一个实例，计数器就会加一，并且对实例添加`count`属性，表示当前实例的编号。

注意，上例为了确保新构造方法继承定义在`MyClass`的原型之上的成员，特别加入`A`行，确保两者的原型对象是一致的。否则，新的构造函数`wrapper`的原型对象，与`MyClass`不同，通不过`instanceof`运算符。

类装饰器也可以返回一个新的类，替代原来所装饰的类。

```
function countInstances(value:any, context:any) {
  let instanceCount = 0;

  return class extends value {
    constructor(...args:any[]) {
      super(...args);
      instanceCount++;
      this.count = instanceCount;
    }
  };
}

@countInstances
class MyClass {}

const inst1 = new MyClass();
inst1 instanceof MyClass // true
inst1.count // 1
```

上面示例中，`@countInstances`返回一个`MyClass`的子类。

下面的例子是通过类装饰器，禁止使用`new`命令新建类的实例。

```
function functionCallable(
  value:any, {kind}:any
):any {
  if (kind === 'class') {
    return function (...args:any) {
```

```
      if (new.target !== undefined) {
        throw new TypeError('This function can’t be new-invoked');
      }
      return new value(...args);
    }
  }
}

@functionCallable
class Person {
  name:string;
  constructor(name:string) {
    this.name = name;
  }
}

// @ts-ignore
const robin = Person('Robin');
robin.name // 'Robin'
```

上面示例中，类装饰器`@functionCallable`返回一个新的构造方法，里面判断`new.target`是否不为空，如果是的，就表示通过`new`命令调用，从而报错。

类装饰器的上下文对象`context`的`addInitializer()`方法，用来定义一个类的初始化函数，在类完全定义结束后执行。

```
function customElement(name: string) {
  return <Input extends new (...args: any) => any>(
    value: Input,
    context: ClassDecoratorContext
  ) => {
    context.addInitializer(function () {
      customElements.define(name, value);
    });
  };
}

@customElement("hello-world")
class MyComponent extends HTMLElement {
  constructor() {
    super();
  }
  connectedCallback() {
    this.innerHTML = `<h1>Hello World</h1>`;
  }
}
```

上面示例中，类`MyComponent`定义完成后，会自动执行类装饰器`@customElement()`给出的初始化函数，该函数会将当前类注册为指定名称（本例为`<hello-world>`）的自定义 HTML 元素。

方法装饰器

方法装饰器用来装饰类的方法（method）。它的类型描述如下。

```
type ClassMethodDecorator = (
  value: Function,
  context: {
    kind: 'method';
    name: string | symbol;
    static: boolean;
    private: boolean;
    access: { get: () => unknown };
    addInitializer(initializer: () => void): void;
  }
) => Function | void;
```

根据上面的类型，方法装饰器是一个函数，接受两个参数：`value`和`context`。

参数`value`是方法本身，参数`context`是上下文对象，有以下属性。

- `kind`：值固定为字符串`method`，表示当前为方法装饰器。
- `name`：所装饰的方法名，类型为字符串或 Symbol 值。
- `static`：布尔值，表示是否为静态方法。该属性为只读属性。
- `private`：布尔值，表示是否为私有方法。该属性为只读属性。
- `access`：对象，包含了方法的存取器，但是只有`get()`方法用来取值，没有`set()`方法进行赋值。
- `addInitializer()`：为方法增加初始化函数。

方法装饰器会改写类的原始方法，实质等同于下面的操作。

```
function trace(decoratedMethod) {
  // ...
}

class C {
  @trace
  toString() {
    return 'C';
  }
}

// `@trace` 等同于
// C.prototype.toString = trace(C.prototype.toString);
```

上面示例中，`@trace`是方法`toString()`的装饰器，它的效果等同于最后一行对`toString()`的改写。

如果方法装饰器返回一个新的函数，就会替代所装饰的原始函数。

```
function replaceMethod(value:any, context:ClassMethodDecoratorContext) {
  return function (this:Person) {
    return `How are you, ${this.name}?`;
```

```
  }
}

class Person {
  name: string;

  constructor(name:string) {
    this.name = name;
  }

  @replaceMethod
  hello() {
    return `Hi ${this.name}!`;
  }
}

const robin = new Person('Robin');

robin.hello() // 'How are you, Robin?'
```

上面示例中，装饰器`@replaceMethod`返回的函数，就成为了新的`hello()`方法。

下面是另一个例子。

```
class Person {
  name: string;
  constructor(name: string) {
    this.name = name;
  }

  @log
  greet() {
    console.log(`Hello, my name is ${this.name}.`);
  }
}

function log(originalMethod:any, context:ClassMethodDecoratorContext) {
  const methodName = String(context.name);

  function replacementMethod(this: any, ...args: any[]) {
    console.log(`LOG: Entering method '${methodName}'.`)
    const result = originalMethod.call(this, ...args);
    console.log(`LOG: Exiting method '${methodName}'.`)
    return result;
  }

  return replacementMethod;
}

const person = new Person('张三');
person.greet()
// "LOG: Entering method 'greet'."
// "Hello, my name is 张三."
// "LOG: Exiting method 'greet'."
```

上面示例中，装饰器`@log`的返回值是一个函数`replacementMethod`，替代了原始方法`greet()`。在`replacementMethod()`内部，通过执行`originalMethod.call()`完成了对原始方法的调用。

利用方法装饰器，可以将类的方法变成延迟执行。

```
function delay(milliseconds: number = 0) {
  return function (value, context) {
    if (context.kind === "method") {
      return function (...args: any[]) {
        setTimeout(() => {
          value.apply(this, args);
        }, milliseconds);
      };
    }
  };
}

class Logger {
  @delay(1000)
  log(msg: string) {
    console.log(`${msg}`);
  }
}

let logger = new Logger();
logger.log("Hello World");
```

上面示例中，方法装饰器`@delay(1000)`将方法`log()`的执行推迟了1秒（1000毫秒）。这里真正的方法装饰器，是`delay()`执行后返回的函数，`delay()`的作用是接收参数，用来设置推迟执行的时间。这种通过高阶函数返回装饰器的做法，称为“工厂模式”，即可以像工厂那样生产出一个模子的装饰器。

方法装饰器的参数`context`对象里面，有一个`addInitializer()`方法。它是一个钩子方法，用来在类的初始化阶段，添加回调函数。这个回调函数就是作为`addInitializer()`的参数传入的，它会在构造方法执行期间执行，早于属性（field）的初始化。

下面是`addInitializer()`方法的一个例子。我们知道，类的方法往往需要在构造方法里面，进行`this`的绑定。

```
class Person {
  name: string;
  constructor(name: string) {
    this.name = name;

    // greet() 绑定 this
    this.greet = this.greet.bind(this);
  }

  greet() {
    console.log(`Hello, my name is ${this.name}.`);
  }
}
```

```
const g = new Person('张三').greet;
g() // "Hello, my name is 张三."
```

上面例子中，类`Person`的构造方法内部，将`this`与`greet()`方法进行了绑定。如果没有这一行，将`greet()`赋值给变量`g`进行调用，就会报错了。

`this`的绑定必须放在构造方法里面，因为这必须在类的初始化阶段完成。现在，它可以移到方法装饰器的`addInitializer()`里面。

```
function bound(
  originalMethod:any, context:ClassMethodDecoratorContext
) {
  const methodName = context.name;
  if (context.private) {
    throw new Error(`不能绑定私有方法 ${methodName as string}`);
  }
  context.addInitializer(function () {
    this[methodName] = this[methodName].bind(this);
  });
}
```

上面示例中，绑定`this`转移到了`addInitializer()`方法里面。

下面再看一个例子，通过`addInitializer()`将选定的方法名，放入一个集合。

```
function collect(
  value,
  {name, addInitializer}
) {
  addInitializer(function () {
    if (!this.collectedMethodKeys) {
      this.collectedMethodKeys = new Set();
    }
    this.collectedMethodKeys.add(name);
  });
}

class C {
  @collect
  toString() {}

  @collect
  [Symbol.iterator]() {}
}

const inst = new C();
inst.collectedMethodKeys // new Set(['toString', Symbol.iterator])
```

上面示例中，方法装饰器`@collect`会将所装饰的成员名字，加入一个 Set 集合`collectedMethodKeys`。

属性装饰器

属性装饰器用来装饰定义在类顶部的属性（field）。它的类型描述如下。

```
type ClassFieldDecorator = (
  value: undefined,
  context: {
    kind: 'field';
    name: string | symbol;
    static: boolean;
    private: boolean;
    access: { get: () => unknown, set: (value: unknown) => void };
    addInitializer(initializer: () => void): void;
  }
) => (initialValue: unknown) => unknown | void;
```

注意，装饰器的第一个参数`value`的类型是`undefined`，这意味着这个参数实际上没用的，装饰器不能从`value`获取所装饰属性的值。另外，第二个参数`context`对象的`kind`属性的值为字符串`field`，而不是“property”或“attribute”，这一点是需要注意的。

属性装饰器要么不返回值，要么返回一个函数，该函数会自动执行，用来对所装饰属性进行初始化。该函数的参数是所装饰属性的初始值，该函数的返回值是该属性的最终值。

```
function logged(value, context) {
  const { kind, name } = context;
  if (kind === 'field') {
    return function (initialValue) {
      console.log(`initializing ${name} with value ${initialValue}`);
      return initialValue;
    };
  }
}

class Color {
  @logged name = 'green';
}

const color = new Color();
// "initializing name with value green"
```

上面示例中，属性装饰器`@logged`装饰属性`name`。`@logged`的返回值是一个函数，该函数用来对属性`name`进行初始化，它的参数`initialValue`就是属性`name`的初始值`green`。新建实例对象`color`时，该函数会自动执行。

属性装饰器的返回值函数，可以用来更改属性的初始值。

```
function twice() {
  return initialValue => initialValue * 2;
}

class C {
```

```
  @twice
  field = 3;
}

const inst = new C();
inst.field // 6
```

上面示例中，属性装饰器`@twice`返回一个函数，该函数的返回值是属性`field`的初始值乘以2，所以属性`field`的最终值是6。

属性装饰器的上下文对象`context`的`access`属性，提供所装饰属性的存取器，请看下面的例子。

```
let acc;

function exposeAccess(
  value, {access}
) {
  acc = access;
}

class Color {
  @exposeAccess
  name = 'green'
}

const green = new Color();
green.name // 'green'

acc.get(green) // 'green'

acc.set(green, 'red');
green.name // 'red'
```

上面示例中，`access`包含了属性`name`的存取器，可以对该属性进行取值和赋值。

getter 装饰器，setter 装饰器

getter 装饰器和 setter 装饰器，是分别针对类的取值器（getter）和存值器（setter）的装饰器。它们的类型描述如下。

```
type ClassGetterDecorator = (
  value: Function,
  context: {
    kind: 'getter';
    name: string | symbol;
    static: boolean;
    private: boolean;
    access: { get: () => unknown };
    addInitializer(initializer: () => void): void;
  }
) => Function | void;
```

```
type ClassSetterDecorator = (
  value: Function,
  context: {
    kind: 'setter';
    name: string | symbol;
    static: boolean;
    private: boolean;
    access: { set: (value: unknown) => void };
    addInitializer(initializer: () => void): void;
  }
) => Function | void;
```

注意，getter 装饰器的上下文对象`context`的`access`属性，只包含`get()`方法；setter 装饰器的`access`属性，只包含`set()`方法。

这两个装饰器要么不返回值，要么返回一个函数，取代原来的取值器或存值器。

下面的例子是将取值器的结果，保存为一个属性，加快后面的读取。

```
class C {
  @lazy
  get value() {
    console.log('正在计算……');
    return '开销大的计算结果';
  }
}

function lazy(
  value:any,
  {kind, name}:any
) {
  if (kind === 'getter') {
    return function (this:any) {
      const result = value.call(this);
      Object.defineProperty(
        this, name,
        {
          value: result,
          writable: false,
        }
      );
      return result;
    };
  }
  return;
}

const inst = new C();
inst.value
// 正在计算……
// '开销大的计算结果'
inst.value
// '开销大的计算结果'
```

上面示例中，第一次读取`inst.value`，会进行计算，然后装饰器`@lazy`将结果存入只读属性`value`，后面再读取这个属性，就不会进行计算了。

accessor 装饰器

装饰器语法引入了一个新的属性修饰符`accessor`。

```
class C {
  accessor x = 1;
}
```

上面示例中，`accessor`修饰符等同于为公开属性`x`自动生成取值器和存值器，它们作用于私有属性`x`。（注意，公开的`x`与私有的`x`不是同一个属性。）也就是说，上面的代码等同于下面的代码。

```
class C {
  #x = 1;

  get x() {
    return this.#x;
  }

  set x(val) {
    this.#x = val;
  }
}
```

`accessor`也可以与静态属性和私有属性一起使用。

```
class C {
  static accessor x = 1;
  accessor #y = 2;
}
```

accessor 装饰器的类型如下。

```
type ClassAutoAccessorDecorator = (
  value: {
    get: () => unknown;
    set: (value: unknown) => void;
  },
  context: {
    kind: "accessor";
    name: string | symbol;
    access: { get(): unknown, set(value: unknown): void };
    static: boolean;
    private: boolean;
    addInitializer(initializer: () => void): void;
  }
) => {
  get?: () => unknown;
```

```
  set?: (value: unknown) => void;
  init?: (initialValue: unknown) => unknown;
} | void;
```

accessor 装饰器的`value`参数，是一个包含`get()`方法和`set()`方法的对象。该装饰器可以不返回值，或者返回一个新的对象，用来取代原来的`get()`方法和`set()`方法。此外，装饰器返回的对象还可以包括一个`init()`方法，用来改变私有属性的初始值。

下面是一个例子。

```
class C {
  @logged accessor x = 1;
}

function logged(value, { kind, name }) {
  if (kind === "accessor") {
    let { get, set } = value;

    return {
      get() {
        console.log(`getting ${name}`);

        return get.call(this);
      },

      set(val) {
        console.log(`setting ${name} to ${val}`);

        return set.call(this, val);
      },

      init(initialValue) {
        console.log(`initializing ${name} with value ${initialValue}`);
        return initialValue;
      }
    };
  }
}

let c = new C();

c.x;
// getting x

c.x = 123;
// setting x to 123
```

上面示例中，装饰器`@logged`为属性`x`的存值器和取值器，加上了日志输出。

装饰器的执行顺序

装饰器的执行分为两个阶段。

（1）评估（evaluation）：计算`@`符号后面的表达式的值，得到的应该是函数。

（2）应用（application）：将评估装饰器后得到的函数，应用于所装饰对象。

也就是说，装饰器的执行顺序是，先评估所有装饰器表达式的值，再将其应用于当前类。

应用装饰器时，顺序依次为方法装饰器和属性装饰器，然后是类装饰器。

请看下面的例子。

```
function d(str:string) {
  console.log(`评估 @d(): ${str}`);
  return (
    value:any, context:any
  ) => console.log(`应用 @d(): ${str}`);
}

function log(str:string) {
  console.log(str);
  return str;
}

@d('类装饰器')
class T {
  @d('静态属性装饰器')
  static staticField = log('静态属性值');

  @d('原型方法')
  [log('计算方法名')]() {}

  @d('实例属性')
  instanceField = log('实例属性值');

  @d('静态方法装饰器')
  static fn(){}
}
```

上面示例中，类`T`有四种装饰器：类装饰器、静态属性装饰器、方法装饰器、属性装饰器。

它的运行结果如下。

```
评估 @d(): 类装饰器
评估 @d(): 静态属性装饰器
评估 @d(): 原型方法
计算方法名
评估 @d(): 实例属性
评估 @d(): 静态方法装饰器
应用 @d(): 静态方法装饰器
应用 @d(): 原型方法
应用 @d(): 静态属性装饰器
应用 @d(): 实例属性
应用 @d(): 类装饰器
静态属性值
```

可以看到，类载入的时候，代码按照以下顺序执行。

（1）装饰器评估：这一步计算装饰器的值，首先是类装饰器，然后是类内部的装饰器，按照它们出现的顺序。

注意，如果属性名或方法名是计算值（本例是“计算方法名”），则它们在对应的装饰器评估之后，也会进行自身的评估。

（2）装饰器应用：实际执行装饰器函数，将它们与对应的方法和属性进行结合。

静态方法装饰器首先应用，然后是原型方法的装饰器和静态属性装饰器，接下来是实例属性装饰器，最后是类装饰器。

注意，“实例属性值”在类初始化的阶段并不执行，直到类实例化时才会执行。

如果一个方法或属性有多个装饰器，则内层的装饰器先执行，外层的装饰器后执行。

```
class Person {
  name: string;
  constructor(name: string) {
    this.name = name;
  }

  @bound
  @log
  greet() {
    console.log(`Hello, my name is ${this.name}.`);
  }
}
```

上面示例中，`greet()`有两个装饰器，内层的`@log`先执行，外层的`@bound`针对得到的结果再执行。

参考链接

- JavaScript metaprogramming with the 2022-03 decorators API
- TS 5.0 Beta: New Decorators Are Here!, Bytefer

装饰器（旧语法）

上一章介绍了装饰器的标准语法，那是在2022年通过成为标准的。但是在此之前，TypeScript 早在2014年就支持装饰器，不过使用的是旧语法。

装饰器的旧语法与标准语法，有相当大的差异。旧语法以后会被淘汰，但是目前大量现有项目依然在使用它，本章就介绍旧语法下的装饰器。

experimentalDecorators 编译选项

使用装饰器的旧语法，需要打开`--experimentalDecorators`编译选项。

```
$ tsc --target ES5 --experimentalDecorators
```

此外，还有另外一个编译选项`--emitDecoratorMetadata`，用来产生一些装饰器的元数据，供其他工具或某些模块（比如 reflect-metadata ）使用。

这两个编译选项可以在命令行设置，也可以在`tsconfig.json`文件里面进行设置。

```
{
  "compilerOptions": {
    "target": "ES6",
    "experimentalDecorators": true,
    "emitDecoratorMetadata": true
  }
}
```

装饰器的种类

按照所装饰的不同对象，装饰器可以分成五类。

- 类装饰器（Class Decorators）：用于类。
- 属性装饰器（Property Decorators）：用于属性。
- 方法装饰器（Method Decorators）：用于方法。
- 存取器装饰器（Accessor Decorators）：用于类的 set 或 get 方法。
- 参数装饰器（Parameter Decorators）：用于方法的参数。

下面是这五种装饰器一起使用的一个示例。

```
@ClassDecorator() //  (A)
class A {

  @PropertyDecorator() //  (B)
  name: string;
```

```
  @MethodDecorator() // (C)
  fly(
    @ParameterDecorator() //  (D)
    meters: number
  ) {
    // code
  }

  @AccessorDecorator() //  (E)
  get egg() {
    // code
  }
  set egg(e) {
    // code
  }
}
```

上面示例中，A 是类装饰器，B 是属性装饰器，C 是方法装饰器，D 是参数装饰器，E 是存取器装饰器。

注意，构造方法没有方法装饰器，只有参数装饰器。类装饰器其实就是在装饰构造方法。

另外，装饰器只能用于类，要么应用于类的整体，要么应用于类的内部成员，不能用于独立的函数。

```
function Decorator() {
  console.log('In Decorator');
}

@Decorator // 报错
function decorated() {
  console.log('in decorated');
}
```

上面示例中，装饰器用于一个普通函数，这是无效的，结果报错。

类装饰器

类装饰器应用于类（class），但实际上是应用于类的构造方法。

类装饰器有唯一参数，就是构造方法，可以在装饰器内部，对构造方法进行各种改造。如果类装饰器有返回值，就会替换掉原来的构造方法。

类装饰器的类型定义如下。

```
type ClassDecorator = <TFunction extends Function>
  (target: TFunction) => TFunction | void;
```

上面定义中，类型参数`TFunction`必须是函数，实际上就是构造方法。类装饰器的返回值，要么是返回处理后的原始构造方法，要么返回一个新的构造方法。

下面就是一个示例。

```
function f(target:any) {
  console.log('apply decorator')
  return target;
}

@f
class A {}
// 输出: apply decorator
```

上面示例中，使用了装饰器`@f`，因此类`A`的构造方法会自动传入`f`。

类`A`不需要新建实例，装饰器也会执行。装饰器会在代码加载阶段执行，而不是在运行时执行，而且只会执行一次。

由于 TypeScript 存在编译阶段，所以装饰器对类的行为的改变，实际上发生在编译阶段。这意味着，TypeScript 装饰器能在编译阶段运行代码，也就是说，它本质就是编译时执行的函数。

下面再看一个示例。

```
@sealed
class BugReport {
  type = "report";
  title: string;

  constructor(t:string) {
    this.title = t;
  }
}

function sealed(constructor: Function) {
  Object.seal(constructor);
  Object.seal(constructor.prototype);
}
```

上面示例中，装饰器`@sealed()`会锁定`BugReport`这个类，使得它无法新增或删除静态成员和实例成员。

如果除了构造方法，类装饰器还需要其他参数，可以采取“工厂模式”，即把装饰器写在一个函数里面，该函数可以接受其他参数，执行后返回装饰器。但是，这样就需要调用装饰器的时候，先执行一次工厂函数。

```
function factory(info:string) {
  console.log('received: ', info);
  return function (target:any) {
    console.log('apply decorator');
    return target;
  }
}

@factory('log something')
class A {}
```

上面示例中，函数`factory()`的返回值才是装饰器，所以加载装饰器的时候，要先执行一次`@factory('log something')`，才能得到装饰器。这样做的好处是，可以加入额外的参数，本例是参数`info`。

总之，`@`后面要么是一个函数名，要么是函数表达式，甚至可以写出下面这样的代码。

```
@((constructor: Function) => {
  console.log('log something');
})
class InlineDecoratorExample {
  // ...
}
```

上面示例中，`@`后面是一个箭头函数，这也是合法的。

类装饰器可以没有返回值，如果有返回值，就会替代所装饰的类的构造函数。由于 JavaScript 的类等同于构造函数的语法糖，所以装饰器通常返回一个新的类，对原有的类进行修改或扩展。

```
function decorator(target:any) {
  return class extends target {
    value = 123;
  };
}

@decorator
class Foo {
  value = 456;
}

const foo = new Foo();
console.log(foo.value); // 123
```

上面示例中，装饰器`decorator`返回一个新的类，替代了原来的类。

上例的装饰器参数`target`类型是`any`，可以改成构造方法，这样就更准确了。

```
type Constructor = {
  new(...args: any[]): {}
};

function decorator<T extends Constructor> (
  target: T
) {
  return class extends target {
    value = 123;
  };
}
```

这时，装饰器的行为就是下面这样。

```
@decorator
class A {}
```

```
// 等同于
class A {}
A = decorator(A) || A;
```

上面代码中，装饰器要么返回一个新的类`A`，要么不返回任何值，`A`保持装饰器处理后的状态。

方法装饰器

方法装饰器用来装饰类的方法，它的类型定义如下。

```
type MethodDecorator = <T>(
  target: Object,
  propertyKey: string|symbol,
  descriptor: TypedPropertyDescriptor<T>
) => TypedPropertyDescriptor<T> | void;
```

方法装饰器一共可以接受三个参数。

- target：（对于类的静态方法）类的构造函数，或者（对于类的实例方法）类的原型。
- propertyKey：所装饰方法的方法名，类型为`string|symbol`。
- descriptor：所装饰方法的描述对象。

方法装饰器的返回值（如果有的话），就是修改后的该方法的描述对象，可以覆盖原始方法的描述对象。

下面是一个示例。

```
function enumerable(value: boolean) {
  return function (
    target: any,
    propertyKey: string,
    descriptor: PropertyDescriptor
  ) {
    descriptor.enumerable = value;
  };
}

class Greeter {
  greeting: string;

  constructor(message:string) {
    this.greeting = message;
  }

  @enumerable(false)
  greet() {
    return 'Hello, ' + this.greeting;
  }
}
```

上面示例中，方法装饰器`@enumerable()`装饰 Greeter 类的`greet()`方法，作用是修改该方法的描述对象的可遍历性属性`enumerable`。`@enumerable(false)`表示将该方法修改成不可遍历。

下面再看一个例子。

```
function logger(
  target: any,
  propertyKey: string,
  descriptor: PropertyDescriptor
) {
  const original = descriptor.value;

  descriptor.value = function (...args) {
    console.log('params: ', ...args);
    const result = original.call(this, ...args);
    console.log('result: ', result);
    return result;
  }
}

class C {
  @logger
  add(x: number, y:number ) {
    return x + y;
  }
}

(new C()).add(1, 2)
// params:  1 2
// result:  3
```

上面示例中，方法装饰器`@logger`用来装饰`add()`方法，它的作用是让该方法输出日志。每当`add()`调用一次，控制台就会打印出参数和运行结果。

属性装饰器

属性装饰器用来装饰属性，类型定义如下。

```
type PropertyDecorator =
  (
    target: Object,
    propertyKey: string|symbol
  ) => void;
```

属性装饰器函数接受两个参数。

- target：（对于实例属性）类的原型对象（prototype），或者（对于静态属性）类的构造函数。
- propertyKey：所装饰属性的属性名，注意类型有可能是字符串，也有可能是 Symbol 值。

属性装饰器不需要返回值，如果有的话，也会被忽略。

下面是一个示例。

```
function ValidRange(min:number, max:number) {
  return (target:Object, key:string) => {
    Object.defineProperty(target, key, {
      set: function(v:number) {
        if (v < min || v > max) {
          throw new Error(`Not allowed value ${v}`);
        }
      }
    });
  }
}

// 输出 Installing ValidRange on year
class Student {
  @ValidRange(1920, 2020)
  year!: number;
}

const stud = new Student();

// 报错 Not allowed value 2022
stud.year = 2022;
```

上面示例中，装饰器`ValidRange`对属性`year`设立了一个上下限检查器，只要该属性赋值时，超过了上下限，就会报错。

注意，属性装饰器的第一个参数，对于实例属性是类的原型对象，而不是实例对象（即不是`this`对象）。这是因为装饰器执行时，类还没有新建实例，所以实例对象不存在。

由于拿不到`this`，所以属性装饰器无法获得实例属性的值。这也是它没有在参数里面提供属性描述对象的原因。

```
function logProperty(target: Object, member: string) {
  const prop = Object.getOwnPropertyDescriptor(target, member);
  console.log(`Property ${member} ${prop}`);
}

class PropertyExample {
  @logProperty
  name:string = 'Foo';
}
// 输出 Property name undefined
```

上面示例中，属性装饰器`@logProperty`内部想要获取实例属性`name`的属性描述对象，结果拿到的是`undefined`。因为上例的`target`是类的原型对象，不是实例对象，所以拿不到`name`属性，也就是说`target.name`是不存在的，所以拿到的是`undefined`。只有通过`this.name`才能拿到`name`属性，但是这时`this`还不存在。

属性装饰器不仅无法获得实例属性的值，也不能初始化或修改实例属性，而且它的返回值也会被忽略。因此，它的作用很有限。

不过，如果属性装饰器设置了当前属性的存取器（getter/setter），然后在构造函数里面就可以对实例属性进行读写。

```
function Min(limit:number) {
  return function(
    target: Object,
    propertyKey: string
  ) {
    let value: string;

    const getter = function() {
      return value;
    };

    const setter = function(newVal:string) {
      if(newVal.length < limit) {
        throw new Error(`Your password should be bigger than ${limit}`);
      }
      else {
        value = newVal;
      }
    };
    Object.defineProperty(target, propertyKey, {
      get: getter,
      set: setter
    });
  }
}

class User {
  username: string;

  @Min(8)
  password: string;

  constructor(username: string, password: string){
    this.username = username;
    this.password = password;
  }
}

const u = new User('Foo', 'pass');
// 报错 Your password should be bigger than 8
```

上面示例中，属性装饰器`@Min`通过设置存取器，拿到了实例属性的值。

存取器装饰器

存取器装饰器用来装饰类的存取器（accessor）。所谓“存取器”指的是某个属性的取值器（getter）和存值器（setter）。

存取器装饰器的类型定义，与方法装饰器一致。

```
type AccessorDecorator = <T>(
  target: Object,
  propertyKey: string|symbol,
  descriptor: TypedPropertyDescriptor<T>
) => TypedPropertyDescriptor<T> | void;
```

存取器装饰器有三个参数。

- target：（对于静态属性的存取器）类的构造函数，或者（对于实例属性的存取器）类的原型。
- propertyKey：存取器的属性名。
- descriptor：存取器的属性描述对象。

存取器装饰器的返回值（如果有的话），会作为该属性新的描述对象。

下面是一个示例。

```
function configurable(value: boolean) {
  return function (
    target: any,
    propertyKey: string,
    descriptor: PropertyDescriptor
  ) {
    descriptor.configurable = value;
  };
}

class Point {
  private _x: number;
  private _y: number;
  constructor(x:number, y:number) {
    this._x = x;
    this._y = y;
  }

  @configurable(false)
  get x() {
    return this._x;
  }

  @configurable(false)
  get y() {
    return this._y;
  }
}
```

上面示例中，装饰器`@configurable(false)`关闭了所装饰属性（`x`和`y`）的属性描述对象的`configurable`键（即关闭了属性的可配置性）。

下面的示例是将装饰器用来验证属性值，如果赋值不满足条件就报错。

```
function validator(
  target: Object,
  propertyKey: string,
  descriptor: PropertyDescriptor
){
  const originalGet = descriptor.get;
  const originalSet = descriptor.set;

  if (originalSet) {
    descriptor.set = function (val) {
      if (val > 100) {
        throw new Error(`Invalid value for ${propertyKey}`);
      }
      originalSet.call(this, val);
    };
  }
}

class C {
  #foo!: number;

  @validator
  set foo(v) {
    this.#foo = v;
  }

  get foo() {
    return this.#foo;
  }
}

const c = new C();
c.foo = 150;
// 报错
```

上面示例中，装饰器用自己定义的存值器，取代了原来的存值器，加入了验证条件。

TypeScript 不允许对同一个属性的存取器（getter 和 setter）使用同一个装饰器，也就是说只能装饰两个存取器里面的一个，且必须是排在前面的那一个，否则报错。

```
// 报错
class Person {
  #name:string;

  @Decorator
  set name(n:string) {
    this.#name = n;
  }
```

```
  @Decorator // 报错
  get name() {
    return this.#name;
  }
}
```

上面示例中，`@Decorator`同时装饰`name`属性的存值器和取值器，所以报错。

但是，下面的写法不会报错。

```
class Person {
  #name:string;

  @Decorator
  set name(n:string) {
    this.#name = n;
  }
  get name() {
    return this.#name;
  }
}
```

上面示例中，`@Decorator`只装饰它后面第一个出现的存值器（`set name()`），并不装饰取值器（`get name()`），所以不报错。

装饰器之所以不能同时用于同一个属性的存值器和取值器，原因是装饰器可以从属性描述对象上面，同时拿到取值器和存值器，因此只调用一次就够了。

参数装饰器

参数装饰器用来装饰构造方法或者其他方法的参数。它的类型定义如下。

```
type ParameterDecorator = (
  target: Object,
  propertyKey: string|symbol,
  parameterIndex: number
) => void;
```

参数装饰器接受三个参数。

- target：（对于静态方法）类的构造函数，或者（对于类的实例方法）类的原型对象。
- propertyKey：所装饰的方法的名字，类型为`string|symbol`。
- parameterIndex：当前参数在方法的参数序列的位置（从0开始）。

该装饰器不需要返回值，如果有的话会被忽略。

下面是一个示例。

```
function log(
  target: Object,
  propertyKey: string|symbol,
  parameterIndex: number
) {
  console.log(`${String(propertyKey)} NO.${parameterIndex} Parameter`);
}

class C {
  member(
    @log x:number,
    @log y:number
  ) {
    console.log(`member Parameters: ${x} ${y}`);
  }
}

const c = new C();
c.member(5, 5);
// member NO.1 Parameter
// member NO.0 Parameter
// member Parameters: 5 5
```

上面示例中，参数装饰器会输出参数的位置序号。注意，后面的参数会先输出。

跟其他装饰器不同，参数装饰器主要用于输出信息，没有办法修改类的行为。

装饰器的执行顺序

前面说过，装饰器只会执行一次，就是在代码解析时执行，哪怕根本没有调用类新建实例，也会执行，而且从此就不再执行了。

执行装饰器时，按照如下顺序执行。

1. 实例相关的装饰器。
2. 静态相关的装饰器。
3. 构造方法的参数装饰器。
4. 类装饰器。

请看下面的示例。

```
function f(key:string):any {
  return function () {
    console.log('执行: ', key);
  };
}

@f('类装饰器')
class C {
  @f('静态方法')
  static method() {}
```

```
  @f('实例方法')
  method() {}

  constructor(@f('构造方法参数') foo:any) {}
}
```

加载上面的示例，输出如下。

```
执行:  实例方法
执行:  静态方法
执行:  构造方法参数
执行:  类装饰器
```

同一级装饰器的执行顺序，是按照它们的代码顺序。但是，参数装饰器的执行总是早于方法装饰器。

```
function f(key:string):any {
  return function () {
    console.log('执行: ', key);
  };
}

class C {
  @f('方法1')
  m1(@f('参数1') foo:any) {}

  @f('属性1')
  p1: number;

  @f('方法2')
  m2(@f('参数2') foo:any) {}

  @f('属性2')
  p2: number;
}
```

加载上面的示例，输出如下。

```
执行:  参数1
执行:  方法1
执行:  属性1
执行:  参数2
执行:  方法2
执行:  属性2
```

上面示例中，实例装饰器的执行顺序，完全是按照代码顺序的。但是，同一个方法的参数装饰器，总是早于该方法的方法装饰器执行。

如果同一个方法或属性有多个装饰器，那么装饰器将顺序加载、逆序执行。

```
function f(key:string):any {
  console.log('加载: ', key);
```

```
  return function () {
    console.log('执行: ', key);
  };
}

class C {
  @f('A')
  @f('B')
  @f('C')
  m1() {}
}
// 加载:  A
// 加载:  B
// 加载:  C
// 执行:  C
// 执行:  B
// 执行:  A
```

如果同一个方法有多个参数，那么参数也是顺序加载、逆序执行。

```
function f(key:string):any {
  console.log('加载: ', key);
  return function () {
    console.log('执行: ', key);
  };
}

class C {
  method(
    @f('A') a:any,
    @f('B') b:any,
    @f('C') c:any,
  ) {}
}
// 加载:  A
// 加载:  B
// 加载:  C
// 执行:  C
// 执行:  B
// 执行:  A
```

为什么装饰器不能用于函数?

装饰器只能用于类和类的方法，不能用于函数，主要原因是存在函数提升。

JavaScript 的函数不管在代码的什么位置，都会提升到代码顶部。

```
addOne(1);
function addOne(n:number) {
  return n + 1;
}
```

上面示例中，函数`addOne()`不会因为在定义之前执行而报错，原因就是函数存在提升，会自动提升到代码顶部。

如果允许装饰器可以用于普通函数，那么就有可能导致意想不到的情况。

```
let counter = 0;

let add = function (target:any) {
  counter++;
};

@add
function foo() {
  //...
}
```

上面示例中，本来的意图是装饰器`@add`每使用一次，变量`counter`就加`1`，但是实际上会报错，因为函数提升的存在，使得实际执行的代码是下面这样。

```
@add // 报错
function foo() {
  //...
}

let counter = 0;
let add = function (target:any) {
  counter++;
};
```

上面示例中，`@add`还没有定义就调用了，从而报错。

总之，由于存在函数提升，使得装饰器不能用于函数。类是不会提升的，所以就没有这方面的问题。

另一方面，如果一定要装饰函数，可以采用高阶函数的形式直接执行，没必要写成装饰器。

```
function doSomething(name) {
  console.log('Hello, ' + name);
}

function loggingDecorator(wrapped) {
  return function() {
    console.log('Starting');
    const result = wrapped.apply(this, arguments);
    console.log('Finished');
    return result;
  }
}

const wrapped = loggingDecorator(doSomething);
```

上面示例中，`loggingDecorator()`是一个装饰器，只要把原始函数传入它执行，就能起到装饰器的效果。

多个装饰器的合成

多个装饰器可以应用于同一个目标对象，可以写在一行。

```
@f @g x
```

上面示例中，装饰器`@f`和`@g`同时装饰目标对象`x`。

多个装饰器也可以写成多行。

```
@f
@g
x
```

多个装饰器的效果，类似于函数的合成，按照从里到外的顺序执行。对于上例来说，就是执行`f(g(x))`。

前面也说过，如果`f`和`g`是表达式，那么需要先从外到里求值。

参考链接

- A Complete Guide to TypeScript Decorators, by Saul Mirone
- Deep introduction to using and implementing TypeScript decorators, by David Herron
- Deep introduction to property decorators in TypeScript, by David Herron
- Deep introduction to accessor decorators in TypeScript, by David Herron
- Using Property Decorators in Typescript with a real example, by Dany Paredes

declare 关键字

简介

declare 关键字用来告诉编译器，某个类型是存在的，可以在当前文件中使用。

它的主要作用，就是让当前文件可以使用其他文件声明的类型。举例来说，自己的脚本使用外部库定义的函数，编译器会因为不知道外部函数的类型定义而报错，这时就可以在自己的脚本里面使用`declare`关键字，告诉编译器外部函数的类型。这样的话，编译单个脚本就不会因为使用了外部类型而报错。

declare 关键字可以描述以下类型。

- 变量（const、let、var 命令声明）
- type 或者 interface 命令声明的类型
- class
- enum
- 函数（function）
- 模块（module）
- 命名空间（namespace）

declare 关键字的重要特点是，它只是通知编译器某个类型是存在的，不用给出具体实现。比如，只描述函数的类型，不给出函数的实现，如果不使用`declare`，这是做不到的。

declare 只能用来描述已经存在的变量和数据结构，不能用来声明新的变量和数据结构。另外，所有 declare 语句都不会出现在编译后的文件里面。

declare variable

declare 关键字可以给出外部变量的类型描述。

举例来说，当前脚本使用了其他脚本定义的全局变量`x`。

```
x = 123; // 报错
```

上面示例中，变量`x`是其他脚本定义的，当前脚本不知道它的类型，编译器就会报错。

这时使用 declare 命令给出它的类型，就不会报错了。

```
declare let x:number;
x = 1;
```

如果 declare 关键字没有给出变量的具体类型，那么变量类型就是`any`。

```
declare let x;
x = 1;
```

上面示例中，变量`x`的类型为`any`。

下面的例子是脚本使用浏览器全局对象`document`。

```
declare var document;
document.title = 'Hello';
```

上面示例中，declare 告诉编译器，变量`document`的类型是外部定义的（具体定义在 TypeScript 内置文件`lib.d.ts`）。

如果 TypeScript 没有找到`document`的外部定义，这里就会假定它的类型是`any`。

注意，declare 关键字只用来给出类型描述，是纯的类型代码，不允许设置变量的初始值，即不能涉及值。

```
// 报错
declare let x:number = 1;
```

上面示例中，declare 设置了变量的初始值，结果就报错了。

declare function

declare 关键字可以给出外部函数的类型描述。

下面是一个例子。

```
declare function sayHello(
  name:string
):void;

sayHello('张三');
```

上面示例中，declare 命令给出了`sayHello()`的类型描述，表示这个函数是由外部文件定义的，因此这里可以直接使用该函数。

注意，这种单独的函数类型声明语句，只能用于`declare`命令后面。一方面，TypeScript 不支持单独的函数类型声明语句；另一方面，declare 关键字后面也不能带有函数的具体实现。

```
// 报错
function sayHello(
  name:string
):void;

let foo = 'bar';

function sayHello(name:string) {
```

```
  return '你好，' + name;
}
```

上面示例中，单独写函数的类型声明就会报错。

declare class

declare 给出 class 类型描述的写法如下。

```
declare class Animal {
  constructor(name:string);
  eat():void;
  sleep():void;
}
```

下面是一个复杂一点的例子。

```
declare class C {
  // 静态成员
  public static s0():string;
  private static s1:string;

  // 属性
  public a:number;
  private b:number;

  // 构造函数
  constructor(arg:number);

  // 方法
  m(x:number, y:number):number;

  // 存取器
  get c():number;
  set c(value:number);

  // 索引签名
  [index:string]:any;
}
```

同样的，declare 后面不能给出 Class 的具体实现或初始值。

declare module，declare namespace

如果想把变量、函数、类组织在一起，可以将 declare 与 module 或 namespace 一起使用。

```
declare namespace AnimalLib {
  class Animal {
    constructor(name:string);
    eat():void;
```

```
    sleep():void;
  }

  type Animals = 'Fish' | 'Dog';
}

// 或者
declare module AnimalLib {
  class Animal {
    constructor(name:string);
    eat(): void;
    sleep(): void;
  }

  type Animals = 'Fish' | 'Dog';
}
```

上面示例中，declare 关键字给出了 module 或 namespace 的类型描述。

declare module 和 declare namespace 里面，加不加 export 关键字都可以。

```
declare namespace Foo {
  export var a: boolean;
}

declare module 'io' {
  export function readFile(filename:string):string;
}
```

上面示例中，namespace 和 module 里面使用了 export 关键字。

下面的例子是当前脚本使用了`myLib`这个外部库，它有方法`makeGreeting()`和属性`numberOfGreetings`。

```
let result = myLib.makeGreeting('你好');
console.log('欢迎词: ' + result);

let count = myLib.numberOfGreetings;
```

`myLib`的类型描述就可以这样写。

```
declare namespace myLib {
  function makeGreeting(s:string): string;
  let numberOfGreetings: number;
}
```

declare 关键字的另一个用途，是为外部模块添加属性和方法时，给出新增部分的类型描述。

```
import { Foo as Bar } from 'moduleA';

declare module 'moduleA' {
  interface Foo {
    custom: {
```

```
      prop1: string;
    }
  }
}
```

上面示例中，从模块`moduleA`导入了类型`Foo`，它是一个接口（interface），并将其重命名为`Bar`，然后用 declare 关键字为`Foo`增加一个属性`custom`。这里需要注意的是，虽然接口`Foo`改名为`Bar`，但是扩充类型时，还是扩充原始的接口`Foo`，因为同名 interface 会自动合并类型声明。

下面是另一个例子。一个项目有多个模块，可以在一个模块中，对另一个模块的接口进行类型扩展。

```
// a.ts
export interface A {
  x: number;
}

// b.ts
import { A } from './a';

declare module './a' {
  interface A {
    y: number;
  }
}

const a:A = { x: 0, y: 0 };
```

上面示例中，脚本`a.ts`定义了一个接口`A`，脚本`b.ts`为这个接口添加了属性`y`。`declare module './a' {}`表示对`a.ts`里面的模块，进行类型声明，而同名 interface 会自动合并，所以等同于扩展类型。

使用这种语法进行模块的类型扩展时，有三点需要注意：

（1）`declare module NAME`语法里面的模块名`NAME`，跟 import 和 export 的模块名规则是一样的，且必须跟当前文件加载该模块的语句写法（上例`import { A } from './a'`）保持一致。

（2）不能创建新的顶层类型。也就是说，只能对`a.ts`模块中已经存在的类型进行扩展，不允许增加新的顶层类型，比如新定义一个接口`B`。

（3）不能对默认的`default`接口进行扩展，只能对 export 命令输出的命名接口进行扩充。这是因为在进行类型扩展时，需要依赖输出的接口名。

某些第三方模块，原始作者没有提供接口类型，这时可以在自己的脚本顶部加上下面一行命令。

```
// 语法
declare module "模块名";

// 例子
declare module "hot-new-module";
```

加上上面的命令以后，外部模块即使没有类型声明，也可以通过编译。但是，从该模块输入的所有接口都将为`any`类型。

declare module 描述的模块名可以使用通配符。

```
declare module 'my-plugin-*' {
  interface PluginOptions {
    enabled: boolean;
    priority: number;
  }

  function initialize(options: PluginOptions): void;
  export = initialize;
}
```

上面示例中，模块名`my-plugin-*`表示适配所有以`my-plugin-`开头的模块名（比如`my-plugin-logger`）。

declare global

如果要为 JavaScript 引擎的原生对象添加属性和方法，可以使用`declare global {}`语法。

```
export {};

declare global {
  interface String {
    toSmallString(): string;
  }
}

String.prototype.toSmallString = ():string => {
  // 具体实现
  return '';
};
```

上面示例中，为 JavaScript 原生的`String`对象添加了`toSmallString()`方法。declare global 给出这个新增方法的类型描述。

这个示例第一行的空导出语句`export {}`，作用是强制编译器将这个脚本当作模块处理。这是因为`declare global`必须用在模块里面。

下面的示例是为 window 对象（类型接口为`Window`）添加一个属性`myAppConfig`。

```
export {};

declare global {
  interface Window {
    myAppConfig:object;
  }
}
```

```
const config = window.myAppConfig;
```

declare global 只能扩充现有对象的类型描述，不能增加新的顶层类型。

declare enum

declare 关键字给出 enum 类型描述的例子如下，下面的写法都是允许的。

```
declare enum E1 {
  A,
  B,
}

declare enum E2 {
  A = 0,
  B = 1,
}

declare const enum E3 {
  A,
  B,
}

declare const enum E4 {
  A = 0,
  B = 1,
}
```

declare module 用于类型声明文件

我们可以为每个模块脚本，定义一个`.d.ts`文件，把该脚本用到的类型定义都放在这个文件里面。但是，更方便的做法是为整个项目，定义一个大的`.d.ts`文件，在这个文件里面使用`declare module`定义每个模块脚本的类型。

下面的示例是`node.d.ts`文件的一部分。

```
declare module "url" {
  export interface Url {
    protocol?: string;
    hostname?: string;
    pathname?: string;
  }

  export function parse(
    urlStr: string,
    parseQueryString?,
    slashesDenoteHost?
  ): Url;
}
```

```
declare module "path" {
  export function normalize(p: string): string;
  export function join(...paths: any[]): string;
  export var sep: string;
}
```

上面示例中，`url`和`path`都是单独的模块脚本，但是它们的类型都定义在`node.d.ts`这个文件里面。

另一种情况是，使用`declare module`命令，为模块名指定加载路径。

```
declare module "lodash" {
  export * from "../../dependencies/lodash";
  export default from "../../dependencies/lodash";
}
```

上面示例中，`declare module "lodash"`为模块`lodash`，指定具体的加载路径。

使用时，自己的脚本使用三斜杠命令，加载这个类型声明文件。

```
/// <reference path="node.d.ts"/>
```

如果没有上面这一行命令，自己的脚本使用外部模块时，就需要在脚本里面使用 declare 命令单独给出外部模块的类型。

参考链接

- How Does The Declare Keyword Work In TypeScript?, Tim Mouskhelichvili

d.ts 类型声明文件

简介

单独使用的模块，一般会同时提供一个单独的类型声明文件（declaration file），把本模块的外部接口的所有类型都写在这个文件里面，便于模块使用者了解接口，也便于编译器检查使用者的用法是否正确。

类型声明文件里面只有类型代码，没有具体的代码实现。它的文件名一般为`[模块名].d.ts`的形式，其中的`d`表示 declaration（声明）。

举例来说，有一个模块的代码如下。

```
const maxInterval = 12;

function getArrayLength(arr) {
  return arr.length;
}

module.exports = {
  getArrayLength,
  maxInterval,
};
```

它的类型声明文件可以写成下面这样。

```
export function getArrayLength(arr: any[]): number;
export const maxInterval: 12;
```

类型声明文件也可以使用`export =`命令，输出对外接口。下面是 moment 模块的类型声明文件的例子。

```
declare module 'moment' {
  function moment(): any;
  export = moment;
}
```

上面示例中，模块`moment`内部有一个函数`moment()`，而`export =`表示`module.exports`输出的就是这个函数。

除了使用`export =`，模块输出在类型声明文件中，也可以使用`export default`表示。

```
// 模块输出
module.exports = 3.142;

// 类型输出文件
// 写法一
declare const pi: number;
export default pi;
```

```
// 写法二
declare const pi: number;
export= pi;
```

上面示例中，模块输出的是一个数字，那么可以用`export default`或`export =`表示输出这个值。

下面是一个如何使用类型声明文件的简单例子。有一个类型声明文件`types.d.ts`。

```
// types.d.ts
export interface Character {
  catchphrase?: string;
  name: string;
}
```

然后，就可以在 TypeScript 脚本里面导入该文件声明的类型。

```
// index.ts
import { Character } from "./types";

export const character:Character = {
  catchphrase: "Yee-haw!",
  name: "Sandy Cheeks",
};
```

类型声明文件也可以包括在项目的 tsconfig.json 文件里面，这样的话，编译器打包项目时，会自动将类型声明文件加入编译，而不必在每个脚本里面加载类型声明文件。比如，moment 模块的类型声明文件是`moment.d.ts`，使用 moment 模块的项目可以将其加入项目的 tsconfig.json 文件。

```
{
  "compilerOptions": {},
  "files": [
    "src/index.ts",
    "typings/moment.d.ts"
  ]
}
```

类型声明文件的来源

类型声明文件主要有以下三种来源。

- TypeScript 编译器自动生成。
- TypeScript 内置类型文件。
- 外部模块的类型声明文件，需要自己安装。

自动生成

只要使用编译选项`declaration`，编译器就会在编译时自动生成单独的类型声明文件。

下面是在`tsconfig.json`文件里面，打开这个选项。

```
{
  "compilerOptions": {
    "declaration": true
  }
}
```

你也可以在命令行打开这个选项。

```
$ tsc --declaration
```

内置声明文件

安装 TypeScript 语言时，会同时安装一些内置的类型声明文件，主要是内置的全局对象（JavaScript 语言接口和运行环境 API）的类型声明。

这些内置声明文件位于 TypeScript 语言安装目录的`lib`文件夹内，数量大概有几十个，下面是其中一些主要文件。

- lib.d.ts
- lib.dom.d.ts
- lib.es2015.d.ts
- lib.es2016.d.ts
- lib.es2017.d.ts
- lib.es2018.d.ts
- lib.es2019.d.ts
- lib.es2020.d.ts
- lib.es5.d.ts
- lib.es6.d.ts

这些内置声明文件的文件名统一为“lib.[description].d.ts”的形式，其中`description`部分描述了文件内容。比如，`lib.dom.d.ts`这个文件就描述了 DOM 结构的类型。

如果开发者想了解全局对象的类型接口（比如 ES6 全局对象的类型），那么就可以去查看这些内置声明文件。

TypeScript 编译器会自动根据编译目标`target`的值，加载对应的内置声明文件，所以不需要特别的配置。但是，可以使用编译选项`lib`，指定加载哪些内置声明文件。

```
{
  "compilerOptions": {
    "lib": ["dom", "es2021"]
  }
}
```

上面示例中，`lib`选项指定加载`dom`和`es2021`这两个内置类型声明文件。

编译选项`noLib`会禁止加载任何内置声明文件。

外部类型声明文件

如果项目中使用了外部的某个第三方代码库，那么就需要这个库的类型声明文件。

这时又分成三种情况。

（1）这个库自带了类型声明文件。

一般来说，如果这个库的源码包含了`[vendor].d.ts`文件，那么就自带了类型声明文件。其中的`vendor`表示这个库的名字，比如`moment`这个库就自带`moment.d.ts`。使用这个库可能需要单独加载它的类型声明文件。

（2）这个库没有自带，但是可以找到社区制作的类型声明文件。

第三方库如果没有提供类型声明文件，社区往往会提供。TypeScript 社区主要使用 DefinitelyTyped 仓库，各种类型声明文件都会提交到那里，已经包含了几千个第三方库。

这些声明文件都会作为一个单独的库，发布到 npm 的`@types`名称空间之下。比如，jQuery 的类型声明文件就发布成`@types/jquery`这个库，使用时安装这个库就可以了。

```
$ npm install @types/jquery --save-dev
```

执行上面的命令，`@types/jquery`这个库就安装到项目的`node_modules/@types/jquery`目录，里面的`index.d.ts`文件就是 jQuery 的类型声明文件。如果类型声明文件不是`index.d.ts`，那么就需要在`package.json`的`types`或`typings`字段，指定类型声明文件的文件名。

TypeScript 会自动加载`node_modules/@types`目录下的模块，但可以使用编译选项`typeRoots`改变这种行为。

```
{
  "compilerOptions": {
    "typeRoots": ["./typings", "./vendor/types"]
  }
}
```

上面示例表示，TypeScript 不再去`node_modules/@types`目录，而是去跟当前`tsconfig.json`同级的`typings`和`vendor/types`子目录，加载类型模块了。

默认情况下，TypeScript 会自动加载`typeRoots`目录里的所有模块，编译选项`types`可以指定加载哪些模块。

```
{
  "compilerOptions": {
    "types" : ["jquery"]
  }
}
```

上面设置中，`types`属性是一个数组，成员是所要加载的类型模块，要加载几个模块，这个数组就有几个成员，每个类型模块在`typeRoots`目录下都有一个自己的子目录。这样的话，TypeScript 就会自动去`jquery`子目录，加载 jQuery 的类型声明文件。

（3）找不到类型声明文件，需要自己写。

有时实在没有第三方库的类型声明文件，又很难完整给出该库的类型描述，这时你可以告诉 TypeScript 相关对象的类型是`any`。比如，使用 jQuery 的脚本可以写成下面这样。

```
declare var $:any

// 或者
declare type JQuery = any;
declare var $:JQuery;
```

上面代码表示，jQuery 的`$`对象是外部引入的，类型是`any`，也就是 TypeScript 不用对它进行类型检查。

也可以采用下面的写法，将整个外部模块的类型设为`any`。

```
declare module '模块名';
```

有了上面的命令，指定模块的所有接口都将视为`any`类型。

declare 关键字

类型声明文件只包含类型描述，不包含具体实现，所以非常适合使用 declare 语句来描述类型。declare 关键字的具体用法，详见《declare 关键字》一章，这里讲解如何在类型声明文件里面使用它。

类型声明文件里面，变量的类型描述必须使用`declare`命令，否则会报错，因为变量声明语句是值相关代码。

```
declare let foo:string;
```

interface 类型有没有`declare`都可以，因为 interface 是完全的类型代码。

```
interface Foo {} // 正确
declare interface Foo {} // 正确
```

类型声明文件里面，顶层可以使用`export`命令，也可以不用，除非使用者脚本会显式使用`import`命令输入类型。

```
export interface Data {
  version: string;
}
```

下面是类型声明文件的一些例子。先看 moment 模块的类型描述文件`moment.d.ts`。

```
declare module 'moment' {
  export interface Moment {
    format(format:string): string;

    add(
      amount: number,
      unit: 'days' | 'months' | 'years'
    ): Moment;

    subtract(
      amount:number,
      unit:'days' | 'months' | 'years'
    ): Moment;
  }

  function moment(
    input?: string | Date
  ): Moment;

  export default moment;
}
```

上面示例中，可以注意一下默认接口`moment()`的写法。

下面是 D3 库的类型声明文件`D3.d.ts`。

```
declare namespace D3 {
  export interface Selectors {
    select: {
      (selector: string): Selection;
      (element: EventTarget): Selection;
    };
  }

  export interface Event {
    x: number;
    y: number;
  }

  export interface Base extends Selectors {
    event: Event;
  }
}

declare var d3: D3.Base;
```

模块发布

当前模块如果包含自己的类型声明文件，可以在 package.json 文件里面添加一个`types`字段或`typings`字段，指明类型声明文件的位置。

```
{
  "name": "awesome",
  "author": "Vandelay Industries",
  "version": "1.0.0",
  "main": "./lib/main.js",
  "types": "./lib/main.d.ts"
}
```

上面示例中，`types`字段给出了类型声明文件的位置。

注意，如果类型声明文件名为`index.d.ts`，且在项目的根目录中，那就不需要在`package.json`里面注明了。

有时，类型声明文件会单独发布成一个 npm 模块，这时用户就必须同时加载该模块。

```
{
  "name": "browserify-typescript-extension",
  "author": "Vandelay Industries",
  "version": "1.0.0",
  "main": "./lib/main.js",
  "types": "./lib/main.d.ts",
  "dependencies": {
    "browserify": "latest",
    "@types/browserify": "latest",
    "typescript": "next"
  }
}
```

上面示例是一个模块的 package.json 文件，该模块需要 browserify 模块。由于后者的类型声明文件是一个单独的模块`@types/browserify`，所以还需要加载那个模块。

三斜杠命令

如果类型声明文件的内容非常多，可以拆分成多个文件，然后入口文件使用三斜杠命令，加载其他拆分后的文件。

举例来说，入口文件是`main.d.ts`，里面的接口定义在`interfaces.d.ts`，函数定义在`functions.d.ts`。那么，`main.d.ts`里面可以用三斜杠命令，加载后面两个文件。

```
/// <reference path="./interfaces.d.ts" />
/// <reference path="./functions.d.ts" />
```

三斜杠命令（`///`）是一个 TypeScript 编译器命令，用来指定编译器行为。它只能用在文件的头部，如果用在其他地方，会被当作普通的注释。另外，若一个文件中使用了三斜线命令，那么在三斜线命令之前只允许使用单行注释、多行注释和其他三斜线命令，否则三斜杠命令也会被当作普通的注释。

除了拆分类型声明文件，三斜杠命令也可以用于普通脚本加载类型声明文件。

三斜杠命令主要包含三个参数，代表三种不同的命令。

- path
- types
- lib

下面依次进行讲解。

/// <reference path="" />

`/// <reference path="" />`是最常见的三斜杠命令，告诉编译器在编译时需要包括的文件，常用来声明当前脚本依赖的类型文件。

```
/// <reference path="./lib.ts" />

let count = add(1, 2);
```

上面示例表示，当前脚本依赖于`./lib.ts`，里面是`add()`的定义。编译当前脚本时，还会同时编译`./lib.ts`。编译产物会有两个 JS 文件，一个当前脚本，另一个就是`./lib.js`。

下面的例子是当前脚本依赖于 Node.js 类型声明文件。

```
/// <reference path="node.d.ts"/>
import * as URL from "url";
let myUrl = URL.parse("https://www.typescriptlang.org");
```

编译器会在预处理阶段，找出所有三斜杠引用的文件，将其添加到编译列表中，然后一起编译。

`path`参数指定了所引入文件的路径。如果该路径是一个相对路径，则基于当前脚本的路径进行计算。

使用该命令时，有以下两个注意事项。

- `path`参数必须指向一个存在的文件，若文件不存在会报错。
- `path`参数不允许指向当前文件。

默认情况下，每个三斜杠命令引入的脚本，都会编译成单独的 JS 文件。如果希望编译后只产出一个合并文件，可以使用编译选项`outFile`。但是，`outFile`编译选项不支持合并 CommonJS 模块和 ES 模块，只有当编译参数`module`的值设为 None、System 或 AMD 时，才能编译成一个文件。

如果打开了编译参数`noResolve`，则忽略三斜杠指令。将其当作一般的注释，原样保留在编译产物中。

/// <reference types="" />

types 参数用来告诉编译器当前脚本依赖某个 DefinitelyTyped 类型库，通常安装在`node_modules/@types`目录。

types 参数的值是类型库的名称，也就是安装到`node_modules/@types`目录中的子目录的名字。

```
/// <reference types="node" />
```

上面示例中，这个三斜杠命令表示编译时添加 Node.js 的类型库，实际添加的脚本是`node_modules`目录里面的`@types/node/index.d.ts`。

可以看到，这个命令的作用类似于`import`命令。

注意，这个命令只在你自己手写类型声明文件（`.d.ts`文件）时，才有必要用到，也就是说，只应该用在`.d.ts`文件中，普通的`.ts`脚本文件不需要写这个命令。如果是普通的`.ts`脚本，可以使用`tsconfig.json`文件的`types`属性指定依赖的类型库。

`/// <reference lib="" />`

`/// <reference lib="..." />`命令允许脚本文件显式包含内置 lib 库，等同于在`tsconfig.json`文件里面使用`lib`属性指定 lib 库。

前文说过，安装 TypeScript 软件包时，会同时安装一些内置的类型声明文件，即内置的 lib 库。这些库文件位于 TypeScript 安装目录的`lib`文件夹中，它们描述了 JavaScript 语言和引擎的标准 API。

库文件并不是固定的，会随着 TypeScript 版本的升级而更新。库文件统一使用“lib.[description].d.ts”的命名方式，而`/// <reference lib="" />`里面的`lib`属性的值就是库文件名的`description`部分，比如`lib="es2015"`就表示加载库文件`lib.es2015.d.ts`。

```
/// <reference lib="es2017.string" />
```

上面示例中，`es2017.string`对应的库文件就是`lib.es2017.string.d.ts`。

TypeScript 类型运算符

TypeScript 提供强大的类型运算能力，可以使用各种类型运算符，对已有的类型进行计算，得到新类型。

keyof 运算符

简介

keyof 是一个单目运算符，接受一个对象类型作为参数，返回该对象的所有键名组成的联合类型。

```
type MyObj = {
  foo: number,
  bar: string,
};

type Keys = keyof MyObj; // 'foo'|'bar'
```

上面示例中，`keyof MyObj`返回`MyObj`的所有键名组成的联合类型，即`'foo'|'bar'`。

下面是另一个例子。

```
interface T {
  0: boolean;
  a: string;
  b(): void;
}

type KeyT = keyof T; // 0 | 'a' | 'b'
```

由于 JavaScript 对象的键名只有三种类型，所以对于任意对象的键名的联合类型就是`string|number|symbol`。

```
// string | number | symbol
type KeyT = keyof any;
```

对于没有自定义键名的类型使用 keyof 运算符，返回`never`类型，表示不可能有这样类型的键名。

```
type KeyT = keyof object;  // never
```

上面示例中，由于`object`类型没有自身的属性，也就没有键名，所以`keyof object`返回`never`类型。

由于 keyof 返回的类型是`string|number|symbol`，如果有些场合只需要其中的一种类型，那么可以采用交叉类型的写法。

```
type Capital<T extends string> = Capitalize<T>;

type MyKeys<Obj extends object> = Capital<keyof Obj>; // 报错
```

上面示例中，类型`Capital`只接受字符串作为类型参数，传入`keyof Obj`会报错，原因是这时的类型参数是`string|number|symbol`，跟字符串不兼容。采用下面的交叉类型写法，就不会报错。

```
type MyKeys<Obj extends object> = Capital<string & keyof Obj>;
```

上面示例中，`string & keyof Obj`等同于`string & string|number|symbol`进行交集运算，最后返回`string`，因此`Capital<T extends string>`就不会报错了。

如果对象属性名采用索引形式，keyof 会返回属性名的索引类型。

```
// 示例一
interface T {
  [prop: number]: number;
}

// number
type KeyT = keyof T;

// 示例二
interface T {
  [prop: string]: number;
}

// string|number
type KeyT = keyof T;
```

上面的示例二，`keyof T`返回的类型是`string|number`，原因是 JavaScript 属性名为字符串时，包含了属性名为数值的情况，因为数值属性名会自动转为字符串。

如果 keyof 运算符用于数组或元组类型，得到的结果可能出人意料。

```
type Result = keyof ['a', 'b', 'c'];
// 返回 number | "0" | "1" | "2"
// | "length" | "pop" | "push" | ···
```

上面示例中，keyof 会返回数组的所有键名，包括数字键名和继承的键名。

对于联合类型，keyof 返回成员共有的键名。

```
type A = { a: string; z: boolean };
type B = { b: string; z: boolean };

// 返回 'z'
type KeyT = keyof (A | B);
```

对于交叉类型，keyof 返回所有键名。

```
type A = { a: string; x: boolean };
type B = { b: string; y: number };

// 返回 'a' | 'x' | 'b' | 'y'
type KeyT = keyof (A & B);

// 相当于
keyof (A & B) ≡ keyof A | keyof B
```

keyof 取出的是键名组成的联合类型，如果想取出键值组成的联合类型，可以像下面这样写。

```
type MyObj = {
  foo: number,
  bar: string,
};

type Keys = keyof MyObj;

type Values = MyObj[Keys]; // number|string
```

上面示例中，`Keys`是键名组成的联合类型，而`MyObj[Keys]`会取出每个键名对应的键值类型，组成一个新的联合类型，即`number|string`。

keyof 运算符的用途

keyof 运算符往往用于精确表达对象的属性类型。

举例来说，取出对象的某个指定属性的值，JavaScript 版本可以写成下面这样。

```
function prop(obj, key) {
  return obj[key];
}
```

上面这个函数添加类型，只能写成下面这样。

```
function prop(
  obj: { [p:string]: any },
  key: string
):any {
  return obj[key];
}
```

上面的类型声明有两个问题，一是无法表示参数`key`与参数`obj`之间的关系，二是返回值类型只能写成`any`。

有了 keyof 以后，就可以解决这两个问题，精确表达返回值类型。

```
function prop<Obj, K extends keyof Obj>(
  obj:Obj, key:K
```

```
):Obj[K] {
  return obj[key];
}
```

上面示例中，`K extends keyof Obj`表示`K`是`Obj`的一个属性名，传入其他字符串会报错。返回值类型`Obj[K]`就表示`K`这个属性值的类型。

keyof 的另一个用途是用于属性映射，即将一个类型的所有属性逐一映射成其他值。

```
type NewProps<Obj> = {
  [Prop in keyof Obj]: boolean;
};

// 用法
type MyObj = { foo: number; };

// 等于 { foo: boolean; }
type NewObj = NewProps<MyObj>;
```

上面示例中，类型`NewProps`是类型`Obj`的映射类型，前者继承了后者的所有属性，但是把所有属性值类型都改成了`boolean`。

下面的例子是去掉 readonly 修饰符。

```
type Mutable<Obj> = {
  -readonly [Prop in keyof Obj]: Obj[Prop];
};

// 用法
type MyObj = {
  readonly foo: number;
}

// 等于 { foo: number; }
type NewObj = Mutable<MyObj>;
```

上面示例中，`[Prop in keyof Obj]`是`Obj`类型的所有属性名，`-readonly`表示去除这些属性的只读特性。对应地，还有`+readonly`的写法，表示添加只读属性设置。

下面的例子是让可选属性变成必有的属性。

```
type Concrete<Obj> = {
  [Prop in keyof Obj]-?: Obj[Prop];
};

// 用法
type MyObj = {
  foo?: number;
}
```

```
// 等于 { foo: number; }
type NewObj = Concrete<MyObj>;
```

上面示例中，`[Prop in keyof Obj]`后面的`-?`表示去除可选属性设置。对应地，还有`+?`的写法，表示添加可选属性设置。

in 运算符

JavaScript 语言中，`in`运算符用来确定对象是否包含某个属性名。

```
const obj = { a: 123 };

if ('a' in obj)
  console.log('found a');
```

上面示例中，`in`运算符用来判断对象`obj`是否包含属性`a`。

`in`运算符的左侧是一个字符串，表示属性名，右侧是一个对象。它的返回值是一个布尔值。

TypeScript 语言的类型运算中，`in`运算符有不同的用法，用来取出（遍历）联合类型的每一个成员类型。

```
type U = 'a'|'b'|'c';

type Foo = {
  [Prop in U]: number;
};
// 等同于
type Foo = {
  a: number,
  b: number,
  c: number
};
```

上面示例中，`[Prop in U]`表示依次取出联合类型`U`的每一个成员。

上一小节的例子也提到，`[Prop in keyof Obj]`表示取出对象`Obj`的每一个键名。

方括号运算符

方括号运算符（`[]`）用于取出对象的键值类型，比如`T[K]`会返回对象`T`的属性`K`的类型。

```
type Person = {
  age: number;
  name: string;
  alive: boolean;
};
```

```
// Age 的类型是 number
type Age = Person['age'];
```

上面示例中，`Person['age']`返回属性`age`的类型，本例是`number`。

方括号的参数如果是联合类型，那么返回的也是联合类型。

```
type Person = {
  age: number;
  name: string;
  alive: boolean;
};

// number|string
type T = Person['age'|'name'];

// number|string|boolean
type A = Person[keyof Person];
```

上面示例中，方括号里面是属性名的联合类型，所以返回的也是对应的属性值的联合类型。

如果访问不存在的属性，会报错。

```
type T = Person['notExisted']; // 报错
```

方括号运算符的参数也可以是属性名的索引类型。

```
type Obj = {
  [key:string]: number,
};

// number
type T = Obj[string];
```

上面示例中，`Obj`的属性名是字符串的索引类型，所以可以写成`Obj[string]`，代表所有字符串属性名，返回的就是它们的类型`number`。

这个语法对于数组也适用，可以使用`number`作为方括号的参数。

```
// MyArray 的类型是 { [key:number]: string }
const MyArray = ['a','b','c'];

// 等同于 (typeof MyArray)[number]
// 返回 string
type Person = typeof MyArray[number];
```

上面示例中，`MyArray`是一个数组，它的类型实际上是属性名的数值索引，而`typeof MyArray[number]`的`typeof`运算优先级高于方括号，所以返回的是所有数值键名的键值类型`string`。

注意，方括号里面不能有值的运算。

```
// 示例一
const key = 'age';
type Age = Person[key]; // 报错

// 示例二
type Age = Person['a' + 'g' + 'e']; // 报错
```

上面两个示例，方括号里面都涉及值的运算，编译时不会进行这种运算，所以会报错。

extends...?: 条件运算符

TypeScript 提供类似 JavaScript 的`?:`运算符这样的三元运算符，但多出了一个`extends`关键字。

条件运算符`extends...?:`可以根据当前类型是否符合某种条件，返回不同的类型。

```
T extends U ? X : Y
```

上面式子中的`extends`用来判断，类型`T`是否可以赋值给类型`U`，即`T`是否为`U`的子类型，这里的`T`和`U`可以是任意类型。

如果`T`能够赋值给类型`U`，表达式的结果为类型`X`，否则结果为类型`Y`。

```
// true
type T = 1 extends number ? true : false;
```

上面示例中，`1`是`number`的子类型，所以返回`true`。

下面是另外一个例子。

```
interface Animal {
  live(): void;
}
interface Dog extends Animal {
  woof(): void;
}

// number
type T1 = Dog extends Animal ? number : string;

// string
type T2 = RegExp extends Animal ? number : string;
```

上面示例中，`Dog`是`Animal`的子类型，所以`T1`的类型是`number`。`RegExp`不是`Animal`的子类型，所以`T2`的类型是`string`。

一般来说，调换`extends`两侧类型，会返回相反的结果。举例来说，有两个类`Cat`和`Animal`，前者是后者的子类型，那么`Cat extends Animal`就为真，而`Animal extends Cat`就为伪。

如果对泛型使用 extends 条件运算，有一个地方需要注意。当泛型的类型参数是一个联合类型时，那么条件运算符会展开这个类型参数，即`T<A|B> = T<A> | T<B>`，所以 extends 对类型参数的每个部分是分别计算的。

```
type Cond<T> = T extends U ? X : Y;

type MyType = Cond<A|B>;
// 等同于 Cond<A> | Cond<B>
// 等同于 (A extends U ? X : Y) | (B extends U ? X : Y)
```

上面示例中，泛型`Cond`的类型参数`A|B`是一个联合类型，进行条件运算时，相当于`A`和`B`分别进行条件运算，返回结果组成一个联合类型。也就是说，如果类型参数是联合类型，条件运算的返回结果依然是一个联合类型。

如果不希望联合类型被条件运算符展开，可以把`extends`两侧的操作数都放在方括号里面。

```
// 示例一
type ToArray<Type> =
  Type extends any ? Type[] : never;

// 返回结果 string[]|number[]
type T = ToArray<string|number>;

// 示例二
type ToArray<Type> =
  [Type] extends [any] ? Type[] : never;

// 返回结果 (string | number)[]
type T = ToArray<string|number>;
```

上面的示例一，泛型`ToArray<Type>`的类型参数`string|number`是一个联合类型，所以会被展开，返回的也是联合类型`string[]|number[]`。示例二是`extends`两侧的运算数都放在方括号里面，左侧是`[Type]`，右侧是`[any]`，这时传入的联合类型不会展开，返回的是一个数组`(string|number)[]`。

条件运算符还可以嵌套使用。

```
type LiteralTypeName<T> =
  T extends undefined ? "undefined" :
  T extends null ? "null" :
  T extends boolean ? "boolean" :
  T extends number ? "number" :
  T extends bigint ? "bigint" :
  T extends string ? "string" :
  never;
```

上面示例是一个多重判断，返回一个字符串的值类型，对应当前类型。下面是它的用法。

```
// "bigint"
type Result1 = LiteralTypeName<123n>;
```

```
// "string" | "number" | "boolean"
type Result2 = LiteralTypeName<true | 1 | 'a'>;
```

infer 关键字

`infer`关键字用来定义泛型里面推断出来的类型参数，而不是外部传入的类型参数。

它通常跟条件运算符一起使用，用在`extends`关键字后面的父类型之中。

```
type Flatten<Type> =
  Type extends Array<infer Item> ? Item : Type;
```

上面示例中，`infer Item`表示`Item`这个参数是 TypeScript 自己推断出来的，不用显式传入，而`Flatten<Type>`则表示`Type`这个类型参数是外部传入的。`Type extends Array<infer Item>`则表示，如果参数`Type`是一个数组，那么就将该数组的成员类型推断为`Item`，即`Item`是从`Type`推断出来的。

一旦使用`infer Item`定义了`Item`，后面的代码就可以直接调用`Item`了。下面是上例的泛型`Flatten<Type>`的用法。

```
// string
type Str = Flatten<string[]>;

// number
type Num = Flatten<number>;
```

上面示例中，第一个例子`Flatten<string[]>`传入的类型参数是`string[]`，可以推断出`Item`的类型是`string`，所以返回的是`string`。第二个例子`Flatten<number>`传入的类型参数是`number`，它不是数组，所以直接返回自身。

如果不用`infer`定义类型参数，那么就要传入两个类型参数。

```
type Flatten<Type, Item> =
  Type extends Array<Item> ? Item : Type;
```

上面是不使用`infer`的写法，每次调用`Flatten`的时候，都要传入两个参数，就比较麻烦。

下面的例子使用`infer`，推断函数的参数类型和返回值类型。

```
type ReturnPromise<T> =
  T extends (...args: infer A) => infer R
  ? (...args: A) => Promise<R>
  : T;
```

上面示例中，如果`T`是函数，就返回这个函数的 Promise 版本，否则原样返回。`infer A`表示该函数的参数类型为`A`，`infer R`表示该函数的返回值类型为`R`。

如果不使用`infer`，就不得不把`ReturnPromise<T>`写成`ReturnPromise<T, A, R>`，这样就很麻烦，相当于开发者必须人肉推断编译器可以完成的工作。

下面是`infer`提取对象指定属性的例子。

```
type MyType<T> =
  T extends {
    a: infer M,
    b: infer N
  } ? [M, N] : never;

// 用法示例
type T = MyType<{ a: string; b: number }>;
// [string, number]
```

上面示例中，`infer`提取了参数对象的属性`a`和属性`b`的类型。

下面是`infer`通过正则匹配提取类型参数的例子。

```
type Str = 'foo-bar';

type Bar = Str extends `foo-${infer rest}` ? rest : never // 'bar'
```

上面示例中，`rest`是从模板字符串提取的类型参数。

is 运算符

函数返回布尔值的时候，可以使用`is`运算符，限定返回值与参数之间的关系。

`is`运算符用来描述返回值属于`true`还是`false`。

```
function isFish(
  pet: Fish|Bird
):pet is Fish {
  return (pet as Fish).swim !== undefined;
}
```

上面示例中，函数`isFish()`的返回值类型为`pet is Fish`，表示如果参数`pet`类型为`Fish`，则返回`true`，否则返回`false`。

`is`运算符总是用于描述函数的返回值类型，写法采用`parameterName is Type`的形式，即左侧为当前函数的参数名，右侧为某一种类型。它返回一个布尔值，表示左侧参数是否属于右侧的类型。

```
type A = { a: string };
type B = { b: string };

function isTypeA(x: A|B): x is A {
  if ('a' in x) return true;
```

```
  return false;
}
```

上面示例中，返回值类型`x is A`可以准确描述函数体内部的运算逻辑。

`is`运算符可以用于类型保护。

```
function isCat(a:any): a is Cat {
  return a.name === 'kitty';
}

let x:Cat|Dog;

if (isCat(x)) {
  x.meow(); // 正确，因为 x 肯定是 Cat 类型
}
```

上面示例中，函数`isCat()`的返回类型是`a is Cat`，它是一个布尔值。后面的`if`语句就用这个返回值进行判断，从而起到类型保护的作用，确保`x`是 Cat 类型，从而`x.meow()`不会报错（假定`Cat`类型拥有`meow()`方法）。

`is`运算符还有一种特殊用法，就是用在类（class）的内部，描述类的方法的返回值。

```
class Teacher {
  isStudent():this is Student {
    return false;
  }
}

class Student {
  isStudent():this is Student {
    return true;
  }
}
```

上面示例中，`isStudent()`方法的返回值类型，取决于该方法内部的`this`是否为`Student`对象。如果是的，就返回布尔值`true`，否则返回`false`。

注意，`this is T`这种写法，只能用来描述方法的返回值类型，而不能用来描述属性的类型。

模板字符串

TypeScript 允许使用模板字符串，构建类型。

模板字符串的最大特点，就是内部可以引用其他类型。

```
type World = "world";
```

```
// "hello world"
type Greeting = `hello ${World}`;
```

上面示例中，类型`Greeting`是一个模板字符串，里面引用了另一个字符串类型`world`，因此`Greeting`实际上是字符串`hello world`。

注意，模板字符串可以引用的类型一共7种，分别是 string、number、bigint、boolean、null、undefined、Enum。引用这7种以外的类型会报错。

```
type Num = 123;
type Obj = { n : 123 };

type T1 = `${Num} received`; // 正确
type T2 = `${Obj} received`; // 报错
```

上面示例中，模板字符串引用数值类型的别名`Num`是可以的，但是引用对象类型的别名`Obj`就会报错。

模板字符串里面引用的类型，如果是一个联合类型，那么它返回的也是一个联合类型，即模板字符串可以展开联合类型。

```
type T = 'A'|'B';

// "A_id"|"B_id"
type U = `${T}_id`;
```

上面示例中，类型`U`是一个模板字符串，里面引用了一个联合类型`T`，导致最后得到的也是一个联合类型。

如果模板字符串引用两个联合类型，它会交叉展开这两个类型。

```
type T = 'A'|'B';

type U = '1'|'2';

// 'A1'|'A2'|'B1'|'B2'
type V = `${T}${U}`;
```

上面示例中，`T`和`U`都是联合类型，各自有两个成员，模板字符串里面引用了这两个类型，最后得到的就是一个4个成员的联合类型。

satisfies 运算符

`satisfies`运算符用来检测某个值是否符合指定类型。有时候，不方便将某个值指定为某种类型，但是希望这个值符合类型条件，这时候就可以用`satisfies`运算符对其进行检测。TypeScript 4.9添加了这个运算符。

举例来说，有一个对象的属性名拼写错误。

```
const palette = {
  red: [255, 0, 0],
```

```
  green: "#00ff00",
  bleu: [0, 0, 255] // 属性名拼写错误
};
```

上面示例中，对象`palette`的属性名拼写错了，将`blue`拼成了`bleu`，我们希望通过指定类型，发现这个错误。

```
type Colors = "red" | "green" | "blue";
type RGB = [number, number, number];

const palette: Record<Colors, string|RGB> = {
  red: [255, 0, 0],
  green: "#00ff00",
  bleu: [0, 0, 255] // 报错
};
```

上面示例中，变量`palette`的类型被指定为`Record<Colors, string|RGB>`，这是一个类型工具，用来返回一个对象，详细介绍见《类型工具》一章。简单说，它的第一个类型参数指定对象的属性名，第二个类型参数指定对象的属性值。

本例的`Record<Colors, string|RGB>`，就表示变量`palette`的属性名应该符合类型`Colors`，属性值应该符合类型`string|RGB`，要么是字符串，要么是元组`RGB`。属性名`bleu`不符合类型`Colors`，所以就报错了。

这样的写法，虽然可以发现属性名的拼写错误，但是带来了新的问题。

```
const greenComponent = palette.green.substring(1, 6); // 报错
```

上面示例中，`palette.green`属性调用`substring()`方法会报错，原因是这个方法只有字符串才有，而`palette.green`的类型是`string|RGB`，除了字符串，还可能是元组`RGB`，而元组并不存在`substring()`方法，所以报错了。

如果要避免报错，要么精确给出变量`palette`每个属性的类型，要么对`palette.green`的值进行类型缩小。两种做法都比较麻烦，也不是很有必要。

这时就可以使用`satisfies`运算符，对`palette`进行类型检测，但是不改变 TypeScript 对`palette`的类型推断。

```
type Colors = "red" | "green" | "blue";
type RGB = [number, number, number];

const palette = {
  red: [255, 0, 0],
  green: "#00ff00",
  bleu: [0, 0, 255] // 报错
} satisfies Record<Colors, string|RGB>;

const greenComponent = palette.green.substring(1); // 不报错
```

上面示例中，变量`palette`的值后面增加了`satisfies Record<Colors, string|RGB>`，表示该值必须满足`Record<Colors, string|RGB>`这个条件，所以能够检测出属性名`bleu`的拼写错误。同时，它不会改变`palette`的类型推断，所以，TypeScript 知道`palette.green`是一个字符串，对其调用`substring()`方法就不会报错。

`satisfies`也可以检测属性值。

```
const palette = {
  red: [255, 0, 0],
  green: "#00ff00",
  blue: [0, 0] // 报错
} satisfies Record<Colors, string|RGB>;
```

上面示例中，属性`blue`的值只有两个成员，不符合元组`RGB`必须有三个成员的条件，从而报错了。

TypeScript 的类型映射

简介

映射（mapping）指的是，将一种类型按照映射规则，转换成另一种类型，通常用于对象类型。

举例来说，现有一个类型`A`和另一个类型`B`。

```
type A = {
  foo: number;
  bar: number;
};

type B = {
  foo: string;
  bar: string;
};
```

上面示例中，这两个类型的属性结构是一样的，但是属性的类型不一样。如果属性数量多的话，逐个写起来就很麻烦。

使用类型映射，就可以从类型`A`得到类型`B`。

```
type A = {
  foo: number;
  bar: number;
};

type B = {
  [prop in keyof A]: string;
};
```

上面示例中，类型`B`采用了属性名索引的写法，`[prop in keyof A]`表示依次得到类型`A`的所有属性名，然后将每个属性的类型改成`string`。

在语法上，`[prop in keyof A]`是一个属性名表达式，表示这里的属性名需要计算得到。具体的计算规则如下：

- `prop`：属性名变量，名字可以随便起。
- `in`：运算符，用来取出右侧的联合类型的每一个成员。
- `keyof A`：返回类型`A`的每一个属性名，组成一个联合类型。

下面是复制原始类型的例子。

```
type A = {
  foo: number;
  bar: string;
```

```
};

type B = {
  [prop in keyof A]: A[prop];
};
```

上面示例中，类型`B`原样复制了类型`A`。

为了增加代码复用性，可以把常用的映射写成泛型。

```
type ToBoolean<Type> = {
  [Property in keyof Type]: boolean;
};
```

上面示例中，定义了一个泛型，可以将其他对象的所有属性值都改成 boolean 类型。

下面是另一个例子。

```
type MyObj = {
  [P in 0|1|2]: string;
};

// 等同于
type MyObj = {
  0: string;
  1: string;
  2: string;
};
```

上面示例中，联合类型`0|1|2`映射成了三个属性名。

不使用联合类型，直接使用某种具体类型进行属性名映射，也是可以的。

```
type MyObj = {
  [p in 'foo']: number;
};

// 等同于
type MyObj = {
  foo: number;
};
```

上面示例中，`p in 'foo'`可以看成只有一个成员的联合类型，因此得到了只有这一个属性的对象类型。

甚至还可以写成`p in string`。

```
type MyObj = {
  [p in string]: boolean;
};

// 等同于
type MyObj = {
```

```
  [p: string]: boolean;
};
```

上面示例中，`[p in string]`就是属性名索引形式`[p: string]`的映射写法。

通过映射，可以把某个对象的所有属性改成可选属性。

```
type A = {
  a: string;
  b: number;
};

type B = {
  [Prop in keyof A]?: A[Prop];
};
```

上面示例中，类型`B`在类型`A`的所有属性名后面添加问号，使得这些属性都变成了可选属性。

事实上，TypeScript 的内置工具类型`Partial<T>`，就是这样实现的。

TypeScript内置的工具类型`Readonly<T>`可以将所有属性改为只读属性，实现也是通过映射。

```
// 将 T 的所有属性改为只读属性
type Readonly<T> = {
  readonly [P in keyof T]: T[P];
};
```

它的用法如下。

```
type T = { a: string; b: number };

type ReadonlyT = Readonly<T>;
// {
//   readonly a: string;
//   readonly b: number;
// }
```

映射修饰符

映射会原样复制原始对象的可选属性和只读属性。

```
type A = {
  a?: string;
  readonly b: number;
};

type B = {
  [Prop in keyof A]: A[Prop];
};
```

```
// 等同于
type B = {
  a?: string;
  readonly b: number;
};
```

上面示例中，类型`B`是类型`A`的映射，把`A`的可选属性和只读属性都保留下来。

如果要删改可选和只读这两个特性，并不是很方便。为了解决这个问题，TypeScript 引入了两个映射修饰符，用来在映射时添加或移除某个属性的`?`修饰符和`readonly`修饰符。

- `+`修饰符：写成`+?`或`+readonly`，为映射属性添加`?`修饰符或`readonly`修饰符。
- `-`修饰符：写成`-?`或`-readonly`，为映射属性移除`?`修饰符或`readonly`修饰符。

下面是添加或移除可选属性的例子。

```
// 添加可选属性
type Optional<Type> = {
  [Prop in keyof Type]+?: Type[Prop];
};

// 移除可选属性
type Concrete<Type> = {
  [Prop in keyof Type]-?: Type[Prop];
};
```

注意，`+?`或`-?`要写在属性名的后面。

下面是添加或移除只读属性的例子。

```
// 添加 readonly
type CreateImmutable<Type> = {
  +readonly [Prop in keyof Type]: Type[Prop];
};

// 移除 readonly
type CreateMutable<Type> = {
  -readonly [Prop in keyof Type]: Type[Prop];
};
```

注意，`+readonly`和`-readonly`要写在属性名的前面。

如果同时增删`?`和`readonly`这两个修饰符，写成下面这样。

```
// 增加
type MyObj<T> = {
  +readonly [P in keyof T]+?: T[P];
};

// 移除
type MyObj<T> = {
```

```
  -readonly [P in keyof T]-?: T[P];
}
```

TypeScript 原生的工具类型`Required<T>`专门移除可选属性，就是使用`-?`修饰符实现的。

注意，`-?`修饰符移除了可选属性以后，该属性就不能等于`undefined`了，实际变成必选属性了。但是，这个修饰符不会移除`null`类型。

另外，`+?`修饰符可以简写成`?`，`+readonly`修饰符可以简写成`readonly`。

```
type A<T> = {
  +readonly [P in keyof T]+?: T[P];
};

// 等同于
type A<T> = {
  readonly [P in keyof T]?: T[P];
};
```

键名重映射

语法

TypeScript 4.1 引入了键名重映射（key remapping），允许改变键名。

```
type A = {
  foo: number;
  bar: number;
};

type B = {
  [p in keyof A as `${p}ID`]: number;
};

// 等同于
type B = {
  fooID: number;
  barID: number;
};
```

上面示例中，类型`B`是类型`A`的映射，但在映射时把属性名改掉了，在原始属性名后面加上了字符串`ID`。

可以看到，键名重映射的语法是在键名映射的后面加上`as + 新类型`子句。这里的“新类型”通常是一个模板字符串，里面可以对原始键名进行各种操作。

下面是另一个例子。

```
interface Person {
  name: string;
  age: number;
```

```
  location: string;
}

type Getters<T> = {
  [P in keyof T
    as `get${Capitalize<string & P>}`]: () => T[P];
};

type LazyPerson = Getters<Person>;
// 等同于
type LazyPerson = {
  getName: () => string;
  getAge: () => number;
  getLocation: () => string;
}
```

上面示例中，类型`LazyPerson`是类型`Person`的映射，并且把键名改掉了。

它的修改键名的代码是一个模板字符串`get${Capitalize<string & P>}`，下面是各个部分的解释。

- `get`：为键名添加的前缀。
- `Capitalize<T>`：一个原生的工具泛型，用来将`T`的首字母变成大写。
- `string & P`：一个交叉类型，其中的`P`是 keyof 运算符返回的键名联合类型`string|number|symbol`，但是`Capitalize<T>`只能接受字符串作为类型参数，因此`string & P`只返回`P`的字符串属性名。

属性过滤

键名重映射还可以过滤掉某些属性。下面的例子是只保留字符串属性。

```
type User = {
  name: string,
  age: number
}

type Filter<T> = {
  [K in keyof T
    as T[K] extends string ? K : never]: string
}

type FilteredUser = Filter<User> // { name: string }
```

上面示例中，映射`K in keyof T`获取类型`T`的每一个属性以后，然后使用`as Type`修改键名。

它的键名重映射`as T[K] extends string ? K : never`，使用了条件运算符。如果属性值`T[K]`的类型是字符串，那么属性名不变，否则属性名类型改为`never`，即这个属性名不存在。这样就等于过滤了不符合条件的属性，只保留属性值为字符串的属性。

联合类型的映射

由于键名重映射可以修改键名类型，所以原始键名的类型不必是`string|number|symbol`，任意的联合类型都可以用来进行键名重映射。

```
type S = {
  kind: 'square',
  x: number,
  y: number,
};

type C = {
  kind: 'circle',
  radius: number,
};

type MyEvents<Events extends { kind: string }> = {
  [E in Events as E['kind']]: (event: E) => void;
}

type Config = MyEvents<S|C>;
// 等同于
type Config = {
  square: (event:S) => void;
  circle: (event:C) => void;
}
```

上面示例中，原始键名的映射是`E in Events`，这里的`Events`是两个对象组成的联合类型`S|C`。所以，`E`是一个对象，然后再通过键名重映射，得到字符串键名`E['kind']`。

参考链接

- Mapped Type Modifiers in TypeScript, Marius Schulz

TypeScript 类型工具

TypeScript 提供了一些内置的类型工具，用来方便地处理各种类型，以及生成新的类型。

这些类型工具都是语言本身提供的，可以直接使用。

Awaited<Type>

`Awaited<Type>`用来取出 Promise 的返回值类型，适合用在描述`then()`方法和 await 命令的参数类型。

```
// string
type A = Awaited<Promise<string>>;
```

上面示例中，`Awaited<Type>`会返回 Promise 的返回值类型（string）。

它也可以返回多重 Promise 的返回值类型。

```
// number
type B = Awaited<Promise<Promise<number>>>;
```

如果它的类型参数不是 Promise 类型，那么就会原样返回。

```
// number | boolean
type C = Awaited<boolean | Promise<number>>;
```

上面示例中，类型参数是一个联合类型，其中的`boolean`会原样返回，所以最终返回的是`number|boolean`。

`Awaited<Type>`的实现如下。

```
type Awaited<T> =
  T extends null | undefined ? T :
  T extends object & {
    then(
      onfulfilled: infer F,
      ...args: infer _
    ): any;
  } ? F extends (
    value: infer V,
    ...args: infer _
  ) => any ? Awaited<V> : never:
  T;
```

ConstructorParameters<Type>

`ConstructorParameters<Type>`提取构造方法`Type`的参数类型，组成一个元组类型返回。

```
type T1 = ConstructorParameters<
  new (x: string, y: number) => object
>; // [x: string, y: number]

type T2 = ConstructorParameters<
  new (x?: string) => object
>; // [x?: string | undefined]
```

它可以返回一些内置构造方法的参数类型。

```
type T1 = ConstructorParameters<
  ErrorConstructor
>; // [message?: string]

type T2 = ConstructorParameters<
  FunctionConstructor
>; // string[]

type T3 = ConstructorParameters<
  RegExpConstructor
>; // [pattern:string|RegExp, flags?:string]
```

如果参数类型不是构造方法，就会报错。

```
type T1 = ConstructorParameters<string>; // 报错

type T2 = ConstructorParameters<Function>; // 报错
```

`any`类型和`never`类型是两个特殊值，分别返回`unknown[]`和`never`。

```
type T1 = ConstructorParameters<any>;  // unknown[]

type T2 = ConstructorParameters<never>; // never
```

`ConstructorParameters<Type>`的实现如下。

```
type ConstructorParameters<
  T extends abstract new (...args: any) => any
> = T extends abstract new (...args: infer P)
  => any ? P : never
```

Exclude<UnionType, ExcludedMembers>

`Exclude<UnionType, ExcludedMembers>`用来从联合类型`UnionType`里面，删除某些类型`ExcludedMembers`，组成一个新的类型返回。

```
type T1 = Exclude<'a'|'b'|'c', 'a'>; // 'b'|'c'
type T2 = Exclude<'a'|'b'|'c', 'a'|'b'>; // 'c'
type T3 = Exclude<string|(() => void), Function>; // string
```

```
type T4 = Exclude<string | string[], any[]>; // string
type T5 = Exclude<(() => void) | null, Function>; // null
type T6 = Exclude<200 | 400, 200 | 201>; // 400
type T7 = Exclude<number, boolean>; // number
```

`Exclude<UnionType, ExcludedMembers>`的实现如下。

```
type Exclude<T, U> = T extends U ? never : T;
```

上面代码中，等号右边的部分，表示先判断`T`是否兼容`U`，如果是的就返回`never`类型，否则返回当前类型`T`。由于`never`类型是任何其他类型的子类型，它跟其他类型组成联合类型时，可以直接将`never`类型从联合类型中“消掉”，因此`Exclude<T, U>`就相当于删除兼容的类型，剩下不兼容的类型。

Extract<UnionType, Union>

`Extract<UnionType, Union>`用来从联合类型`UnionType`之中，提取指定类型`Union`，组成一个新类型返回。它与`Exclude<T, U>`正好相反。

```
type T1 = Extract<'a'|'b'|'c', 'a'>; // 'a'
type T2 = Extract<'a'|'b'|'c', 'a'|'b'>; // 'a'|'b'
type T3 = Extract<'a'|'b'|'c', 'a'|'d'>; // 'a'
type T4 = Extract<string | string[], any[]>; // string[]
type T5 = Extract<(() => void) | null, Function>; // () => void
type T6 = Extract<200 | 400, 200 | 201>; // 200
```

如果参数类型`Union`不包含在联合类型`UnionType`之中，则返回`never`类型。

```
type T = Extract<string|number, boolean>; // never
```

`Extract<UnionType, Union>`的实现如下。

```
type Extract<T, U> = T extends U ? T : never;
```

InstanceType<Type>

`InstanceType<Type>`提取构造函数的返回值的类型（即实例类型），参数`Type`是一个构造函数，等同于构造函数的`ReturnType<Type>`。

```
type T = InstanceType<
  new () => object
>; // object
```

上面示例中，类型参数是一个构造函数`new () => object`，返回值是该构造函数的实例类型（`object`）。

下面是一些例子。

```
type A = InstanceType<ErrorConstructor>; // Error
type B = InstanceType<FunctionConstructor>; // Function
type C = InstanceType<RegExpConstructor>; // RegExp
```

上面示例中，`InstanceType<T>`的参数都是 TypeScript 内置的原生对象的构造函数类型，`InstanceType<T>`的返回值就是这些构造函数的实例类型。

由于 Class 作为类型，代表实例类型。要获取它的构造方法，必须把它当成值，然后用`typeof`运算符获取它的构造方法类型。

```
class C {
  x = 0;
  y = 0;
}

type T = InstanceType<typeof C>; // C
```

上面示例中，`typeof C`是`C`的构造方法类型，然后 InstanceType 就能获得实例类型，即`C`本身。

如果类型参数不是构造方法，就会报错。

```
type T1 = InstanceType<string>; // 报错

type T2 = InstanceType<Function>; // 报错
```

如果类型参数是`any`或`never`两个特殊值，分别返回`any`和`never`。

```
type T1 = InstanceType<any>; // any

type T2 = InstanceType<never>; // never
```

`InstanceType<Type>`的实现如下。

```
type InstanceType<
  T extends abstract new (...args:any) => any
> = T extends abstract new (...args: any) => infer R ? R :
  any;
```

NonNullable<Type>

`NonNullable<Type>`用来从联合类型`Type`删除`null`类型和`undefined`类型，组成一个新类型返回，也就是返回`Type`的非空类型版本。

```
// string|number
type T1 = NonNullable<string|number|undefined>;

// string[]
type T2 = NonNullable<string[]|null|undefined>;
```

```
type T3 = NonNullable<boolean>; // boolean
type T4 = NonNullable<number|null>; // number
type T5 = NonNullable<string|undefined>; // string
type T6 = NonNullable<null|undefined>; // never
```

`NonNullable<Type>`的实现如下。

```
type NonNullable<T> = T & {}
```

上面代码中，`T & {}`等同于求`T & Object`的交叉类型。由于 TypeScript 的非空值都属于`Object`的子类型，所以会返回自身；而`null`和`undefined`不属于`Object`，会返回`never`类型。

Omit<Type, Keys>

`Omit<Type, Keys>`用来从对象类型`Type`中，删除指定的属性`Keys`，组成一个新的对象类型返回。

```
interface A {
  x: number;
  y: number;
}

type T1 = Omit<A, 'x'>;       // { y: number }
type T2 = Omit<A, 'y'>;       // { x: number }
type T3 = Omit<A, 'x' | 'y'>; // { }
```

上面示例中，`Omit<Type, Keys>`从对象类型`A`里面删除指定属性，返回剩下的属性。

指定删除的键名`Keys`可以是对象类型`Type`中不存在的属性，但必须兼容`string|number|symbol`。

```
interface A {
  x: number;
  y: number;
}

type T = Omit<A, 'z'>; // { x: number; y: number }
```

上面示例中，对象类型`A`中不存在属性`z`，所以就原样返回了。

`Omit<Type, Keys>`的实现如下。

```
type Omit<T, K extends keyof any>
  = Pick<T, Exclude<keyof T, K>>;
```

OmitThisParameter<Type>

`OmitThisParameter<Type>`从函数类型中移除 this 参数。

```
function toHex(this: number) {
  return this.toString(16);
}

type T = OmitThisParameter<typeof toHex>; // () => string
```

上面示例中，`OmitThisParameter<T>`给出了函数`toHex()`的类型，并将其中的`this`参数删除。

如果函数没有 this 参数，则返回原始函数类型。

`OmitThisParameter<Type>`的实现如下。

```
type OmitThisParameter<T> =
  unknown extends ThisParameterType<T> ? T :
  T extends (...args: infer A) => infer R ?
  (...args: A) => R : T;
```

Parameters<Type>

`Parameters<Type>`从函数类型`Type`里面提取参数类型，组成一个元组返回。

```
type T1 = Parameters<() => string>; // []

type T2 = Parameters<(s:string) => void>; // [s:string]

type T3 = Parameters<<T>(arg: T) => T>;    // [arg: unknown]

type T4 = Parameters<
  (x:{ a: number; b: string }) => void
>; // [x: { a: number, b: string }]

type T5 = Parameters<
  (a:number, b:number) => number
>; // [a:number, b:number]
```

上面示例中，`Parameters<Type>`的返回值会包括函数的参数名，这一点需要注意。

如果参数类型`Type`不是带有参数的函数形式，会报错。

```
// 报错
type T1 = Parameters<string>;

// 报错
type T2 = Parameters<Function>;
```

由于`any`和`never`是两个特殊值，会返回`unknown[]`和`never`。

```
type T1 = Parameters<any>; // unknown[]
```

```
type T2 = Parameters<never>; // never
```

`Parameters<Type>`主要用于从外部模块提供的函数类型中，获取参数类型。

```
interface SecretName {
  first: string;
  last: string;
}

interface SecretSanta {
  name: SecretName;
  gift: string;
}

export function getGift(
  name: SecretName,
  gift: string
): SecretSanta {
 // ...
}
```

上面示例中，模块只输出了函数`getGift()`，没有输出参数`SecretName`和返回值`SecretSanta`。这时就可以通过`Parameters<T>`和`ReturnType<T>`拿到这两个接口类型。

```
type ParaT = Parameters<typeof getGift>[0]; // SecretName

type ReturnT = ReturnType<typeof getGift>; // SecretSanta
```

`Parameters<Type>`的实现如下。

```
type Parameters<T extends (...args: any) => any> =
  T extends (...args: infer P)
  => any ? P : never
```

Partial<Type>

`Partial<Type>`返回一个新类型，将参数类型`Type`的所有属性变为可选属性。

```
interface A {
  x: number;
  y: number;
}

type T = Partial<A>; // { x?: number; y?: number; }
```

`Partial<Type>`的实现如下。

```
type Partial<T> = {
  [P in keyof T]?: T[P];
```

```
};
```

Pick<Type, Keys>

`Pick<Type, Keys>`返回一个新的对象类型，第一个参数`Type`是一个对象类型，第二个参数`Keys`是`Type`里面被选定的键名。

```
interface A {
  x: number;
  y: number;
}

type T1 = Pick<A, 'x'>; // { x: number }
type T2 = Pick<A, 'y'>; // { y: number }
type T3 = Pick<A, 'x'|'y'>;  // { x: number; y: number }
```

上面示例中，`Pick<Type, Keys>`会从对象类型`A`里面挑出指定的键名，组成一个新的对象类型。

指定的键名`Keys`必须是对象类型`Type`里面已经存在的键名，否则会报错。

```
interface A {
  x: number;
  y: number;
}

type T = Pick<A, 'z'>; // 报错
```

上面示例中，对象类型`A`不存在键名`z`，所以报错了。

`Pick<Type, Keys>`的实现如下。

```
type Pick<T, K extends keyof T> = {
  [P in K]: T[P];
};
```

Readonly<Type>

`Readonly<Type>`返回一个新类型，将参数类型`Type`的所有属性变为只读属性。

```
interface A {
  x: number;
  y?: number;
}

// { readonly x: number; readonly y?: number; }
type T = Readonly<A>;
```

上面示例中，`y`是可选属性，`Readonly<Type>`不会改变这一点，只会让`y`变成只读。

`Readonly<Type>`的实现如下。

```
type Readonly<T> = {
  readonly [P in keyof T]: T[P];
};
```

我们可以自定义类型工具`Mutable<Type>`，将参数类型的所有属性变成可变属性。

```
type Mutable<T> = {
  -readonly [P in keyof T]: T[P];
};
```

上面代码中，`-readonly`表示去除属性的只读标志。

相应地，`+readonly`就表示增加只读标志，等同于`readonly`。因此，`Readonly<Type>`的实现也可以写成下面这样。

```
type Readonly<T> = {
  +readonly [P in keyof T]: T[P];
};
```

`Readonly<Type>`可以与`Partial<Type>`结合使用，将所有属性变成只读的可选属性。

```
interface Person {
  name: string;
  age: number;
}

const worker: Readonly<Partial<Person>>
  = { name: '张三' };

worker.name = '李四'; // 报错
```

Record<Keys, Type>

`Record<Keys, Type>`返回一个对象类型，参数`Keys`用作键名，参数`Type`用作键值类型。

```
// { a: number }
type T = Record<'a', number>;
```

上面示例中，`Record<Keys, Type>`的第一个参数`a`，用作对象的键名，第二个参数`number`是`a`的键值类型。

参数`Keys`可以是联合类型，这时会依次展开为多个键。

```
// { a: number, b: number }
type T = Record<'a'|'b', number>;
```

上面示例中，第一个参数是联合类型`'a'|'b'`，展开成两个键名`a`和`b`。

如果参数`Type`是联合类型，就表明键值是联合类型。

```
// { a: number|string }
type T = Record<'a', number|string>;
```

参数`Keys`的类型必须兼容`string|number|symbol`，否则不能用作键名，会报错。

`Record<Keys, Type>`的实现如下。

```
type Record<K extends string|number|symbol, T>
  = { [P in K]: T; }
```

Required<Type>

`Required<Type>`返回一个新类型，将参数类型`Type`的所有属性变为必选属性。它与`Partial<Type>`的作用正好相反。

```
interface A {
  x?: number;
  y: number;
}

type T = Required<A>; // { x: number; y: number; }
```

`Required<Type>`的实现如下。

```
type Required<T> = {
  [P in keyof T]-?: T[P];
};
```

上面代码中，符号`-?`表示去除可选属性的“问号”，使其变成必选属性。

相对应地，符号`+?`表示增加可选属性的“问号”，等同于`?`。因此，前面的`Partial<Type>`的定义也可以写成下面这样。

```
type Partial<T> = {
  [P in keyof T]+?: T[P];
};
```

ReadonlyArray<Type>

`ReadonlyArray<Type>`用来生成一个只读数组类型，类型参数`Type`表示数组成员的类型。

```
const values: ReadonlyArray<string>
  = ['a', 'b', 'c'];
```

```
values[0] = 'x'; // 报错
values.push('x'); // 报错
values.pop(); // 报错
values.splice(1, 1); // 报错
```

上面示例中，变量`values`的类型是一个只读数组，所以修改成员会报错，并且那些会修改源数组的方法`push()`、`pop()`、`splice()`等都不存在。

`ReadonlyArray<Type>`的实现如下。

```
interface ReadonlyArray<T> {
  readonly length: number;

  readonly [n: number]: T;

  // ...
}
```

ReturnType<Type>

`ReturnType<Type>`提取函数类型`Type`的返回值类型，作为一个新类型返回。

```
type T1 = ReturnType<() => string>; // string

type T2 = ReturnType<() => {
  a: string; b: number
}>; // { a: string; b: number }

type T3 = ReturnType<(s:string) => void>; // void

type T4 = ReturnType<() => () => any[]>; // () => any[]

type T5 = ReturnType<typeof Math.random>; // number

type T6 = ReturnType<typeof Array.isArray>; // boolean
```

如果参数类型是泛型函数，返回值取决于泛型类型。如果泛型不带有限制条件，就会返回`unknown`。

```
type T1 = ReturnType<<T>() => T>; // unknown

type T2 = ReturnType<
  <T extends U, U extends number[]>() => T
>; // number[]
```

如果类型不是函数，会报错。

```
type T1 = ReturnType<boolean>; // 报错

type T2 = ReturnType<Function>; // 报错
```

`any`和`never`是两个特殊值，分别返回`any`和`never`。

```
type T1 = ReturnType<any>; // any

type T2 = ReturnType<never>; // never
```

`ReturnType<Type>`的实现如下。

```
type ReturnType<
  T extends (...args: any) => any
> =
  T extends (...args: any) => infer R ? R : any;
```

ThisParameterType<Type>

`ThisParameterType<Type>`提取函数类型中`this`参数的类型。

```
function toHex(this:number) {
  return this.toString(16);
}

type T = ThisParameterType<typeof toHex>; // number
```

如果函数没有`this`参数，则返回`unknown`。

`ThisParameterType<Type>`的实现如下。

```
type ThisParameterType<T> =
  T extends (
    this: infer U,
    ...args: never
  ) => any ? U : unknown;
```

ThisType<Type>

`ThisType<Type>`不返回类型，只用来跟其他类型组成交叉类型，用来提示 TypeScript 其他类型里面的`this`的类型。

```
interface HelperThisValue {
  logError: (error:string) => void;
}

let helperFunctions:
  { [name: string]: Function } &
  ThisType<HelperThisValue>
= {
  hello: function() {
    this.logError("Error: Something wrong!"); // 正确
```

```
    this.update(); // 报错
  }
}
```

上面示例中，变量`helperFunctions`的类型是一个正常的对象类型与`ThisType<HelperThisValue>`组成的交叉类型。

这里的`ThisType`的作用是提示 TypeScript，变量`helperFunctions`的`this`应该满足`HelperThisValue`的条件。所以，`this.logError()`可以正确调用，而`this.update()`会报错，因为`HelperThisValue`里面没有这个方法。

注意，使用这个类型工具时，必须打开`noImplicitThis`设置。

下面是另一个例子。

```
let obj: ThisType<{ x: number }> &
  { getX: () => number };

obj = {
  getX() {
    return this.x + this.y; // 报错
  },
};
```

上面示例中，`getX()`里面的`this.y`会报错，因为根据`ThisType<{ x: number }>`，这个对象的`this`不包含属性`y`。

`ThisType<Type>`的实现就是一个空接口。

```
interface ThisType<T> { }
```

字符串类型工具

TypeScript 内置了四个字符串类型工具，专门用来操作字符串类型。这四个工具类型都定义在 TypeScript 自带的`.d.ts`文件里面。

它们的实现都是在底层调用 JavaScript 引擎提供 JavaScript 字符操作方法。

Uppercase<StringType>

`Uppercase<StringType>`将字符串类型的每个字符转为大写。

```
type A = 'hello';

// "HELLO"
type B = Uppercase<A>;
```

上面示例中，`Uppercase<T>`将 hello 转为 HELLO。

Lowercase<StringType>

`Lowercase<StringType>`将字符串的每个字符转为小写。

```
type A = 'HELLO';

// "hello"
type B = Lowercase<A>;
```

上面示例中，`Lowercase<T>`将 HELLO 转为 hello。

Capitalize<StringType>

`Capitalize<StringType>`将字符串的第一个字符转为大写。

```
type A = 'hello';

// "Hello"
type B = Capitalize<A>;
```

上面示例中，`Capitalize<T>`将 hello 转为 Hello。

Uncapitalize<StringType>

`Uncapitalize<StringType>` 将字符串的第一个字符转为小写。

```
type A = 'HELLO';

// "hELLO"
type B = Uncapitalize<A>;
```

上面示例中，`Uncapitalize<T>`将 HELLO 转为 hELLO。

参考链接

- What is TypeScript's ThisType used for?

TypeScript 的注释指令

TypeScript 接受一些注释指令。

所谓“注释指令”，指的是采用 JS 双斜杠注释的形式，向编译器发出的命令。

// @ts-nocheck

`// @ts-nocheck` 告诉编译器不对当前脚本进行类型检查，可以用于 TypeScript 脚本，也可以用于 JavaScript 脚本。

```
// @ts-nocheck

const element = document.getElementById(123);
```

上面示例中，`document.getElementById(123)` 存在类型错误，但是编译器不对该脚本进行类型检查，所以不会报错。

// @ts-check

如果一个 JavaScript 脚本顶部添加了 `// @ts-check`，那么编译器将对该脚本进行类型检查，不论是否启用了 `checkJs` 编译选项。

```
// @ts-check
let isChecked = true;

console.log(isChceked); // 报错
```

上面示例是一个 JavaScript 脚本，`// @ts-check` 告诉 TypeScript 编译器对其进行类型检查，所以最后一行会报错，提示拼写错误。

// @ts-ignore

`// @ts-ignore` 告诉编译器不对下一行代码进行类型检查，可以用于 TypeScript 脚本，也可以用于 JavaScript 脚本。

```
let x:number;

x = 0;

// @ts-ignore
x = false; // 不报错
```

上面示例中，最后一行是类型错误，变量`x`的类型是`number`，不能等于布尔值。但是因为前面加上了`// @ts-ignore`，编译器会跳过这一行的类型检查，所以不会报错。

// @ts-expect-error

`// @ts-expect-error`主要用在测试用例，当下一行有类型错误时，它会压制 TypeScript 的报错信息（即不显示报错信息），把错误留给代码自己处理。

```
function doStuff(abc: string, xyz: string) {
  assert(typeof abc === "string");
  assert(typeof xyz === "string");
  // do some stuff
}

expect(() => {
  // @ts-expect-error
  doStuff(123, 456);
}).toThrow();
```

上面示例是一个测试用例，倒数第二行的`doStuff(123, 456)`的参数类型与定义不一致，TypeScript 引擎会报错。但是，测试用例本身测试的就是这个错误，已经有专门的处理代码，所以这里可以使用`// @ts-expect-error`，不显示引擎的报错信息。

如果下一行没有类型错误，`// @ts-expect-error`则会显示一行提示。

```
// @ts-expect-error
console.log(1 + 1);
// 输出 Unused '@ts-expect-error' directive.
```

上面示例中，第二行是正确代码，这时系统会给出一个提示，表示`@ts-expect-error`没有用到。

JSDoc

TypeScript 直接处理 JS 文件时，如果无法推断出类型，会使用 JS 脚本里面的 JSDoc 注释。

使用 JSDoc 时，有两个基本要求。

（1）JSDoc 注释必须以`/**`开始，其中星号（`*`）的数量必须为两个。若使用其他形式的多行注释，则 JSDoc 会忽略该条注释。

（2）JSDoc 注释必须与它描述的代码处于相邻的位置，并且注释在上，代码在下。

下面是 JSDoc 的一个简单例子。

```
/**
 * @param {string} somebody
 */
function sayHello(somebody) {
```

```
  console.log('Hello ' + somebody);
}
```

上面示例中，注释里面的`@param`是一个 JSDoc 声明，表示下面的函数`sayHello()`的参数`somebody`类型为`string`。

TypeScript 编译器支持大部分的 JSDoc 声明，下面介绍其中的一些。

@typedef

`@typedef`命令创建自定义类型，等同于 TypeScript 里面的类型别名。

```
/**
 * @typedef {(number | string)} NumberLike
 */
```

上面示例中，定义了一个名为`NumberLike`的新类型，它是由`number`和`string`构成的联合类型，等同于 TypeScript 的如下语句。

```
type NumberLike = string | number;
```

@type

`@type`命令定义变量的类型。

```
/**
 * @type {string}
 */
let a;
```

上面示例中，`@type`定义了变量`a`的类型为`string`。

在`@type`命令中可以使用由`@typedef`命令创建的类型。

```
/**
 * @typedef {(number | string)} NumberLike
 */

/**
 * @type {NumberLike}
 */
let a = 0;
```

在`@type`命令中允许使用 TypeScript 类型及其语法。

```
/**@type {true | false} */
let a;

/** @type {number[]} */
```

```
let b;

/** @type {Array<number>} */
let c;

/** @type {{ readonly x: number, y?: string }} */
let d;

/** @type {(s: string, b: boolean) => number} */
let e;
```

@param

`@param`命令用于定义函数参数的类型。

```
/**
 * @param {string}  x
 */
function foo(x) {}
```

如果是可选参数，需要将参数名放在方括号`[]`里面。

```
/**
 * @param {string}  [x]
 */
function foo(x) {}
```

方括号里面，还可以指定参数默认值。

```
/**
 * @param {string} [x="bar"]
 */
function foo(x) {}
```

上面示例中，参数`x`的默认值是字符串`bar`。

@return，@returns

`@return`和`@returns`命令的作用相同，指定函数返回值的类型。

```
/**
 * @return {boolean}
 */
function foo() {
  return true;
}

/**
 * @returns {number}
 */
function bar() {
```

```
  return 0;
}
```

@extends 和类型修饰符

`@extends`命令用于定义继承的基类。

```
/**
 * @extends {Base}
 */
class Derived extends Base {
}
```

`@public`、`@protected`、`@private`分别指定类的公开成员、保护成员和私有成员。

`@readonly`指定只读成员。

```
class Base {
  /**
   * @public
   * @readonly
   */
  x = 0;

  /**
   *  @protected
   */
  y = 0;
}
```

tsconfig.json

简介

`tsconfig.json`是 TypeScript 项目的配置文件，放在项目的根目录。反过来说，如果一个目录里面有`tsconfig.json`，TypeScript 就认为这是项目的根目录。

如果项目源码是 JavaScript，但是想用 TypeScript 处理，那么配置文件的名字是`jsconfig.json`，它跟`tsconfig`的写法是一样的。

`tsconfig.json`文件主要供`tsc`编译器使用，它的命令行参数`--project`或`-p`可以指定`tsconfig.json`的位置（目录或文件皆可）。

```
$ tsc -p ./dir
```

如果不指定配置文件的位置，`tsc`就会在当前目录下搜索`tsconfig.json`文件，如果不存在，就到上一级目录搜索，直到找到为止。

`tsconfig.json`文件的格式，是一个 JSON 对象，最简单的情况可以只放置一个空对象`{}`。下面是一个示例。

```
{
  "compilerOptions": {
    "outDir": "./built",
    "allowJs": true,
    "target": "es5"
  },
  "include": ["./src/**/*"]
}
```

本章后面会详细介绍`tsconfig.json`的各个属性，这里简单说一下，上面示例的四个属性的含义。

- include：指定哪些文件需要编译。
- allowJs：指定源目录的 JavaScript 文件是否原样拷贝到编译后的目录。
- outDir：指定编译产物存放的目录。
- target：指定编译产物的 JS 版本。

`tsconfig.json`文件可以不必手写，使用 tsc 命令的`--init`参数自动生成。

```
$ tsc --init
```

上面命令生成的`tsconfig.json`文件，里面会有一些默认配置。

你也可以使用别人预先写好的 tsconfig.json 文件，npm 的`@tsconfig`名称空间下面有很多模块，都是写好的`tsconfig.json`样本，比如`@tsconfig/recommended`和`@tsconfig/node16`。

这些模块需要安装，以`@tsconfig/deno`为例。

```
$ npm install --save-dev @tsconfig/deno
# 或者
$ yarn add --dev @tsconfig/deno
```

安装以后，就可以在`tsconfig.json`里面引用这个模块，相当于继承它的设置，然后进行扩展。

```
{
  "extends": "@tsconfig/deno/tsconfig.json"
}
```

`@tsconfig`空间下包含的完整 tsconfig 文件目录，可以查看 GitHub。

`tsconfig.json`的一级属性并不多，只有很少几个，但是`compilerOptions`属性有很多二级属性。下面先逐一介绍一级属性，然后再介绍`compilerOptions`的二级属性，按照首字母排序。

exclude

`exclude`属性是一个数组，必须与`include`属性一起使用，用来从编译列表中去除指定的文件。它也支持使用与`include`属性相同的通配符。

```
{
  "include": ["**/*"],
  "exclude": ["**/*.spec.ts"]
}
```

extends

`tsconfig.json`可以继承另一个`tsconfig.json`文件的配置。如果一个项目有多个配置，可以把共同的配置写成`tsconfig.base.json`，其他的配置文件继承该文件，这样便于维护和修改。

`extends`属性用来指定所要继承的配置文件。它可以是本地文件。

```
{
  "extends": "../tsconfig.base.json"
}
```

如果`extends`属性指定的路径不是以`./`或`../`开头，那么编译器将在`node_modules`目录下查找指定的配置文件。

`extends`属性也可以继承已发布的 npm 模块里面的 tsconfig 文件。

```
{
  "extends": "@tsconfig/node12/tsconfig.json"
}
```

`extends`指定的`tsconfig.json`会先加载，然后加载当前的`tsconfig.json`。如果两者有重名的属性，后者会覆盖前者。

files

`files`属性指定编译的文件列表，如果其中有一个文件不存在，就会报错。

它是一个数组，排在前面的文件先编译。

```
{
  "files": ["a.ts", "b.ts"]
}
```

该属性必须逐一列出文件，不支持文件匹配。如果文件较多，建议使用`include`和`exclude`属性。

include

`include`属性指定所要编译的文件列表，既支持逐一列出文件，也支持通配符。文件位置相对于当前配置文件而定。

```
{
  "include": ["src/**/*", "tests/**/*"]
}
```

`include`属性支持三种通配符。

- `?`：指代单个字符
- `*`：指代任意字符，不含路径分隔符
- `**`：指定任意目录层级。

如果不指定文件后缀名，默认包括`.ts`、`.tsx`和`.d.ts`文件。如果打开了`allowJs`，那么还包括`.js`和`.jsx`。

references

`references`属性是一个数组，数组成员为对象，适合一个大项目由许多小项目构成的情况，用来设置需要引用的底层项目。

```
{
  "references": [
    { "path": "../pkg1" },
    { "path": "../pkg2/tsconfig.json" }
```

```
  ]
}
```

`references`数组成员对象的`path`属性，既可以是含有文件`tsconfig.json`的目录，也可以直接是该文件。

与此同时，引用的底层项目的`tsconfig.json`必须启用`composite`属性。

```
{
  "compilerOptions": {
    "composite": true
  }
}
```

compilerOptions

`compilerOptions`属性用来定制编译行为。这个属性可以省略，这时编译器将使用默认设置。

allowJs

`allowJs`允许 TypeScript 项目加载 JS 脚本。编译时，也会将 JS 文件，一起拷贝到输出目录。

```
{
  "compilerOptions": {
    "allowJs": true
  }
}
```

alwaysStrict

`alwaysStrict`确保脚本以 ECMAScript 严格模式进行解析，因此脚本头部不用写`"use strict"`。它的值是一个布尔值，默认为`true`。

allowSyntheticDefaultImports

`allowSyntheticDefaultImports`允许`import`命令默认加载没有`default`输出的模块。

比如，打开这个设置，就可以写`import React from "react";`，而不是`import * as React from "react";`。

allowUnreachableCode

`allowUnreachableCode`设置是否允许存在不可能执行到的代码。它的值有三种可能。

- `undefined`： 默认值，编辑器显示警告。
- `true`：忽略不可能执行到的代码。
- `false`：编译器报错。

allowUnusedLabels

`allowUnusedLabels`设置是否允许存在没有用到的代码标签（label）。它的值有三种可能。

- `undefined`：默认值，编辑器显示警告。
- `true`：忽略没有用到的代码标签。
- `false`：编译器报错。

baseUrl

`baseUrl`的值为字符串，指定 TypeScript 项目的基准目录。

由于默认是以 tsconfig.json 的位置作为基准目录，所以一般情况不需要使用该属性。

```
{
  "compilerOptions": {
    "baseUrl": "./"
  }
}
```

上面示例中，`baseUrl`为当前目录`./`。那么，当遇到下面的语句，TypeScript 将以`./`为起点，寻找`hello/world.ts`。

```
import { helloWorld } from "hello/world";
```

checkJs

`checkJS`设置对 JS 文件同样进行类型检查。打开这个属性，也会自动打开`allowJs`。它等同于在 JS 脚本的头部添加`// @ts-check`命令。

```
{
  "compilerOptions":{
    "checkJs": true
  }
}
```

composite

`composite`打开某些设置，使得 TypeScript 项目可以进行增量构建，往往跟`incremental`属性配合使用。

declaration

`declaration`设置编译时是否为每个脚本生成类型声明文件`.d.ts`。

```
{
  "compilerOptions": {
    "declaration": true
```

```
  }
}
```

declarationDir

`declarationDir`设置生成的`.d.ts`文件所在的目录。

```
{
  "compilerOptions": {
    "declaration": true,
    "declarationDir": "./types"
  }
}
```

declarationMap

`declarationMap`设置生成`.d.ts`类型声明文件的同时，还会生成对应的 Source Map 文件。

```
{
  "compilerOptions": {
    "declaration": true,
    "declarationMap": true
  }
}
```

emitBOM

`emitBOM`设置是否在编译结果的文件头添加字节顺序标志 BOM，默认值是`false`。

emitDeclarationOnly

`emitDeclarationOnly`设置编译后只生成`.d.ts`文件，不生成`.js`文件。

esModuleInterop

`esModuleInterop`修复了一些 CommonJS 和 ES6 模块之间的兼容性问题。

如果`module`属性为`node16`或`nodenext`，则`esModuleInterop`默认为`true`，其他情况默认为`false`。

打开这个属性，使用`import`命令加载 CommonJS 模块时，TypeScript 会严格检查兼容性问题是否存在。

```
import * as moment from 'moment'
moment(); // 报错
```

上面示例中，根据 ES6 规范，`import * as moment`里面的`moment`是一个对象，不能当作函数调用，所以第二行报错了。

解决方法就是改写上面的语句，改成加载默认接口。

```
import moment from 'moment'
moment(); // 不报错
```

打开`esModuleInterop`以后，如果将上面的代码编译成 CommonJS 模块格式，就会加入一些辅助函数，保证编译后的代码行为正确。

注意，打开`esModuleInterop`，将自动打开`allowSyntheticDefaultImports`。

exactOptionalPropertyTypes

`exactOptionalPropertyTypes`设置可选属性不能赋值为`undefined`。

```
// 打开 exactOptionalPropertyTypes
interface MyObj {
  foo?: 'A' | 'B';
}

let obj:MyObj = { foo: 'A' };

obj.foo = undefined; // 报错
```

上面示例中，`foo`是可选属性，打开`exactOptionalPropertyTypes`以后，该属性就不能显式赋值为`undefined`。

forceConsistentCasingInFileNames

`forceConsistentCasingInFileNames`设置文件名是否为大小写敏感，默认为`true`。

incremental

`incremental`让 TypeScript 项目构建时产生文件`tsbuildinfo`，从而完成增量构建。

inlineSourceMap

`inlineSourceMap`设置将 SourceMap 文件写入编译后的 JS 文件中，否则会单独生成一个`.js.map`文件。

inlineSources

`inlineSources`设置将原始的`.ts`代码嵌入编译后的 JS 中。

它要求`sourceMap`或`inlineSourceMap`至少打开一个。

isolatedModules

`isolatedModules`设置如果当前 TypeScript 脚本作为单个模块编译，是否会因为缺少其他脚本的类型信息而报错，主要便于非官方的编译工具（比如 Babel）正确编译单个脚本。

jsx

`jsx`设置如何处理`.tsx`文件。它可以取以下五个值。

- `preserve`：保持 jsx 语法不变，输出的文件名为`.jsx`。
- `react`：将`<div />`编译成`React.createElement("div")`，输出的文件名为`.js`。
- `react-native`：保持 jsx 语法不变，输出的文件后缀名为`.js`。
- `react-jsx`：将`<div />`编译成`_jsx("div")`，输出的文件名为`.js`。
- `react-jsxdev`：跟`react-jsx`类似，但是为`_jsx()`加上更多的开发调试项，输出的文件名为`.js`。

```
{
  "compilerOptions": {
    "jsx": "preserve"
  }
}
```

lib

`lib`值是一个数组，描述项目需要加载的 TypeScript 内置类型描述文件，跟三斜线指令`/// <reference lib="" />`作用相同。

```
{
  "compilerOptions": {
    "lib": ["dom", "es2021"]
  }
}
```

TypeScript 内置的类型描述文件，主要有以下一些，完整的清单可以参考 TypeScript 源码。

- ES5
- ES2015
- ES6
- ES2016
- ES7
- ES2017
- ES2018
- ES2019
- ES2020
- ES2021
- ES2022
- ESNext
- DOM
- WebWorker
- ScriptHost

listEmittedFiles

`listEmittedFiles`设置编译时在终端显示，生成了哪些文件。

```
{
  "compilerOptions": {
    "listEmittedFiles": true
  }
}
```

listFiles

`listFiles`设置编译时在终端显示，参与本次编译的文件列表。

```
{
  "compilerOptions": {
    "listFiles": true
  }
}
```

mapRoot

`mapRoot`指定 SourceMap 文件的位置，而不是默认的生成位置。

```
{
  "compilerOptions": {
    "sourceMap": true,
    "mapRoot": "https://my-website.com/debug/sourcemaps/"
  }
}
```

module

`module`指定编译产物的模块格式。它的默认值与`target`属性有关，如果`target`是`ES3`或`ES5`，它的默认值是`commonjs`，否则就是`ES6/ES2015`。

```
{
  "compilerOptions": {
    "module": "commonjs"
  }
}
```

它可以取以下值：none、commonjs、amd、umd、system、es6/es2015、es2020、es2022、esnext、node16、nodenext。

moduleResolution

`moduleResolution`确定模块路径的算法，即如何查找模块。它可以取以下四种值。

- `node`：采用 Node.js 的 CommonJS 模块算法。
- `node16`或`nodenext`：采用 Node.js 的 ECMAScript 模块算法，从 TypeScript 4.7 开始支持。
- `classic`：TypeScript 1.6 之前的算法，新项目不建议使用。
- `bundler`：TypeScript 5.0 新增的选项，表示当前代码会被其他打包器（比如 Webpack、Vite、esbuild、Parcel、rollup、swc）处理，从而放宽加载规则，它要求`module`设为`es2015`或更高版本，详见加入该功能的 PR 说明。

它的默认值与`module`属性有关，如果`module`为`AMD`、`UMD`、`System`或`ES6/ES2015`，默认值为`classic`；如果`module`为`node16`或`nodenext`，默认值为这两个值；其他情况下,默认值为`Node`。

moduleSuffixes

`moduleSuffixes`指定模块的后缀名。

```
{
  "compilerOptions": {
    "moduleSuffixes": [".ios", ".native", ""]
  }
}
```

上面的设置使得 TypeScript 对于语句`import * as foo from "./foo";`，会搜索以下脚本`./foo.ios.ts`、`./foo.native.ts`和`./foo.ts`。

newLine

`newLine`设置换行符为`CRLF`（Windows）还是`LF`（Linux）。

noEmit

`noEmit`设置是否产生编译结果。如果不生成，TypeScript 编译就纯粹作为类型检查了。

noEmitHelpers

`noEmitHelpers`设置在编译结果文件不插入 TypeScript 辅助函数，而是通过外部引入辅助函数来解决，比如 NPM 模块`tslib`。

noEmitOnError

`noEmitOnError`指定一旦编译报错，就不生成编译产物，默认为`false`。

noFallthroughCasesInSwitch

`noFallthroughCasesInSwitch`设置是否对没有`break`语句（或者`return`和`throw`语句）的 switch 分支报错，即`case`代码里面必须有终结语句（比如`break`）。

noImplicitAny

`noImplicitAny`设置当一个表达式没有明确的类型描述、且编译器无法推断出具体类型时，是否允许将它推断为`any`类型。

它是一个布尔值，默认为`true`，即只要推断出`any`类型就报错。

noImplicitReturns

`noImplicitReturns`设置是否要求函数任何情况下都必须返回一个值，即函数必须有`return`语句。

noImplicitThis

`noImplicitThis`设置如果`this`被推断为`any`类型是否报错。

noUnusedLocals

`noUnusedLocals`设置是否允许未使用的局部变量。

noUnusedParameters

`noUnusedParameters`设置是否允许未使用的函数参数。

outDir

`outDir`指定编译产物的存放目录。如果不指定，编译出来的`.js`文件存放在对应的`.ts`文件的相同位置。

outFile

`outFile`设置将所有非模块的全局文件，编译在同一个文件里面。它只有在`module`属性为`None`、`System`、`AMD`时才生效，并且不能用来打包 CommonJS 或 ES6 模块。

paths

`paths`设置模块名和模块路径的映射，也就是 TypeScript 如何导入`require`或`imports`语句加载的模块。

`paths`基于`baseUrl`进行加载，所以必须同时设置后者。

```
{
  "compilerOptions": {
    "baseUrl": "./",
    "paths": {
      "b": ["bar/b"]
    }
  }
}
```

上面示例中，paths 设置的是执行`require('b')`时，即加载的是`./bar/b`。

它还可以使用通配符“*”，表明模块名与模块位置的对应关系。

```
{
  "compilerOptions": {
    "baseUrl": "./",
    "paths": {
      "@bar/*": ["bar/*"]
    }
  }
}
```

preserveConstEnums

`preserveConstEnums`将`const enum`结构保留下来，不替换成常量值。

```
{
  "compilerOptions": {
    "preserveConstEnums": true
  }
}
```

pretty

`pretty`设置美化输出终端的编译信息，默认为`true`。

removeComments

`removeComments`移除 TypeScript 脚本里面的注释，默认为`false`。

resolveJsonModule

`resolveJsonModule`允许 import 命令导入 JSON 文件。

rootDir

`rootDir`设置源码脚本所在的目录，主要跟编译后的脚本结构有关。`rootDir`对应目录下的所有脚本，会成为输出目录里面的顶层脚本。

rootDirs

`rootDirs`把多个不同目录，合并成一个虚拟目录，便于模块定位。

```
{
  "compilerOptions": {
    "rootDirs": ["bar", "foo"]
  }
}
```

上面示例中，`rootDirs`将`bar`和`foo`组成一个虚拟目录。

sourceMap

`sourceMap`设置编译时是否生成 SourceMap 文件。

sourceRoot

`sourceRoot`在 SourceMap 里面设置 TypeScript 源文件的位置。

```
{
  "compilerOptions": {
    "sourceMap": true,
    "sourceRoot": "https://my-website.com/debug/source/"
  }
}
```

strict

`strict`用来打开 TypeScript 的严格检查。它的值是一个布尔值，默认是关闭的。

```
{
  "compilerOptions": {
    "strict": true
  }
}
```

这个设置相当于同时打开以下的一系列设置。

- alwaysStrict
- strictNullChecks
- strictBindCallApply
- strictFunctionTypes
- strictPropertyInitialization
- noImplicitAny
- noImplicitThis
- useUnknownInCatchVariables

打开`strict`的时候，允许单独关闭其中一项。

```
{
  "compilerOptions": {
    "strict": true,
    "alwaysStrict": false
  }
}
```

strictBindCallApply

`strictBindCallApply`设置是否对函数的`call()`、`bind()`、`apply()`这三个方法进行类型检查。

如果不打开`strictBindCallApply`编译选项，编译器不会对以上三个方法进行类型检查，参数类型都是`any`，传入任何参数都不会产生编译错误。

```
function fn(x: string) {
  return parseInt(x);
}

// strictBindCallApply:false
const n = fn.call(undefined, false);
// 以上不报错
```

strictFunctionTypes

`strictFunctionTypes`允许对函数更严格的参数检查。具体来说，如果函数 B 的参数是函数 A 参数的子类型，那么函数 B 不能替代函数 A。

```
function fn(x:string) {
  console.log('Hello, ' + x.toLowerCase());
}

type StringOrNumberFunc = (ns:string|number) => void;

// 打开 strictFunctionTypes，下面代码会报错
let func:StringOrNumberFunc = fn;
```

上面示例中，函数`fn()`的参数是`StringOrNumberFunc`参数的子集，因此`fn`不能替代`StringOrNumberFunc`。

strictNullChecks

不打开`strictNullChecks`的情况下，一个变量不管类型是什么，都可以赋值为`undefined`或`null`。

```
// 不打开 strictNullChecks 的情况
let x:number;

x = undefined; // 不报错
x = null; // 不报错
```

上面示例中，不打开`strictNullChecks`时，变量`x`的类型是`number`，但是赋值为`undefined`或`null`都不会报错。这是为了继承 JavaScript 的设定：当变量没有赋值时，它的值就为`undefined`。

一旦打开`strictNullChecks`，就使用严格类型，禁止变量赋值为`undefined`和`null`，除非变量原本就是这两种类型。它相当于从变量的值里面，排除了`undefined`和`null`。

```
// 打开 strictNullChecks 的情况
let x:number;
```

```
x = undefined; // 报错
x = null; // 报错
```

上面示例中，打开`strictNullChecks`时，变量`x`作为`number`类型，就不能赋值为`undefined`和`null`。

下面是一个例子。

```
// 打开 strickNullChecks 时，类型 A 为 number
// 不打开时，类型 A 为 string
type A = unknown extends {} ? string : number;
```

上面示例中，`{}`代表了 Object 类型，不打开`strictNullChecks`时，它包括了`undefined`和`null`，就相当于包括了所有类型的值，所以这时`unknown`类型可以赋值给`{}`类型，类型`A`就为`string`。打开`strictNullChecks`时，`{}`就排除掉了`undefined`和`null`，这时`unknown`类型就不能赋值给`{}`类型后，类型`A`就为`number`。

最后，`strict`属性包含了`strictNullChecks`，如果打开`strict`属性，就相当于打开了`strictNullChecks`。

strictPropertyInitialization

`strictPropertyInitialization`设置类的实例属性都必须初始化，包括以下几种情况。

- 设为`undefined`类型
- 显式初始化
- 构造函数中赋值

注意，使用该属性的同时，必须打开`strictNullChecks`。

```
// strictPropertyInitialization: true
class User {
  // 报错，属性 username 没有初始化
  username: string;
}

// 解决方法一
class User {
  username = '张三';
}

// 解决方法二
class User {
  username:string|undefined;
}

// 解决方法三
class User {
  username:string;

  constructor(username:string) {
    this.username = username;
```

```
  }
}
// 或者
class User {
  constructor(public username:string) {}
}

// 解决方法四：赋值断言
class User {
  username!:string;

  constructor(username:string) {
    this.initialize(username);
  }

  private initialize(username:string) {
    this.username = username;
  }
}
```

suppressExcessPropertyErrors

`suppressExcessPropertyErrors`关闭对象字面量的多余参数的报错。

target

`target`指定编译出来的 JavaScript 代码的 ECMAScript 版本，比如`es2021`，默认是`es3`。

它可以取以下值。

- es3
- es5
- es6/es2015
- es2016
- es2017
- es2018
- es2019
- es2020
- es2021
- es2022
- esnext

注意，如果编译的目标版本过老，比如`"target": "es3"`，有些语法可能无法编译，`tsc`命令会报错。

traceResolution

`traceResolution`设置编译时，在终端输出模块解析的具体步骤。

```
{
  "compilerOptions": {
    "traceResolution": true
  }
}
```

typeRoots

`typeRoots`设置类型模块所在的目录，默认是`node_modules/@types`，该目录里面的模块会自动加入编译。一旦指定了该属性，就不会再用默认值`node_modules/@types`里面的类型模块。

该属性的值是一个数组，数组的每个成员就是一个目录，它们的路径是相对于`tsconfig.json`位置。

```
{
  "compilerOptions": {
    "typeRoots": ["./typings", "./vendor/types"]
  }
}
```

types

默认情况下，`typeRoots`目录下所有模块都会自动加入编译，如果指定了`types`属性，那么只有其中列出的模块才会自动加入编译。

```
{
  "compilerOptions": {
    "types": ["node", "jest", "express"]
  }
}
```

上面的设置表示，默认情况下，只有`./node_modules/@types/node`、`./node_modules/@types/jest`和`./node_modules/@types/express`会自动加入编译，其他`node_modules/@types/`目录下的模块不会加入编译。

如果`"types": []`，就表示不会自动将所有`@types`模块加入编译。

useDefineForClassFields

`useDefineForClassFields`这个设置针对的是，在类（class）的顶部声明的属性。TypeScript 早先对这一类属性的处理方法，与写入 ES2022 标准的处理方法不一致。这个设置设为`true`，就用来开启 ES2022 的处理方法，设为`false`就是 TypeScript 原有的处理方法。

它的默认值跟`target`属性有关，如果编译目标是`ES2022`或更高，那么`useDefineForClassFields`默认值为`true`，否则为`false`。

useUnknownInCatchVariables

`useUnknownInCatchVariables`设置`catch`语句捕获的`try`抛出的返回值类型，从`any`变成`unknown`。

```
try {
  someExternalFunction();
} catch (err) {
  err; // 类型 any
}
```

上面示例中，默认情况下，`catch`语句的参数`err`类型是`any`，即可以是任何值。

打开`useUnknownInCatchVariables`以后，`err`的类型抛出的错误将是`unknown`类型。这带来的变化就是使用`err`之前，必须缩小它的类型，否则会报错。

```
try {
  someExternalFunction();
} catch (err) {
  if (err instanceof Error) {
    console.log(err.message);
  }
}
```

参考链接

- Strict Property Initialization in TypeScript, Marius Schulz

tsc 命令行编译器

简介

tsc 是 TypeScript 官方的命令行编译器，用来检查代码，并将其编译成 JavaScript 代码。

tsc 默认使用当前目录下的配置文件`tsconfig.json`，但也可以接受独立的命令行参数。命令行参数会覆盖`tsconfig.json`，比如命令行指定了所要编译的文件，那么 tsc 就会忽略`tsconfig.json`的`files`属性。

tsc 的基本用法如下。

```
# 使用 tsconfig.json 的配置
$ tsc

# 只编译 index.ts
$ tsc index.ts

# 编译 src 目录的所有 .ts 文件
$ tsc src/*.ts

# 指定编译配置文件
$ tsc --project tsconfig.production.json

# 只生成类型声明文件，不编译出 JS 文件
$ tsc index.js --declaration --emitDeclarationOnly

# 多个 TS 文件编译成单个 JS 文件
$ tsc app.ts util.ts --target esnext --outfile index.js
```

命令行参数

tsc 的命令行参数，大部分与 tsconfig.json 的属性一一对应。

下面只是按照首字母排序，简单罗列出主要的一些参数，详细解释可以参考《tsconfig.json 配置文件》一章。

`--all`：输出所有可用的参数。

`--allowJs`：允许 TS 脚本加载 JS 模块，编译时将 JS 一起拷贝到输出目录。

`--allowUnreachableCode`：如果 TS 脚本有不可能运行到的代码，不报错。

`--allowUnusedLabels`：如果 TS 脚本有没有用到的标签，不报错。

`--alwaysStrict`：总是在编译产物的头部添加`use strict`。

`--baseUrl`：指定非相对位置的模块定位的基准 URL。

`--build`：启用增量编译。

`--checkJs`：对 JS 脚本进行类型检查。

`--declaration`：为 TS 脚本生成一个类型生成文件。

`--declarationDir`：指定生成的类型声明文件的所在目录。

`--declarationMap`：为`.d.ts`文件生成 SourceMap 文件。

`--diagnostics`：构建后输出编译性能信息。

`--emitBOM`：在编译输出的 UTF-8 文件头部加上 BOM 标志。

`--emitDeclarationOnly`：只编译输出类型声明文件，不输出 JS 文件。

`--esModuleInterop`：更容易使用 import 命令加载 CommonJS 模块。

`--exactOptionalPropertyTypes`：不允许将可选属性设置为`undefined`。

`--experimentalDecorators`：支持早期的装饰器语法。

`--explainFiles`：输出进行编译的文件信息。

`--forceConsistentCasingInFileNames`：文件名大小写敏感，默认打开。

`--help`：输出帮助信息。

`--importHelpers`：从外部库（比如 tslib）输入辅助函数。

`--incremental`：启用增量构建。

`--init`：在当前目录创建一个全新的`tsconfig.json`文件，里面是预设的设置。

`--inlineSourceMap`：SourceMap 信息嵌入 JS 文件，而不是生成独立的`.js.map`文件。

`--inlineSources`：将 TypeScript 源码作为 SourceMap 嵌入编译出来的 JS 文件。

`--isolatedModules`：确保每个模块能够独立编译，不依赖其他输入的模块。

`--jsx`：设置如何处理 JSX 文件。

`--lib`：设置目标环境需要哪些内置库的类型描述。

`--listEmittedFiles`：编译后输出编译产物的文件名。

`--listFiles`：编译过程中，列出读取的文件名。

`--listFilesOnly`：列出编译所要处理的文件，然后停止编译。

`--locale`：指定编译时输出的语言，不影响编译结果。

`--mapRoot`：指定 SourceMap 文件的位置。

`--module`：指定编译生成的模块格式。

`--moduleResolution`：指定如何根据模块名找到模块的位置。

`--moduleSuffixes`：指定模块文件的后缀名。

`--newLine`：指定编译产物的换行符，可以设为`crlf`或者`lf`。

`--noEmit`：不生成编译产物，只进行类型检查。

`--noEmitHelpers`：不在编译产物中加入辅助函数。

`--noEmitOnError`：一旦报错，就停止编译，没有编译产物。

`--noFallthroughCasesInSwitch`：Switch 结构的`case`分支必须有终止语句（比如`break`）。

`--noImplicitAny`：类型推断只要为`any`类型就报错。

`--noImplicitReturns`：函数内部没有显式返回语句（比如`return`）就报错。

`--noImplicitThis`：如果`this`关键字是`any`类型，就报错。

`--noImplicitUseStrict`：编译产生的 JS 文件头部不添加`use strict`语句。

`--noResolve`：不进行模块定位，除非该模块是由命令行传入。

`--noUnusedLocals`：如果有未使用的局部变量就报错。

`--noUnusedParameters`：如果有未使用的函数参数就报错。

`--outDir`：指定编译产物的存放目录。

`--outFile`：所有编译产物打包成一个指定文件。

`--preserveConstEnums`：不将`const enum`结构在生成的代码中，替换成常量。

`--preserveWatchOutput`： watch 模式下不清屏。

`--pretty`：美化显示编译时的终端输出。这是默认值，但是可以关闭`--pretty false`。

`--project`（或者`-p`）：指定编译配置文件，或者该文件所在的目录。

`--removeComments`：编译结果中移除代码注释。

`--resolveJsonModule`：允许加载 JSON 文件。

`--rootDir`：指定加载文件所在的根目录，该目录里面的目录结构会被复制到输出目录。

`--rootDirs`：允许模块定位时，多个目录被当成一个虚拟目录。

`--skipDefaultLibCheck`：跳过 TypeScript 内置类型声明文件的类型检查。

`--skipLibCheck`：跳过`.d.ts`类型声明文件的类型检查。这样可以加快编译速度。

`--showConfig`：终端输出编译配置信息，而不进行配置。

`--sourcemap`：为编译产生的 JS 文件生成 SourceMap 文件（.map 文件）。

`--sourceRoot`：指定 SourceMap 文件里面的 TypeScript 源码根目录位置。

`--strict`：打开 TypeScript 严格检查模式。

`--strictBindCallApply`：bind, call、apply 这三个函数的类型，匹配原始函数。

`--strictFunctionTypes`：如果函数 B 的参数是函数 A 参数的子类型，那么函数 B 不能替代函数 A。

`--strictNullChecks`：对`null`和`undefined`进行严格类型检查。

`--strictPropertyInitialization`：类的属性必须进行初始值，但是允许在构造函数里面赋值。

`--suppressExcessPropertyErrors`：关闭对象字面量的多余参数的报错。

`--target`：指定编译出来的 JS 代码的版本，TypeScript 还会在编译时自动加入对应的库类型声明文件。

`--traceResolution`：编译时在终端输出模块解析（moduleResolution）的具体步骤。

`--typeRoots`：设置类型模块所在的目录，替代默认的`node_modules/@types`。

`--types`：设置`typeRoots`目录下需要包括在编译之中的类型模块。

`--version`：终端输出 tsc 的版本号。

`--watch`（或者`-w`）：进入观察模式，只要文件有修改，就会自动重新编译。